

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

**PHÁT TRIỂN ỨNG DỤNG PHIÊN DỊCH VÀ TẠO PHỤ ĐỀ
SONG NGỮ THỜI GIAN THỰC CHO VIDEO/AUDIO SỬ DỤNG
MACHINE LEARNING**

Ngành: Công nghệ thông tin
Chuyên ngành: Phát triển phần mềm di động

Sinh viên thực hiện:

Họ và tên: Đoàn Nam Sơn

Mã sinh viên: CT060134

Người hướng dẫn:

TS. Mai Đức Thọ

Khoa mật mã – Học viện Kỹ thuật mật mã

Hà Nội - 2026

LỜI CAM ĐOAN

Em xin cam đoan đồ án tốt nghiệp với đề tài “**phát triển ứng dụng phiên dịch và tạo phụ đề song ngữ thời gian thực cho video/audio sử dụng Machine Learning**” là công trình nghiên cứu, thiết kế và triển khai do em thực hiện dưới sự hướng dẫn của TS. Mai Đức Thọ.

Các nội dung trình bày trong đồ án được xây dựng dựa trên quá trình tìm hiểu lý thuyết, phân tích yêu cầu, thiết kế hệ thống, triển khai ứng dụng và thực nghiệm của bản thân em. Những kết quả, hình ảnh, bảng biểu và số liệu thực nghiệm trong báo cáo được tổng hợp từ quá trình xây dựng và kiểm thử hệ thống. Các tài liệu, công trình nghiên cứu và nguồn tham khảo được sử dụng trong đồ án đều được trích dẫn đầy đủ trong phần Tài liệu tham khảo.

Em xin cam đoan không sao chép nội dung của đồ án, luận văn hoặc công trình nghiên cứu khác. Nếu có bất kỳ sai phạm nào liên quan đến tính trung thực của nội dung đồ án, em xin hoàn toàn chịu trách nhiệm trước Học viện và Khoa Công nghệ thông tin.

Hà Nội, ngày 01 tháng 06 năm 2026

Sinh viên thực hiện

Đoàn Nam Sơn

LỜI CẢM ƠN

Lời đầu tiên, em xin được bày tỏ lòng biết ơn sâu sắc nhất đến các thầy, cô giáo trong Ban Giám đốc Học viện Kỹ thuật Mật mã, cùng toàn thể các thầy, cô giáo thuộc Khoa Mật mã và Khoa Công nghệ thông tin. Trong suốt những năm tháng học tập và rèn luyện dưới mái trường, em đã được truyền đạt những kiến thức chuyên môn quý báu, những phương pháp luận nghiên cứu khoa học chuẩn mực, cùng với tinh thần kỷ luật và trách nhiệm cao trong công việc. Đó là nền tảng vững chắc giúp em có đủ tự tin và năng lực để thực hiện đề tài tốt nghiệp này.

Đặc biệt, em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới TS. Mai Đức Thọ , giảng viên Khoa Mật mã – Học viện Kỹ thuật Mật mã , người đã trực tiếp định hướng, dành nhiều thời gian quý báu để tận tình hướng dẫn, chỉ bảo và động viên em trong suốt quá trình nghiên cứu và triển khai đề án. Những lời khuyên chuyên môn định hướng sâu sắc, sự nghiêm túc và tâm huyết của Thầy chính là kim chỉ nam giúp em vượt qua các thách thức kỹ thuật phức tạp để hoàn thiện sản phẩm một cách trọn vẹn nhất.

Cuối cùng, em xin cảm ơn gia đình, bạn bè cùng các bạn sinh viên lớp chuyên ngành đã luôn ở bên cạnh động viên, chia sẻ khó khăn và hỗ trợ em về mọi mặt trong suốt thời gian qua.

Mặc dù đã dành nhiều thời gian, công sức và sự nghiêm túc để hoàn thành báo cáo này, song do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn của bản thân, đề án chắc chắn không tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những ý kiến đóng góp, phê bình quý báu từ các thầy, cô giáo trong Hội đồng chấm đề án tốt nghiệp để nghiên cứu của em ngày một hoàn thiện và có tính ứng dụng cao hơn trong thực tế.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI CAM ĐOAN.....	i
LỜI CẢM ƠN	ii
MỤC LỤC.....	iii
DANH MỤC VIẾT TẮT	v
DANH MỤC HÌNH ẢNH	vii
DANH MỤC BẢNG BIỂU	vii
LỜI NÓI ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN XỬ LÝ TIẾNG NÓI VÀ CƠ SỞ CÔNG NGHỆ	4
1.1. Cơ sở lý thuyết về Xử lý tín hiệu tiếng nói và Ngôn ngữ tự nhiên (NLP)	4
1.1.1. Tổng quan về bài toán Nhận dạng giọng nói tự động (Automatic Speech Recognition - ASR).....	4
1.1.2. Kiến trúc mạng Transformer trong xử lý chuỗi	5
1.1.3. Mô hình nhận dạng tiếng nói OpenAI Whisper.....	6
1.2. Cơ sở lý thuyết về Dịch máy (Machine Translation) và Kỹ thuật sinh phụ đề song ngữ	8
1.2.1. Bài toán Dịch máy nơ-ron (Neural Machine Translation – NMT).....	9
1.2.2. Kỹ thuật chuyển đổi ký tự thành âm vị (G2P) sinh hệ thống phiên âm (IPA/Pinyin)	9
1.3. Cơ sở công nghệ và Nền tảng phát triển hệ thống	10
1.3.1. Công nghệ xử lý AI và Tín hiệu âm thanh (AI Engine Service)	10
1.3.2. Công nghệ phát triển hệ thống máy chủ (Backend API & Worker).....	11
1.3.3. Công nghệ phát triển ứng dụng phía người dùng (Client App)	13
1.4. Kết luận chương 1	13
CHƯƠNG 2. PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG PHỤ ĐỀ AI THỜI GIAN THỰC	15
2.1. Đặc tả yêu cầu và thiết kế kiến trúc vĩ mô của hệ thống	15
2.1.1. Phân tích yêu cầu hệ thống.....	15
2.1.2. Sơ đồ kiến trúc tổng quan hệ thống (High-Level System Architecture)	17

2.1.3. Mô hình hóa chức năng hệ thống qua Biểu đồ Ca sử dụng (Use Case Diagram)	19
2.1.4. Mô hình hóa dữ liệu quan hệ hệ thống (ERD)	21
2.1.5. Quy trình điều phối luồng dữ liệu động của AI Pipeline	29
2.2. Xử lý tín hiệu âm thanh kỹ thuật số và nhận dạng giọng nói (ASR)	31
2.2.1. Quy trình tiền xử lý âm thanh và Phân đoạn giọng nói	31
2.2.2. Định tuyến ngôn ngữ và Cổng kiểm soát độ tin cậy văn bản nguồn	35
2.2.3. Trích xuất chuỗi ký tự nguồn qua các mô hình ASR	40
2.3. Căn chỉnh thời gian cấp từ (Alignment) và phiên dịch ngữ nghĩa (NMT)	42
2.3.1. Thuật toán Căn chỉnh thời gian và Quy hoạch động Monotonic LCS	42
2.3.2. Quy trình dịch thuật song ngữ đa ngữ cảnh bằng mô hình NLLB-200	44
2.4. Tích hợp hạ tầng, nghiệp vụ máy chủ và hiển thị trải nghiệm người dùng	47
2.4.1. Nghiệp vụ máy chủ và điều phối hàng đợi (Application Logic Layer & Message Queue Layer)	47
2.4.2. Thiết kế Giao diện Ứng dụng và tương tác người dùng	55
2.5. Kết luận chương 2	61
CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ	62
3.1. Môi trường và kịch bản thực nghiệm	62
3.1.1. Hạ tầng và môi trường triển khai hệ thống	62
3.1.2. Bộ dữ liệu mẫu và kịch bản thực nghiệm	63
3.2. Tiêu chí và phương pháp đánh giá	63
3.3. Kết quả hiệu năng xử lý	64
3.4. Đánh giá chất lượng nhận dạng văn bản nguồn	65
3.5. Đánh giá chất lượng bản dịch và khả năng sử dụng phụ đề	66
3.6. Minh chứng kết quả đầu ra của hệ thống	69
3.7. Đánh giá thực nghiệm	70
3.8. Vai trò của các giải pháp kỹ thuật trong kết quả thực nghiệm	72
3.9. Kết luận chương 3	73
KẾT LUẬN	73
1. Kết luận chung	73
2. Hướng phát triển tiếp theo	74

DANH MỤC VIẾT TẮT

STT	Ký hiệu từ viết tắt	Chữ viết đầy đủ
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	ASR	Automatic Speech Recognition
4	CER	Character Error Rate
5	WER	Word Error Rate
6	CJK	Chinese, Japanese, Korean
7	ERD	Entity Relationship Diagram
8	G2P	Grapheme-to-Phoneme
9	GPU	Graphics Processing Unit
10	JWT	JSON Web Token
11	NLP	Natural Language Processing
12	NMT	Neural Machine Translation
13	ORM	Object-Relational Mapping
14	SaaS	Software as a Service
15	VAD	Voice Activity Detection

DANH MỤC HÌNH ẢNH

Hình 2.1. Sơ đồ tổng quan kiến trúc hệ thống	18
Hình 2.2. Sơ đồ Use case tổng quát	20
Hình 2.3. Phân rã Use case Quản lý thư viện Media	20
Hình 2.4. Phân rã Use case Quản lý trình phát Media	21
Hình 2.5. Biểu đồ Thực thể Quan hệ Cơ sở dữ liệu.....	22
Hình 2.6: Biểu đồ tuần tự luồng xử lý Media của AI Pipeline	29
Hình 2.7. Minh họa mã nguồn AudioProcessor.....	32
Hình 2.8. Minh họa hàm tách giọng nói khỏi bài hát	33
Hình 2.9. Minh họa mã nguồn thuật toán Greedy Merge	35
Hình 2.10. Minh họa mã nguồn dò ngôn ngữ tự động.....	37
Hình 2.11. Minh họa mã nguồn cơ chế phục hồi lỗi.....	38
Hình 2.12. Minh họa mã nguồn kiểm soát độ tin cậy	40
Hình 2.13. Minh họa mã nguồn trích xuất văn bản với mô hình Whisper	41
Hình 2.14. Minh họa mã nguồn trích xuất văn bản với mô hình SenseVoiceSmall	42
Hình 2.15. Minh họa mã nguồn căn chỉnh thời gian với Qwen3-ForcedAligner...	43
Hình 2.16. Minh họa mã nguồn so khớp chuỗi con.....	44
Hình 2.17. Minh họa mã nguồn cấu hình phiên dịch sử dụng NLLB	47
Hình 2.18. Biểu đồ hoạt động luồng Đăng ký và Xác thực tài khoản	48
Hình 2.19. Biểu đồ tuần tự luồng Đăng ký và Xác thực OTP	49
Hình 2.20. Sơ đồ hoạt động luồng Tải lên và xử lý Media.....	50
Hình 2.21. Biểu đồ tuần tự luồng Upload Media.....	51
Hình 2.22: Biểu đồ hoạt động Tra cứu từ vựng	53
Hình 2.23. Biểu đồ tuần tự luồng các tính năng trong Quản lý trình phát Media ..	54
Hình 2.24. Giao diện Đăng nhập và Xác thực OTP.....	56
Hình 2.25. Giao diện Khởi tạo trải nghiệm.....	57
Hình 2.26. Giao diện Tải lên đa phương tiện.....	58

Hình 2.27. Giao diện Quản lý thư viện có thể theo dõi tiến độ xử lý thời gian thực	59
Hình 2.28. Giao diện Trình phát phụ đề và tương tác tra cứu từ nhanh	60
Hình 3.1. Cấu trúc artifact của một media sau khi xử lý trên MinIO	69
Hình 3.2. Ví dụ cấu trúc một segment trong tệp final.json.....	70

DANH MỤC BẢNG BIỂU

Bảng 2.1. Cấu trúc bảng người dùng (users).....	23
Bảng 2.2. Cấu trúc bảng tài nguyên đa phương tiện (media_items).....	24
Bảng 2.3. Cấu trúc bảng gói cước hạn ngạch (subscriptions).....	25
Bảng 2.4. Cấu trúc bảng sở tay từ vựng (user_vocabularies)	26
Bảng 2.5. Cấu trúc bảng danh mục gói cước (subscription_plans)	27
Bảng 2.6. Cấu trúc bảng biến thể định giá (subscription_plan_variants)	27
Bảng 2.7. Cấu trúc bảng đặt trước hạn ngạch AI (ai_credit_reservations).....	28
Bảng 3.1. Kết quả hiệu năng tổng hợp của hệ thống	64
Bảng 3.2. Kết quả nhận dạng văn bản nguồn.....	65
Bảng 3.3. Tiêu chí đánh giá chất lượng bản dịch.....	66
Bảng 3.4. Minh họa chấm điểm chất lượng dịch thuật	67
Bảng 3.5. Tổng hợp kết quả chấm điểm dịch thuật	68

LỜI NÓI ĐẦU

1. Tính cấp thiết của đề tài

Sự bùng nổ của nội dung video trong kỷ nguyên số đã thay đổi căn bản thói quen tiếp cận thông tin, đặc biệt khi phương tiện này hiện chiếm hơn 80% lưu lượng truy cập internet toàn cầu theo báo cáo của Cisco [1], biến các nền tảng như YouTube, Coursera, Udemy trở thành nguồn tài nguyên học liệu khổng lồ. Đối với người học ngoại ngữ, việc tiếp cận nội dung qua video (Video-based Learning) được chứng minh là hiệu quả hơn so với phương pháp truyền thống nhờ khả năng cung cấp ngữ cảnh trực quan. Các nghiên cứu gần đây (ví dụ trên *ScholarSpace*, 2022) cũng chỉ ra rằng việc sử dụng phụ đề song ngữ (Bilingual Subtitles) giúp người học cải thiện đáng kể khả năng ghi nhớ từ vựng và hiểu nghĩa so với chỉ dùng phụ đề đơn ngữ [2], [3].

Tuy nhiên, thực tế triển khai các hệ thống hỗ trợ học tập qua video hiện nay đang tồn tại nhiều rào cản lớn về chi phí và công nghệ. Rào cản đầu tiên là chi phí và thời gian sản xuất thủ công quá lớn. Theo tiêu chuẩn ngành dịch thuật, để tạo ra phụ đề hoàn chỉnh cho 1 giờ âm thanh, con người cần trung bình từ 4 đến 6 giờ làm việc liên tục (tỷ lệ 1:4 đến 1:6), thậm chí lên tới 10 giờ đối với nội dung phức tạp [4], [5]. Điều này khiến việc sản xuất học liệu song ngữ chất lượng cao trở nên đắt đỏ và chậm chạp, không đáp ứng kịp tốc độ ra mắt nội dung mới.

Bên cạnh vấn đề chi phí, các công cụ tự động hiện có cũng bộc lộ một số hạn chế khi áp dụng vào nhu cầu học ngoại ngữ qua video. Mặc dù YouTube và nhiều nền tảng khác đã hỗ trợ tạo phụ đề tự động, chất lượng phụ đề vẫn có thể bị ảnh hưởng bởi nhiều âm thanh, tốc độ nói, giọng vùng miền hoặc nội dung có nhiều người nói. Các báo cáo đánh giá về công nghệ ASR năm 2024 cho thấy tỷ lệ lỗi từ (Word Error Rate - WER) của một số hệ thống trong môi trường nhiễu có thể lên tới 20-30% [6]. Tuy nhiên, đối với người học ngoại ngữ, độ chính xác nhận dạng văn bản chỉ là một phần của trải nghiệm. Người học còn cần bản dịch dễ hiểu, phụ đề đồng bộ với lời thoại, khả năng theo dõi từng đoạn nội dung và các thông tin hỗ trợ phát âm như IPA cho tiếng Anh hoặc Pinyin cho tiếng Trung.

Từ những phân tích trên, đề tài "Phát triển ứng dụng phiên dịch và tạo phụ đề song ngữ thời gian thực cho video/audio sử dụng Machine Learning" được đề xuất nhằm xây dựng một hệ thống hỗ trợ người học ngoại ngữ tiếp cận nội dung video/audio một cách thuận tiện hơn. Hệ thống ứng dụng các kỹ thuật nhận dạng giọng nói, dịch máy, căn chỉnh thời gian và sinh phiên âm để tự động tạo phụ đề song ngữ gồm văn bản gốc, bản dịch tiếng Việt và thông tin hỗ trợ phát âm. Bên

cạnh đó, hệ thống cũng chú trọng đến yếu tố thời gian thực trong trải nghiệm sử dụng, thông qua việc tổ chức xử lý media theo pipeline, cập nhật tiến độ xử lý và cung cấp kết quả phụ đề theo từng giai đoạn khi có thể.

2. Mục tiêu nghiên cứu của đề tài

Mục tiêu tổng quát: Xây dựng hoàn thiện hệ thống ứng dụng hỗ trợ học ngoại ngữ qua video, có khả năng tự động tạo phụ đề song ngữ (Gốc - Dịch - Phiên âm) đồng bộ theo thời gian thực, ứng dụng các kỹ thuật Học sâu (Deep Learning) tiên tiến về xử lý tiếng nói và ngôn ngữ tự nhiên.

Mục tiêu chi tiết:

Nghiên cứu cơ sở công nghệ: Tìm hiểu các kỹ thuật xử lý tiếng nói và ngôn ngữ tự nhiên liên quan đến bài toán tạo phụ đề song ngữ, bao gồm nhận dạng giọng nói tự động (ASR), căn chỉnh thời gian, dịch máy và chuyển đổi ký tự thành âm vị.

Xây dựng pipeline xử lý media và phụ đề: Thiết kế quy trình xử lý dữ liệu đầu vào từ video/audio, bao gồm chuẩn hóa âm thanh, phân đoạn giọng nói, nhận dạng văn bản nguồn, căn chỉnh thời gian, dịch nội dung và sinh dữ liệu phụ đề có thể sử dụng trên trình phát.

Giải quyết bài toán thời gian thực trong xử lý media: Nghiên cứu và xây dựng cơ chế xử lý bất đồng bộ, chia nhỏ dữ liệu, cập nhật tiến độ và xuất kết quả phụ đề theo từng giai đoạn nhằm giảm thời gian chờ của người dùng khi xử lý các video/audio có thời lượng dài. Trong phạm vi đề tài, yếu tố thời gian thực được hiểu theo hướng tối ưu độ trễ xử lý và phản hồi trạng thái của hệ thống, không phải phiên dịch trực tiếp đồng thời với lời nói đầu vào.

Phát triển ứng dụng người dùng: Xây dựng ứng dụng di động và hệ thống máy chủ hỗ trợ các chức năng chính như tải lên media, quản lý thư viện, xem tiến độ xử lý, phát video/audio với phụ đề song ngữ và tra cứu từ vựng nhanh theo ngữ cảnh.

Đánh giá hệ thống: Đánh giá kết quả của hệ thống dựa trên nhiều khía cạnh, bao gồm chất lượng nhận dạng văn bản nguồn, chất lượng bản dịch, mức độ đồng bộ phụ đề, thời gian xử lý, khả năng tạo artifact đầu ra và mức độ hoàn thành các chức năng chính của ứng dụng.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Đề tài tập trung vào quy trình phát triển hệ thống phần mềm ứng dụng các kỹ thuật xử lý tiếng nói và ngôn ngữ tự nhiên trong bài toán tạo phụ đề song ngữ. Các đối tượng chính bao gồm mô hình nhận dạng giọng nói (OpenAI Whisper, Stable-ts), mô hình dịch máy (NMT), kỹ thuật căn chỉnh thời gian (Aligner), kỹ thuật sinh phiên âm (G2P) và các công nghệ xây dựng hệ thống như React Native, NestJS, FastAPI, FFmpeg, PostgreSQL, MinIO, Redis và BullMQ.

Phạm vi nghiên cứu: Hệ thống tập trung hỗ trợ tốt nhất cho cặp ngôn ngữ Anh - Việt và Trung - Việt (có phiên âm IPA/Pinyin), tiếp nhận dữ liệu đầu vào từ tệp tin cục bộ (mp4, mp3) hoặc đường dẫn trực tuyến YouTube. Hệ thống được triển khai theo mô hình self-hosted trong phạm vi đồ án, phục vụ mục tiêu thử nghiệm, đánh giá và minh họa tính khả thi của ứng dụng.

4. Kết quả dự kiến

Đề tài hướng đến việc xây dựng và đánh giá một hệ thống ứng dụng có khả năng tạo phụ đề song ngữ cho video/audio. Trước hết, hệ thống cần hoàn thành được luồng xử lý end-to-end từ bước người dùng tải lên tệp media hoặc nhập đường dẫn YouTube, hệ thống tiếp nhận và xử lý tác vụ trên máy chủ, cho đến khi ứng dụng di động hiển thị được phụ đề song ngữ trên trình phát.

Về đầu ra dữ liệu, hệ thống cần sinh được phụ đề có cấu trúc gồm văn bản gốc, bản dịch tiếng Việt, thông tin phiên âm và mốc thời gian phục vụ hiển thị đồng bộ với video/audio. Ngoài tệp kết quả cuối cùng, hệ thống cũng hướng đến việc tạo các artifact trung gian theo từng giai đoạn xử lý, chẳng hạn dữ liệu nhận dạng ban đầu, các batch phụ đề đã dịch và tệp phụ đề hoàn chỉnh.

Về chất lượng xử lý, hệ thống sẽ được đánh giá trên một bộ ca kiểm thử đại diện theo nhiều khía cạnh: độ chính xác của văn bản nhận dạng, chất lượng bản dịch tiếng Việt, mức độ đồng bộ giữa phụ đề và lời thoại, khả năng sinh phiên âm phù hợp với ngôn ngữ nguồn và mức độ hoàn chỉnh của dữ liệu đầu ra. Trong đó, WER được sử dụng như một chỉ số tham khảo cho bước nhận dạng văn bản nguồn, còn chất lượng bản dịch và phụ đề được đánh giá bổ sung bằng đối chiếu thủ công theo các tiêu chí về đúng nghĩa, tự nhiên, không mất thông tin và phù hợp ngữ cảnh.

Về hiệu năng và yếu tố thời gian thực, đề tài kỳ vọng hệ thống xử lý được các tệp video/audio trong thời gian đủ ngắn để phù hợp với trải nghiệm sử dụng thực tế. Các chỉ số cần đo bao gồm tổng thời gian xử lý, tỷ lệ thời gian xử lý so với thời lượng media, thời điểm xuất hiện kết quả phụ đề đầu tiên, và tỷ lệ ca xử lý hoàn tất thành công. Đây là cơ sở để đánh giá mức độ đáp ứng của hệ thống đối với mục tiêu tạo phụ đề song ngữ theo hướng giảm thời gian chờ và cung cấp phản hồi tiến độ cho người dùng.

CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN XỬ LÝ TIẾNG NÓI VÀ CƠ SỞ CÔNG NGHỆ

1.1. Cơ sở lý thuyết về Xử lý tín hiệu tiếng nói và Ngôn ngữ tự nhiên (NLP)

Để xây dựng một hệ thống tự động tạo phụ đề song ngữ cho video/audio, cần làm rõ các bài toán công nghệ liên quan đến quá trình chuyển đổi dữ liệu âm thanh thành văn bản, bản dịch và thông tin hỗ trợ phát âm. Các kỹ thuật xử lý tín hiệu tiếng nói và xử lý ngôn ngữ tự nhiên đóng vai trò nền tảng trong toàn bộ quy trình này, từ bước nhận dạng lời nói, căn chỉnh thời gian, dịch nội dung cho đến sinh phiên âm phục vụ người dùng cải thiện khả năng học ngoại ngữ.

Trong phạm vi đề tài, các nội dung lý thuyết được trình bày nhằm giải thích cơ sở hoạt động của những thành phần chính được tích hợp vào hệ thống, bao gồm: bài toán nhận dạng giọng nói tự động (Automatic Speech Recognition - ASR), kiến trúc Transformer, mô hình nhận dạng tiếng nói OpenAI Whisper, bài toán căn chỉnh thời gian cưỡng bức (Forced Alignment), dịch máy nơ-ron (Neural Machine Translation - NMT) và kỹ thuật chuyển đổi ký tự thành âm vị (Grapheme-to-Phoneme - G2P). Đây là các nền tảng cần thiết để phân tích cách hệ thống xử lý dữ liệu đầu vào, tổ chức pipeline AI và tạo ra tệp phụ đề song ngữ có thể sử dụng trong ứng dụng di động.

Bên cạnh việc trình bày nguyên lý chung, chương này cũng đề cập đến cơ sở lựa chọn các công nghệ phát triển phần mềm được sử dụng trong hệ thống, bao gồm dịch vụ xử lý AI, máy chủ backend, hàng đợi tác vụ, lưu trữ đối tượng và ứng dụng phía người dùng. Việc kết hợp giữa các thành phần xử lý AI và kiến trúc phần mềm là cơ sở để triển khai hệ thống ở các chương tiếp theo.

1.1.1. Tổng quan về bài toán Nhận dạng giọng nói tự động (Automatic Speech Recognition - ASR)

Bài toán nhận dạng giọng nói tự động về bản chất là quá trình chuyển đổi một chuỗi tín hiệu âm thanh liên tục theo thời gian thành một chuỗi các ký tự văn bản hoặc từ ngữ tương ứng trong một ngôn ngữ cụ thể [8]. Về mặt toán học, nếu gọi $X = (x_1, x_2, \dots, x_T)$ là chuỗi các vector đặc trưng được trích xuất từ tín hiệu sóng âm đầu vào thông qua các kỹ thuật tiền xử lý (như Mel-frequency cepstral coefficients - MFCCs hoặc Log-mel spectrogram), và $Y = (y_1, y_2, \dots, y_U)$ là chuỗi các thực thể từ ngữ thuộc ngôn ngữ đích, mục tiêu tối thượng của một hệ thống ASR là tìm ra chuỗi văn bản tối ưu $\hat{Y}$ sao cho xác suất có điều kiện sau đây đạt giá trị cực đại:

$$\hat{Y} = \underset{Y}{\operatorname{argmax}} P(Y|X)$$

Trong các kiến trúc học máy truyền thống, biểu thức này thường được phân rã dựa trên định lý Bayes thành sự kết hợp giữa mô hình âm học (Acoustic Model - $P(X|Y)$) và mô hình ngôn ngữ (Language Model - $P(Y)$) [8]. Tuy nhiên, các bước tiên công nghệ hiện đại đã dịch chuyển bài toán sang hướng tiếp cận End-to-End (E2E), tức là sử dụng một mạng nơ-ron sâu tích hợp để tối ưu hóa trực tiếp xác suất hậu nghiệm $P(Y|X)$, giúp giảm thiểu sự tích lũy sai số giữa các module độc lập [9].

Để đánh giá chất lượng và độ chính xác của một mô hình ASR, chỉ số Tỷ lệ lỗi từ (Word Error Rate - WER) được sử dụng rộng rãi nhất như một tiêu chuẩn công nghiệp [10]. WER phản ánh khoảng cách biến đổi Levenshtein (Levenshtein distance) để hiệu chỉnh chuỗi văn bản dự đoán trùng khớp hoàn toàn với chuỗi văn bản gốc chuẩn (Ground Truth/Reference), được định nghĩa bằng công thức toán học:

$$WER = \frac{S + D + I}{N}$$

Trong đó:

- S (Substitutions): Số lượng từ bị nhận dạng sai và bị thay thế bằng một từ khác.
- D (Deletions): Số lượng từ bị bỏ sót hoàn toàn trong chuỗi đầu ra của mô hình.
- I (Insertions): Số lượng từ bị chèn thêm một cách dư thừa so với văn bản gốc.
- N : Tổng số lượng từ có trong chuỗi văn bản gốc chuẩn.

Bên cạnh WER, các nhà phát triển hệ thống hiện đại cũng bắt đầu quan tâm đến chỉ số Tỷ lệ lỗi ngữ nghĩa (Semantic Error Rate - SER) nhằm đánh giá xem các lỗi sai từ có làm thay đổi ý nghĩa bản chất của câu lệnh hay không [11]. Trong các môi trường thực tế ngoài phòng thí nghiệm, hiệu năng của các hệ thống ASR thường bị suy giảm nghiêm trọng bởi các tác nhân gây nhiễu tích hợp (bao gồm tạp âm môi trường, tiếng vang) và sự biến dạng tín hiệu do nén băng thông truyền tải trên hạ tầng mạng [12]. Khi tỷ lệ tín hiệu trên nhiễu (Signal-to-Noise Ratio - SNR) hạ thấp, các đặc trưng âm học cơ bản bị bóp méo, đòi hỏi các mô hình nhận dạng phải có tính bền vững (robustness) cực kỳ cao và được huấn luyện trên các tập dữ liệu đa dạng để tránh hiện tượng sụt giảm độ chính xác [10], [12].

1.1.2. Kiến trúc mạng Transformer trong xử lý chuỗi

Sự ra đời của kiến trúc Transformer bởi Vaswani và các cộng sự vào năm 2017 đã thay đổi căn bản cách thức xử lý các bài toán dạng chuỗi (Sequence-to-Sequence), bao gồm cả văn bản và tín hiệu âm thanh [13]. Khác với các kiến trúc tuần tự cổ điển như Mạng nơ-ron hồi quy (RNN) hay Bộ nhớ dài ngắn hạn (LSTM) vốn xử lý thông tin tuyến tính theo từng bước thời gian, Transformer loại bỏ hoàn

toàn tính tuần tự và dựa hoàn toàn trên cơ chế Tự chú ý (Self-Attention) để tính toán song song tất cả các phần tử trong chuỗi [13].

Cơ chế cốt lõi của Transformer là Scaled Dot-Product Attention, hoạt động dựa trên toán học ma trận thông qua việc ánh xạ ba tập hợp vector bao gồm: Truy vấn (Query - Q), Khóa (Key - K), và Giá trị (Value - V). Biểu thức toán học tổng quát được mô tả như sau:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong đó:

- $Q \in \mathbf{R}^{n \times d_k}, K \in \mathbf{R}^{m \times d_k}, V \in \mathbf{R}^{m \times d_v}$ là các ma trận được chiếu tuyến tính từ dữ liệu đầu vào.
- d_k đại diện cho số chiều (dimension) của vector khóa (Key). Hệ số tỷ lệ $\sqrt{d_k}$ đóng vai trò điều chỉnh kích thước của tích vô hướng, nhằm ngăn chặn việc giá trị này tăng quá lớn khi số chiều cao, khiến hàm *softmax* rơi vào các vùng có độ dốc cực nhỏ (hiện tượng bão hòa độ dốc), gây cản trở quá trình cập nhật trọng số của mạng [13].

Để khai thác tối đa ngữ cảnh phức tạp, Transformer mở rộng thành cơ chế Chú ý đa đầu (Multi-Head Attention), cho phép mô hình đồng thời tập trung vào các không gian biểu diễn con khác nhau ở các vị trí khác nhau:

$$Multihead(Q, K, V) = Concat(head_1, head_2, \dots, head_h)W^O$$

$$\text{Với } head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

Đối với các tác vụ xử lý tiếng nói thời lượng dài, các mạng tuần tự truyền thống (LSTM/RNN) bộc lộ hạn chế lớn do vấn đề triệt tiêu độ dốc (vanishing gradient) và rào cản tính toán tuần tự: trạng thái ẩn h_t bắt buộc phải phụ thuộc vào trạng thái ẩn h_{t-1} trước đó, khiến việc tối ưu hóa song song trên phần cứng GPU quy mô lớn là bất khả thi [13]. Ngược lại, Transformer cho phép đường truyền tín hiệu giữa hai vị trí bất kỳ trong chuỗi dữ liệu luôn có độ dài hằng số $O(1)$ [13]. Ưu điểm vượt trội này giúp mô hình bắt giữ được các phụ thuộc ngữ cảnh khoảng cách xa (Long-range dependencies) một cách toàn vẹn, biến nó thành cấu trúc lý tưởng để trích xuất đặc trưng âm học phức tạp của các file đa phương tiện [13], [14].

1.1.3. Mô hình nhận dạng tiếng nói OpenAI Whisper

OpenAI Whisper là một mô hình học sâu đa nhiệm (Multitask Deep Learning) tiêu biểu được xây dựng dựa trên nền tảng kiến trúc Encoder-Decoder Transformer, thiết kế chuyên biệt cho việc tối ưu hóa các bài toán xử lý giọng nói bao gồm nhận

dạng tiếng nói (ASR), dịch thuật tiếng nói (Speech Translation), nhận diện ngôn ngữ (Language Identification), và phát hiện giọng người (Voice Activity Detection) [7].

Sức mạnh đột phá của Whisper so với các mô hình tiền nhiệm không đến từ việc thay đổi kiến trúc mạng nơ-ron mà nằm ở phương pháp huấn luyện giám sát yếu trên quy mô cực kỳ lớn (Large-scale Weak Supervision) [7]. Mô hình được tối ưu hóa dựa trên \$680.000\$ giờ dữ liệu âm thanh đa ngôn ngữ thu thập từ internet, bao gồm cả dữ liệu chứa nhiều ngoài đời thực và phân đoạn dịch nghĩa trực tiếp [7], [15]. Việc tiếp cận nguồn dữ liệu khổng lồ này giúp Whisper đạt được năng lực tổng quát hóa ấn tượng dưới dạng Zero-shot transfer, cho phép mô hình hoạt động hiệu quả trên nhiều tập dữ liệu khác nhau mà không cần tiến hành tinh chỉnh (fine-tuning) lại, đồng thời giảm thiểu tối đa rủi ro quá khớp (overfitting) [7], [16].

Tuy nhiên, khi đưa vào triển khai trong các hệ thống SaaS thực tế, Whisper bộc lộ hai hạn chế cố hữu:

Hiện tượng ảo giác văn bản (Hallucination): Do bộ mã hóa Decoder của Whisper hoạt động theo cơ chế tự hồi quy (Autoregressive)—tức là dự đoán token tiếp theo dựa trên chuỗi token đã tạo ra trước đó—nên khi gặp các phân đoạn video/audio có khoảng lặng dài (Silence), tập âm môi trường phức tạp không chứa giọng người hoặc nhạc nền, mô hình có xu hướng tự sinh ra các cụm từ lặp lại vô nghĩa hoặc bị kẹt trong một vòng lặp từ vựng cố định [7], [15].

Lỗi trôi dạt thời gian (Timestamp Drift): Cơ chế dự đoán nhãn thời gian mặc định của Whisper được nhúng trực tiếp như các token đặc biệt trong chuỗi đầu ra. Cơ chế này thường chỉ chính xác ở mức độ phân đoạn thô (mỗi 30 giây) và rất dễ bị lệch biên độ thời gian khi gặp phải giọng nói có tốc độ quá nhanh hoặc âm thanh bị ngắt quãng, dẫn đến tình trạng chữ hiển thị trên phụ đề không khớp với lời thoại vật lý [17].

Các mô hình ASR được sử dụng trong hệ thống thực tế: Trong quá trình triển khai, hệ thống không sử dụng duy nhất một mô hình ASR cố định cho mọi trường hợp, mà tổ chức cơ chế định tuyến mô hình theo ngôn ngữ và điều kiện xử lý. Cách tiếp cận này xuất phát từ đặc thù của dữ liệu đầu vào: nội dung tiếng Anh, tiếng Trung hoặc các trường hợp đa ngôn ngữ có thể có yêu cầu khác nhau về tốc độ, độ ổn định và khả năng nhận dạng.

Đối với kênh thoại tiếng Anh, hệ thống ưu tiên sử dụng Distil-Whisper, một biến thể chưng cất từ kiến trúc Whisper, nhằm giảm chi phí suy luận và cải thiện tốc độ xử lý trong khi vẫn giữ được chất lượng nhận dạng phù hợp với nhu cầu tạo phụ đề. Đối với kênh thoại tiếng Trung hoặc các nội dung thuộc nhóm CJK, hệ thống sử dụng SenseVoiceSmall thông qua FunASR runtime, do mô hình này được thiết kế theo

hướng nhận dạng tiếng nói đa ngôn ngữ và có khả năng xử lý tốt hơn với một số đặc trưng của tiếng Trung trong thực nghiệm của đề tài.

1.1.4. Bài toán Căn chỉnh thời gian (Forced Alignment) cấp độ từ

Bài toán Căn chỉnh thời gian cưỡng bức (Forced Alignment) là quy trình tự động xác định chính xác ranh giới thời gian (bao gồm thời điểm bắt đầu và thời điểm kết thúc) của từng đơn vị ngôn ngữ học (âm vị, âm tiết hoặc từ) trong một đoạn băng âm thanh dựa trên một văn bản gốc chuẩn đã có sẵn [18]. Đối với một ứng dụng hỗ trợ tiêu thụ nội dung đa phương tiện và học ngoại ngữ trực quan, việc trích xuất được dấu thời gian ở cấp độ từ (Word-level timestamps) đóng vai trò quan trọng [17]. Nó là một nền tảng kỹ thuật quan trọng để triển khai hiệu ứng hiển thị karaoke chạy chữ đồng bộ, hoặc hỗ trợ tính năng tương tác nhấn vào từng từ trên màn hình để kích hoạt bảng tra cứu từ điển nhanh (Word-level interaction). Nếu thiếu đi bước này, trải nghiệm người dùng sẽ bị giới hạn ở các khối phụ đề tĩnh dạng khối thô kệch.

Trong lịch sử xử lý tiếng nói, các giải pháp truyền thống thường áp dụng mô hình Markov ẩn (Hidden Markov Model - HMM) kết hợp với thuật toán Biến đổi thời gian động (Dynamic Time Warping - DTW) để tính toán đường dẫn căn chỉnh tối ưu trên ma trận chi phí khoảng cách tín hiệu [19], [20]. Đối với các hệ thống hiện đại dựa trên nền tảng Transformer như Whisper, một giải pháp tiên tiến hơn nhiều là trích xuất và phân tích trực tiếp ma trận trọng số chú ý chéo (Cross-Attention Weights) nằm giữa các tầng của bộ Encoder và Decoder [17].

Trọng số chú ý chéo phản ánh một cách trực quan mức độ tương quan động giữa các token từ ngữ đang được giải mã và các khung thời gian âm thanh tương ứng trong không gian Mel-spectrogram [17]. Thư viện mã nguồn mở Stable-ts đã tận dụng triệt để nguyên lý này, phối hợp việc trích xuất ma trận Cross-Attention với các kỹ thuật tối ưu hóa lập trình động (dynamic programming) giúp hiệu chỉnh và làm mịn ranh giới thời gian của từ [17]. Giải pháp này giúp giảm đáng kể hiện tượng trôi lệch timestamp của Whisper gốc, từ đó cải thiện độ chính xác ranh giới ở cấp từ trong nhiều trường hợp thực nghiệm.

1.2. Cơ sở lý thuyết về Dịch máy (Machine Translation) và Kỹ thuật sinh phụ đề song ngữ

Để xây dựng một hệ thống phụ đề song ngữ tự động, hai bài toán NLP tiếp theo cần được giải quyết đồng thời là dịch dịch nghĩa văn bản (Machine Translation) và sinh phiên âm phát âm (Grapheme-to-Phoneme).

1.2.1. Bài toán Dịch máy nơ-ron (Neural Machine Translation – NMT)

Bài toán Dịch máy Nơ-ron (Neural Machine Translation - NMT) ứng dụng các mô hình học sâu cấu trúc chuỗi-sang-chuỗi (Sequence-to-Sequence) để chuyển đổi văn bản từ ngôn ngữ nguồn sang ngôn ngữ đích [21]. Khi thực hiện dịch tự động các cặp ngôn ngữ có độ phân tách lớn như Anh - Việt hoặc Trung - Việt, mô hình phải xử lý các thách thức ngôn ngữ học đặc thù:

Sự đảo ngược cấu trúc ngữ pháp: Tiếng Anh thường sử dụng cụm danh từ có các thành phần bổ ngữ, định ngữ đứng trước danh từ chính (ví dụ: "*a high-performance microservice architecture*"), trong khi cấu trúc tiếng Việt bắt buộc bổ ngữ phải đứng sau ("*một kiến trúc vi dịch vụ hiệu năng cao*"). Điều này đòi hỏi mô hình NMT phải thiết lập cơ chế chú ý toàn cục tốt để tái cấu trúc và đảo trật tự từ một cách tự nhiên [22].

Tính đa nghĩa và ngữ cảnh văn hóa: Tiếng Trung Quốc là ngôn ngữ đơn tiết tính, chứa lượng lớn các từ đồng âm dị nghĩa và các thành ngữ bốn chữ cố định (Thành ngữ). Việc dịch thô từng từ đơn lẻ sẽ phá hủy hoàn toàn ngữ nghĩa, bắt buộc mô hình phải sử dụng các cơ chế Pooling trộn mức độ hoặc Attention phân đoạn để nắm bắt ngữ cảnh rộng của toàn câu trước khi đưa ra văn bản dịch tiếng Việt tương ứng [23], [24].

1.2.2. Kỹ thuật chuyển đổi ký tự thành âm vị (G2P) sinh hệ thống phiên âm (IPA/Pinyin)

Song song với việc dịch nghĩa, kỹ thuật Chuyển đổi ký tự thành âm vị (Grapheme-to-Phoneme – G2P) được tích hợp để giải quyết lỗi hỏng "*người học biết nghĩa từ vựng nhưng phát âm sai*". G2P là quy trình thuật toán chuyển đổi chuỗi các ký tự chữ viết (Graphemes) của một từ thành một chuỗi các ký hiệu biểu diễn âm thanh hay âm vị tương ứng (Phonemes) [25]. Từ đó, hệ thống có thể tự động sinh ra bảng phiên âm quốc tế IPA (International Phonetic Alphabet) cho tiếng Anh và hệ thống Bính âm (Pinyin) kèm thanh điệu cho tiếng Trung [25], [26].

Các mô hình G2P hiện đại áp dụng mạng nơ-ron đa nhiệm (Multitask Sequence-to-Sequence) kết hợp với bộ từ điển phát âm chuẩn làm tri thức tiên nghiệm (Prior Dictionary Knowledge) nhằm hỗ trợ xử lý tốt hơn hiện tượng từ đa âm (Polyphones)—các từ có cách viết giống hệt nhau nhưng phát âm hoàn toàn khác biệt dựa theo ngữ cảnh và từ loại (ví dụ từ "*read*" trong tiếng Anh ở thì hiện tại và quá khứ, hay từ "*行*" trong tiếng Trung có thể đọc là *xíng* hoặc *háng*) [26], [27], [28]. Việc hiển thị đồng thời cả ba lớp dữ liệu (Gốc - Dịch - Phiên âm G2P) trên một trình phát media đồng bộ thời gian thực mang lại một giải pháp công nghệ toàn diện tối ưu cho người học ngoại ngữ.

1.3. Cơ sở công nghệ và Nền tảng phát triển hệ thống

Để hiện thực hóa hệ thống phiên dịch và tạo phụ đề song ngữ cho video/audio, việc lựa chọn công nghệ không chỉ dựa trên mức độ phổ biến của từng framework, mà còn phải phù hợp với đặc thù vận hành của bài toán. Hệ thống cần tiếp nhận tệp đa phương tiện dung lượng lớn, xử lý các tác vụ AI có thời gian thực thi dài, quản lý trạng thái xử lý theo thời gian thực, lưu trữ kết quả phụ đề theo từng giai đoạn và đảm bảo ứng dụng di động có thể truy cập kết quả một cách ổn định.

Từ các yêu cầu trên, hệ thống được thiết kế theo kiến trúc phân tán gồm ba nhóm thành phần chính: lõi xử lý AI, máy chủ backend kết hợp worker xử lý trong nền, và ứng dụng di động phía người dùng. Lõi xử lý AI chịu trách nhiệm chuẩn hóa âm thanh, nhận dạng giọng nói, căn chỉnh thời gian, dịch thuật và sinh dữ liệu phụ đề. Backend API và worker đảm nhiệm các nghiệp vụ xác thực, quản lý media, kiểm soát hạn ngạch, điều phối hàng đợi và cập nhật trạng thái xử lý. Ứng dụng di động chịu trách nhiệm tiếp nhận thao tác người dùng, tải lên media, theo dõi tiến độ và hiển thị phụ đề song ngữ trên trình phát.

1.3.1. Công nghệ xử lý AI và Tín hiệu âm thanh (AI Engine Service)

Ngôn ngữ lập trình Python: Python là ngôn ngữ lập trình cấp cao, hướng đối tượng và đa mục đích, được sử dụng rất rộng rãi trong lĩnh vực Khoa học dữ liệu và Trí tuệ nhân tạo [29]. Sức mạnh cốt lõi của Python nằm ở hệ sinh thái phong phú với các thư viện tính toán hiệu năng cao như PyTorch, Hugging Face, và các nhân xử lý tính toán đồ họa chuyên sâu. Trong hệ thống đề xuất, Python được sử dụng làm ngôn ngữ nền tảng cho lõi xử lý AI (AI Engine), đóng vai trò điều khiển luồng tính toán, tương tác trực tiếp với GPU để vận hành mô hình học sâu OpenAI Whisper và thư viện tối ưu hóa thời gian thực Stable-ts [29]. Việc tách riêng AI Engine bằng Python giúp phân xử lý mô hình được triển khai trong môi trường phù hợp, đồng thời không làm phức tạp tầng backend nghiệp vụ vốn được xây dựng bằng TypeScript.

Framework FastAPI: FastAPI là một framework web mã nguồn mở hiện đại, có hiệu năng cao, chuyên dụng để xây dựng các RESTful API bằng Python [30]. Được xây dựng dựa trên tiêu chuẩn ASGI (Asynchronous Server Gateway Interface), FastAPI tận dụng triệt để sức mạnh của thư viện Starlette (cho các tác vụ web nền tảng) và Pydantic (cho việc kiểm định kiểu dữ liệu). Khả năng hỗ trợ lập trình bất đồng bộ toàn diện (async/await) của FastAPI biến nó thành giải pháp lý tưởng để phục vụ các dịch vụ AI Engine, cho phép hệ thống tiếp nhận và điều phối đồng thời nhiều yêu cầu trích xuất đặc trưng âm thanh mà không gây tắc nghẽn (blocking) luồng xử lý chính [30].

Công cụ FFmpeg: FFmpeg là một giải pháp mã nguồn mở hoàn chỉnh, đa nền tảng, được coi là tiêu chuẩn công nghiệp trong việc ghi, chuyển đổi, transcode và truyền trực tuyến các dữ liệu đa phương tiện (audio/video) [31]. Đối với hệ thống này, FFmpeg hoạt động như hạt nhân của module tiền xử lý tín hiệu. Khi người dùng tải lên một tệp tin video (như định dạng .mp4) hoặc luồng dữ liệu thô từ YouTube, FFmpeg thực hiện nhiệm vụ tách luồng âm thanh (audio extraction), chuẩn hóa tần số lấy mẫu (downsampling về mức 16kHz chuẩn theo yêu cầu đầu vào của Whisper), chuyển đổi kênh âm thanh (stereo sang mono), và hỗ trợ phân mảnh dữ liệu âm thanh phục vụ thuật toán phát hiện giọng người (Voice Activity Detection - VAD) nhằm tối ưu hóa bộ nhớ GPU [31], [32]. Đây là một bước xử lý quan trọng vì chất lượng âm thanh đầu vào ảnh hưởng trực tiếp đến chất lượng văn bản nhận dạng và phụ đề cuối cùng.

1.3.2. Công nghệ phát triển hệ thống máy chủ (Backend API & Worker)

Framework NestJS (TypeScript): NestJS là một framework Node.js tiên bộ được xây dựng dựa trên ngôn ngữ TypeScript, kết hợp nhuần nhuyễn các nguyên lý của Lập trình hướng đối tượng (OOP), Lập trình hàm (FP) và Lập trình phản ứng chức năng (FRP) [33]. Trong hệ thống, backend không chỉ đóng vai trò tiếp nhận request từ ứng dụng di động, mà còn chịu trách nhiệm quản lý tài khoản, thư viện media, hạn ngạch xử lý, cấp URL tải tệp, điều phối hàng đợi và đồng bộ trạng thái xử lý về phía người dùng. Các nhóm chức năng này nếu được tổ chức rời rạc sẽ gây khó khăn trong quá trình mở rộng và bảo trì.

Với kiến trúc module hóa, cơ chế tiêm phụ thuộc và hỗ trợ TypeScript, NestJS giúp tách rõ các lớp controller, service, repository và worker xử lý nền. Điều này phù hợp với cấu trúc của hệ thống, trong đó Backend API xử lý các luồng HTTP đồng bộ như đăng nhập, tải lên media, lấy danh sách thư viện, còn Backend Worker xử lý các tác vụ nền như kiểm định tệp, tải audio từ YouTube, tính thời lượng thật và bàn giao job cho AI Engine. Cách tách biệt này giúp luồng request-response của người dùng không bị chặn bởi các tác vụ xử lý media có thời gian thực thi dài.

Hệ quản trị cơ sở dữ liệu PostgreSQL: PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở có độ tin cậy, độ bền vững và tính toàn vẹn dữ liệu cực kỳ cao [34]. Trong mô hình phần mềm dạng dịch vụ (SaaS) của đề tài, PostgreSQL chịu trách nhiệm lưu trữ các cấu trúc dữ liệu quan hệ chặt chẽ bao gồm: thông tin tài khoản người dùng, cơ chế phân quyền, lịch sử giao dịch, ảnh chụp gói cước người dùng (subscription snapshots) và bảng ghi nhận lịch sử tiêu thụ tài nguyên (usage histories). Cơ chế này đảm bảo dữ liệu hạn ngạch sử dụng luôn ở trạng thái sẵn sàng kiểm toán, tránh hiện tượng thất thoát tài nguyên máy chủ GPU [34].

Hệ thống lưu trữ đối tượng MinIO: MinIO là một hệ thống lưu trữ đối tượng (Object Storage) mã nguồn mở hiệu năng cao, được thiết kế chuyên biệt cho kiến trúc đám mây bản địa (cloud-native) và tương thích hoàn toàn với chuẩn API của Amazon S3 [35].

MinIO được lựa chọn vì bài toán của hệ thống cần một lớp lưu trữ tệp tách biệt, có thể được truy cập thống nhất bởi Backend API, Backend Worker, AI Engine và ứng dụng di động. Backend API cần cấp đường dẫn tải lên cho người dùng, Worker cần đọc hoặc ghi audio sau khi kiểm định, AI Engine cần tải audio đã xác thực để xử lý và ghi lại các artifact phụ đề, còn Mobile App cần truy cập kết quả để hiển thị trên trình phát. Việc sử dụng object storage giúp các thành phần này trao đổi dữ liệu thông qua object key và signed URL thay vì phụ thuộc vào đường dẫn cục bộ của một máy chủ cụ thể.

Trong phạm vi đề án, MinIO cũng là một lựa chọn phù hợp vì có thể triển khai self-hosted nhưng vẫn giữ được mô hình làm việc tương tự các dịch vụ lưu trữ đối tượng phổ biến. Điều này giúp hệ thống chủ động hơn trong quá trình phát triển, thử nghiệm và demo, đồng thời vẫn giữ được kiến trúc có khả năng mở rộng nếu sau này chuyển sang các dịch vụ object storage tương thích.

Hàng đợi phân tán Redis và BullMQ: Redis là một kho lưu trữ cấu trúc dữ liệu trong bộ nhớ (in-memory) tốc độ cao, hoạt động như một hệ thống đệm dữ liệu (cache) và môi giới thông điệp (message broker) [36]. BullMQ là một thư viện quản lý hàng đợi phân tán mạnh mẽ dành cho Node.js dựa trên nền tảng của Redis [37]. Redis được sử dụng làm nền tảng lưu trữ tốc độ cao cho các tác vụ cần phản hồi nhanh, đồng thời làm hạ tầng cho cơ chế hàng đợi xử lý nền thông qua BullMQ. Trong hệ thống tạo phụ đề song ngữ, các tác vụ xử lý media không phù hợp với mô hình request-response đồng bộ, vì một yêu cầu có thể kéo dài qua nhiều bước như kiểm định tệp, tải audio, chuẩn hóa âm thanh, chạy ASR, dịch thuật và xuất artifact phụ đề.

BullMQ được lựa chọn vì bài toán của hệ thống có bản chất là quản lý các background jobs có vòng đời rõ ràng, bao gồm tạo job, retry khi lỗi, backoff, kiểm soát số lượng worker thực thi và theo dõi trạng thái xử lý. Đây là các nhu cầu trực tiếp của pipeline media-processing, đặc biệt khi hệ thống cần tách giai đoạn kiểm định/ingestion khỏi giai đoạn xử lý AI trên GPU. Việc sử dụng BullMQ giúp Backend API có thể phản hồi nhanh cho người dùng sau khi tạo tác vụ, trong khi các Worker tiếp tục xử lý từng job theo khả năng tài nguyên thực tế của hệ thống.

Trong phạm vi đề án, hệ thống không yêu cầu các cơ chế truyền thông điệp quá phức tạp như event streaming quy mô lớn, nhiều consumer độc lập hoặc định tuyến message đa tầng. Do đó, một job queue tích hợp tốt với hệ sinh thái Node.js/NestJS

và có chi phí triển khai thấp như BullMQ là lựa chọn phù hợp. Cách lựa chọn này giúp hệ thống đạt được mục tiêu chính là xử lý tác vụ AI bất đồng bộ, có kiểm soát và dễ theo dõi trạng thái, thay vì làm tăng độ phức tạp vận hành bằng một message platform lớn hơn nhu cầu thực tế.

1.3.3. Công nghệ phát triển ứng dụng phía người dùng (Client App)

Framework React Native: React Native là một framework mã nguồn mở do Meta Platforms (Facebook) phát triển, cho phép xây dựng ứng dụng di động đa nền tảng (iOS và Android) sử dụng ngôn ngữ JavaScript và thư viện React [38]. Một ưu điểm của React Native so với nhiều giải pháp hybrid là khả năng biên dịch mã nguồn thành các thành phần giao diện gốc (native components) trên thiết bị di động, đem lại hiệu năng hiển thị mượt mà đồ họa và tốc độ phản hồi cao [38]. Framework này giúp tối ưu hóa thời gian phát triển ứng dụng di động (Client App), đáp ứng tốt yêu cầu xây dựng giao diện trình phát phụ đề song ngữ và các tương tác chạm từ vệt trong ứng dụng [38].

Giao thức WebSocket và Thư viện Socket.io: WebSocket là một giao thức truyền thông mạng hai chiều toàn phần (full-duplex) qua một kết nối TCP duy nhất, cho phép trao đổi dữ liệu liên tục giữa máy chủ và máy khách với chi phí tiêu đề gói tin (overhead) cực thấp [39]. Socket.io là một thư viện JavaScript cao cấp được xây dựng trên nền giao thức WebSocket, bổ sung các tính năng tự động kết nối lại và quản lý phòng truyền thông tin [40]. Trong bài toán tạo phụ đề, do thời gian xử lý AI trên máy chủ GPU cần một khoảng thời gian nhất định, Socket.io được sử dụng để thiết lập kênh streaming sự kiện. Khi các Background Workers cập nhật trạng thái tác vụ trong cơ sở dữ liệu (QUEUED, PROCESSING, phần trăm tiến độ), sự kiện sẽ được đẩy lập tức đến Client qua socket để cập nhật giao diện người dùng theo thời gian thực một cách mượt mà [39], [40].

1.4. Kết luận chương 1

Chương 1 đã trình bày các cơ sở lý thuyết và công nghệ liên quan đến quá trình xây dựng hệ thống tạo phụ đề song ngữ cho video/audio. Các nội dung về nhận dạng giọng nói tự động, kiến trúc Transformer, mô hình Whisper, căn chỉnh thời gian, dịch máy nơ-ron và chuyển đổi ký tự thành âm vị giúp làm rõ vai trò của từng thành phần trong pipeline xử lý âm thanh, văn bản và phụ đề.

Bên cạnh cơ sở lý thuyết, chương này cũng trình bày các công nghệ được lựa chọn để phát triển hệ thống, bao gồm Python/FastAPI và FFmpeg cho tầng xử lý AI, NestJS, PostgreSQL, MinIO và BullMQ/Redis cho tầng máy chủ, cùng React Native và WebSocket cho tầng ứng dụng di động. Việc lựa chọn các công nghệ

này xuất phát từ yêu cầu xử lý media dung lượng lớn, điều phối tác vụ bất đồng bộ, lưu trữ artifact và cập nhật trạng thái xử lý cho người dùng.

Những nội dung trên là cơ sở để triển khai phân phân tích, thiết kế kiến trúc và xây dựng hệ thống ở chương tiếp theo.

CHƯƠNG 2. PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG PHỤ ĐỀ AI THỜI GIAN THỰC

2.1. Đặc tả yêu cầu và thiết kế kiến trúc vĩ mô của hệ thống

Trước khi tiến hành bóc tách kỹ thuật và hiện thực hóa chi tiết từng cấu trúc thành phần theo mô hình Bottom-Up, việc thiết lập một góc nhìn vĩ mô về toàn bộ hệ thống đóng vai trò định hình phạm vi nền tảng vững chắc cho đồ án. Phần này tập trung vào hai nhiệm vụ trọng tâm: phân tích hệ thống các yêu cầu chức năng, phi chức năng theo tiêu chuẩn kỹ nghệ phần mềm và mô hình hóa bức tranh kiến trúc tổng thể dưới dạng sơ đồ khối năm tầng phân tán. Qua đó, mối tương tác logic giữa các tác nhân cốt lõi và các phân hệ dịch vụ sẽ được làm rõ, tạo mốc neo định hướng toàn diện cho các giai đoạn thiết kế và xây dựng module chuyên sâu ở các phân hệ tiếp theo.

2.1.1. Phân tích yêu cầu hệ thống

Giai đoạn phân tích yêu cầu (Software Requirements Analysis) đóng vai trò chuyển dịch nhu cầu thực tế của người học ngoại ngữ thành các đặc tả kỹ thuật tường minh. Theo tiêu chuẩn IEEE 830, các yêu cầu của hệ thống ứng dụng phiên dịch và tạo phụ đề AI thời gian thực được phân rã thành hai nhóm: Yêu cầu chức năng (Functional Requirements) và Yêu cầu phi chức năng (Non-functional Requirements) [41].

a. Yêu cầu chức năng (Functional Requirements)

Dựa trên kiến trúc tổng quan của ứng dụng dịch thuật và học liệu song ngữ đa thông tin, hệ thống điều phối hai tác nhân chính bao gồm Người dùng (User) và Quản trị viên (Admin) với các phân hệ chức năng cụ thể sau:

Nhóm chức năng Người dùng (User):

- Đăng ký tài khoản mới và đăng nhập vào hệ thống.
- Xác thực mã OTP qua Email để kích hoạt tài khoản hoặc thực hiện khôi phục mật khẩu.
- Quản lý thông tin hồ sơ cá nhân, theo dõi trạng thái gói cước hiện tại cũng như hạn ngạch thời gian xử lý còn lại trong tháng.
- Tải lên các tệp tin âm thanh hoặc video cục bộ từ bộ nhớ của thiết bị cá nhân.
- Nhập tài nguyên đa phương tiện vào thư viện bằng cách thông qua đường dẫn (URL) từ nền tảng YouTube.

- Cấu hình ngôn ngữ nguồn (chỉ nhận dạng văn bản hoặc nhận dạng) trước khi gửi yêu cầu xử lý tệp tin.
- Quản lý thư viện media cá nhân và theo dõi tiến độ, trạng thái xử lý AI của từng tác vụ theo thời gian thực.
- Thực hiện xóa các tệp tin media khỏi thư viện cá nhân dưới dạng xóa mềm.
- Tương tác với trình phát media: phát, tạm dừng, tua nhanh, tua lùi và điều chỉnh linh hoạt tốc độ phát từ 0.5x đến 2.0x.
- Tùy biến hiển thị phụ đề: Chủ động bật/tắt độc lập ba lớp thông tin gồm phụ đề gốc, bản dịch tiếng Việt, và ký hiệu phiên âm ngữ âm (IPA/Pinyin).
- Điều hướng trình phát: Nhấn trực tiếp vào một câu thoại trong danh sách để ép video nhảy nhanh đến mốc thời gian (timestamp) tương ứng.
- Tra từ nhanh: Chạm vào một từ đơn lẻ trên màn hình để kích hoạt popup xem định nghĩa từ điển ngữ cảnh, phiên âm và phát âm mẫu.

Nhóm chức năng Quản trị viên (Admin):

- Đăng nhập vào phân hệ quản trị chuyên biệt của hệ thống.
- Quản lý người dùng: Thay đổi vai trò (User/Admin) hoặc tiến hành khóa/mở khóa tài khoản.
- Giám sát hệ thống: Theo dõi trạng thái vận hành của cụm máy chủ, hàng đợi tác vụ nền và hiệu năng xử lý của cụm dịch vụ AI Engine.
- Quản lý kinh doanh: Cấu hình danh mục gói dịch vụ và thiết lập định giá cho các biến thể gói cước thương mại.

b. Yêu cầu phi chức năng (Non-functional Requirements)

Để đảm bảo tính khả thi khi đưa hệ thống vào vận hành thực tế dưới mô hình dịch vụ SaaS và chịu tải các tác vụ AI nặng, hệ thống bắt buộc phải thỏa mãn các tiêu chuẩn khắt khe về mặt kỹ thuật sau:

Hiệu năng và Độ trễ (Performance & Latency): Máy chủ API trung tâm (NestJS) phải có tốc độ phản hồi các truy vấn HTTP cơ bản trong khoảng thời gian < 500ms. Đối với tác vụ AI chuyển đổi tiếng nói bất đồng bộ, tổng thời gian xử lý pipeline (gồm tách âm thanh FFmpeg + nhận dạng Whisper + dịch thuật + G2P) không được phép vượt quá tỷ lệ thời gian thực 1:0.3 (tức là một video dài 10 phút phải được xử lý hoàn tất hoàn toàn, sinh xong file JSON phụ đề song ngữ trong vòng tối đa 3 phút trên hạ tầng GPU cấu hình self-hosted).

Khả năng mở rộng và Chịu tải (Scalability & Load Capacity): Kiến trúc hệ thống bắt buộc phải tách biệt hoàn toàn giữa luồng xử lý Web API và luồng xử lý AI

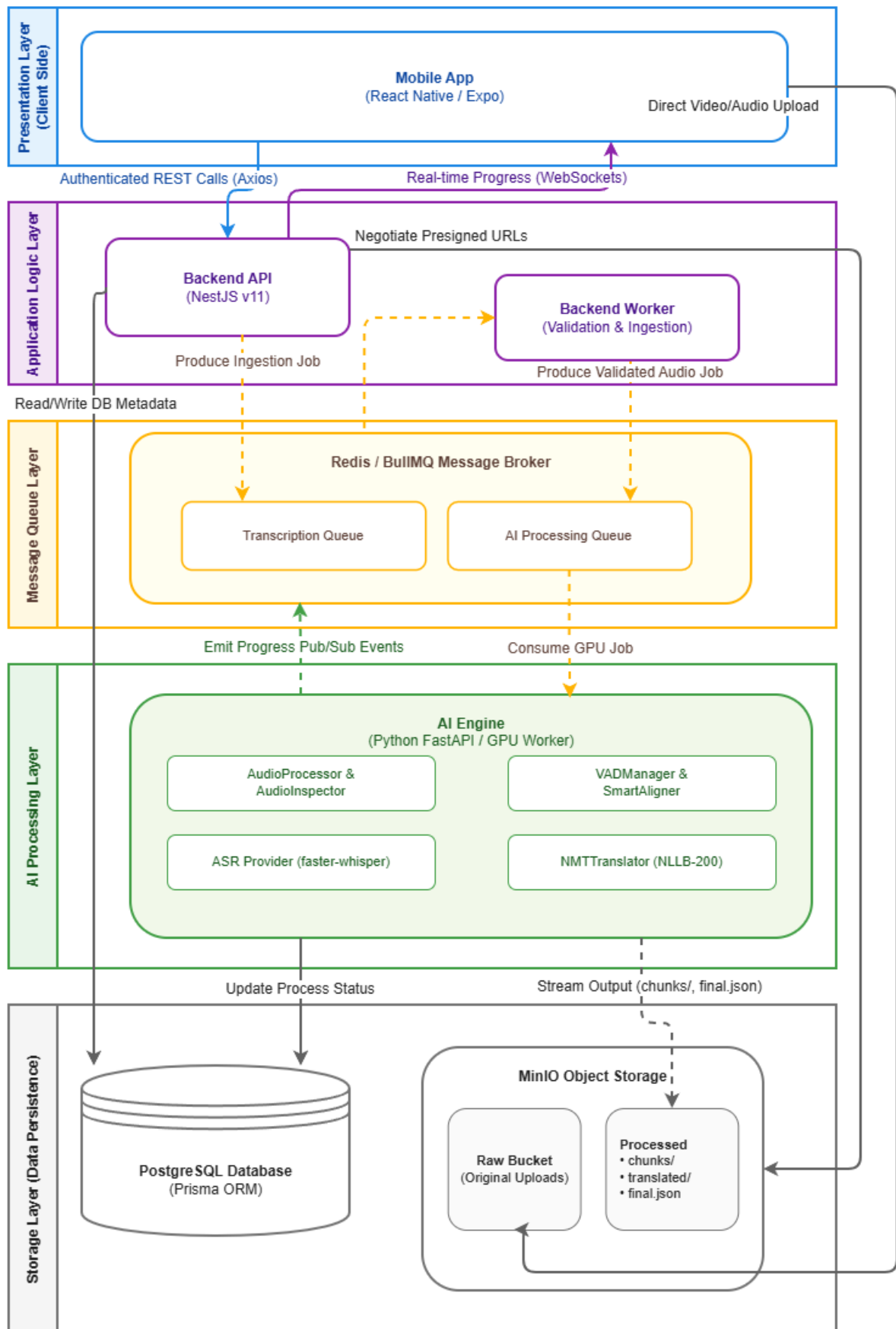
Engine (Microservice architecture). Hệ thống hàng đợi (BullMQ và Redis) phải có khả năng quản lý trạng thái, điều phối ổn định tối thiểu 100 tác vụ xử lý xếp hàng đồng thời mà không gây rơi rớt dữ liệu hoặc làm sập máy chủ backend trung tâm.

Tính sẵn sàng và Tin cậy (Availability & Reliability): Hệ thống phải duy trì chỉ số sẵn sàng tối thiểu đạt mức 99.5% (High Availability). Khi xảy ra sự cố đột ngột ở tầng AI Engine (GPU bị quá nhiệt hoặc hết bộ nhớ VRAM), hệ thống Background Worker của Backend phải có cơ chế bắt lỗi (Exception Handling), tự động cập nhật trạng thái tác vụ về mức FAILED, ghi nhận lý do lỗi cụ thể (failReason) và giải phóng tài nguyên hàng đợi thay vì treo tiến trình vô thời hạn.

Tính bảo mật và Toàn vẹn dữ liệu (Security & Privacy): Mật khẩu người dùng bắt buộc phải được băm bảo mật bằng thuật toán BCrypt kèm chuỗi muối (salt). Giao tiếp API giữa Mobile App và Server phải được bảo vệ qua cơ chế mã thông báo kép (Access/Refresh Token JWT). Tất cả các tệp tin đa phương tiện và file kết quả JSON khi truyền về Client bắt buộc phải đi qua cơ chế URL ký trước (Presigned S3 URL) với thời gian hết hạn nghiêm ngặt.

2.1.2. Sơ đồ kiến trúc tổng quan hệ thống (High-Level System Architecture)

Hệ thống được thiết kế theo kiến trúc phân tán hướng dịch vụ (Distributed Service-Oriented Architecture). Kiến trúc này phân tách độc lập giữa luồng xử lý nghiệp vụ web thông thường và luồng tính toán học sâu chiếm dụng tài nguyên GPU lớn thông qua cơ chế hàng đợi bất đồng bộ. Sơ đồ khối tổng quan của hệ thống được mô tả chi tiết trong Hình 2.1 dưới đây:



Hình 2.1. Sơ đồ tổng quan kiến trúc hệ thống

Kiến trúc tổng quan của hệ thống bao gồm 5 tầng thành phần cốt lõi sau:

Tầng hiển thị (Presentation Layer - Mobile App): Ứng dụng di động được xây dựng trên nền tảng React Native (Expo), chịu trách nhiệm bắt giữ tương tác của người dùng, thực hiện trích xuất âm thanh phía client, điều khiển trình phát media và kết xuất giao diện phụ đề đa lớp. Ứng dụng giao tiếp với máy chủ qua giao thức REST API và nhận các sự kiện cập nhật tiến độ xử lý AI thông qua kết nối WebSocket (Socket.io).

Tầng nghiệp vụ (Application Logic Layer - Backend API & Worker): Vận hành bởi framework NestJS (TypeScript). Thành phần Backend API chịu trách nhiệm xác thực người dùng, kiểm soát hạn ngạch gói cước (SaaS constraints) và ký mã URL bảo mật (Presigned URL) cho việc upload. Thành phần Backend Worker chạy ngầm để tiêu thụ tác vụ, thực hiện tải tài nguyên trực tuyến (YouTube qua yt-dlp) và kiểm định kỹ thuật tệp tin trước khi bàn giao cho tầng AI.

Tầng điều phối hàng đợi (Message Broker & Queue Layer): Vận hành trên nền tảng Redis và thư viện BullMQ. Tầng này thiết lập hai hàng đợi độc lập bao gồm transcription queue (đối với tệp tin thô chờ kiểm định) và ai-processing queue (đối với tệp âm thanh đã chuẩn hóa chờ phân phối tới GPU), đảm bảo hệ thống không bị quá tải khi có nhiều yêu cầu đồng thời.

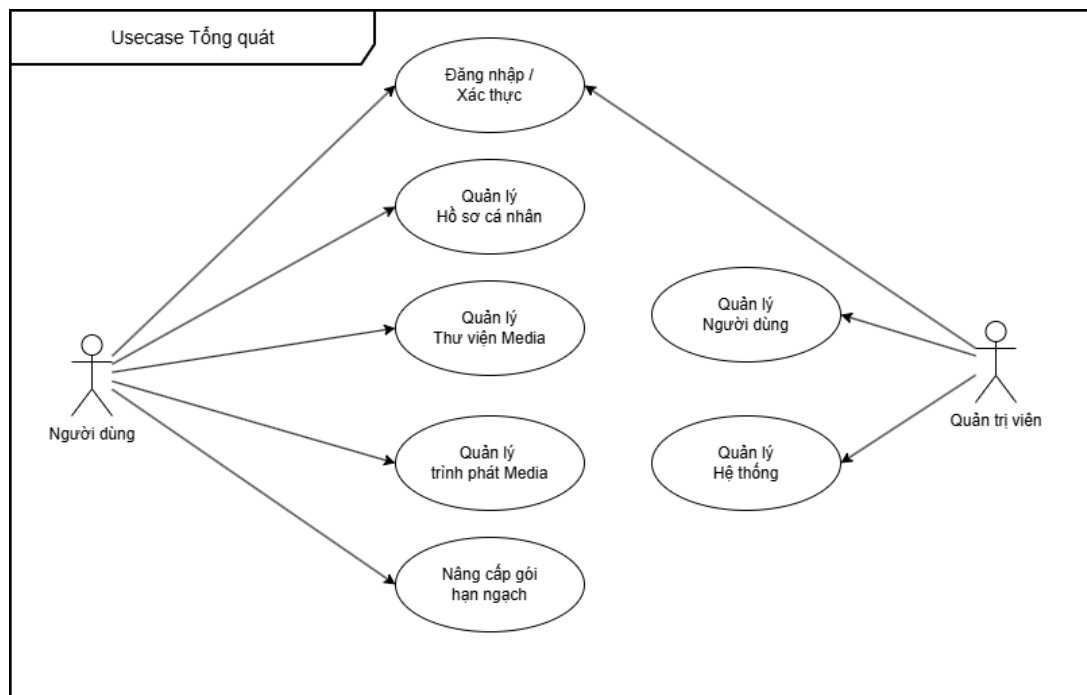
Tầng xử lý trí tuệ nhân tạo (AI Processing Layer - AI Engine): Được xây dựng bằng Python FastAPI, vận hành trực tiếp trên cụm máy chủ GPU. Dịch vụ này tiêu thụ các Job từ hàng đợi, thực thi pipeline AI toàn diện (gồm tiền xử lý FFmpeg, nhận dạng Whisper, căn chỉnh Stable-ts, dịch thuật NMT và phiên âm G2P). AI Engine sẽ xuất trực tiếp dữ liệu phụ đề theo dạng streaming (chunks/, translated_batches/) và tệp tổng hợp cuối cùng (final.json) lên tầng lưu trữ, đồng thời kích hoạt sự kiện Redis Pub/Sub để đồng bộ trạng thái về Mobile.

Tầng lưu trữ (Storage Layer): Gồm hai thành phần hỗ trợ nhau. Hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL (thông qua Prisma ORM) quản lý thông tin người dùng, trạng thái gói cước và nhật ký tiến độ tác vụ. Hệ thống lưu trữ đối tượng MinIO (tương thích chuẩn S3) quản lý toàn bộ tệp đa phương tiện thô và lưu trữ các phân đoạn cấu trúc dữ liệu phụ đề JSON bền vững.

2.1.3. Mô hình hóa chức năng hệ thống qua Biểu đồ Ca sử dụng (Use Case Diagram)

Biểu đồ Use case dùng để biểu diễn tổng quan các chức năng chính của hệ thống thông qua mối quan hệ giữa người dùng (actors) và các trường hợp sử dụng (use cases). Dựa theo các yêu cầu chức năng đã phân tích, hệ thống của ứng dụng

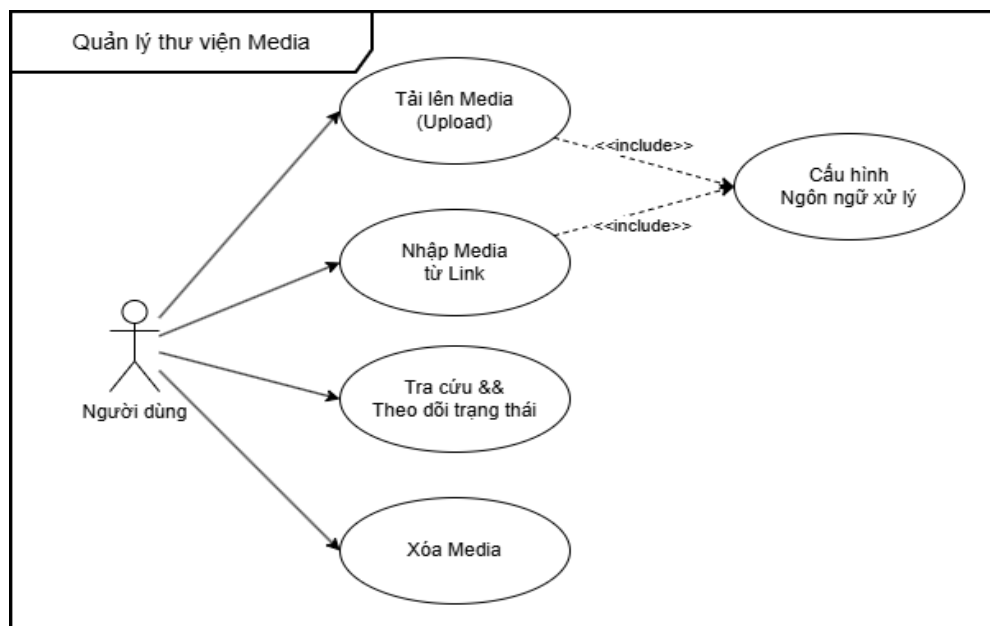
gồm 2 actor chính Người dùng (User) và Quản trị viên (Admin) với các tương tác tổng quát được mô tả trong Hình 2.2:



Hình 2.2. Sơ đồ Use case tổng quát

Phân rã Use case Quản lý thư viện Media:

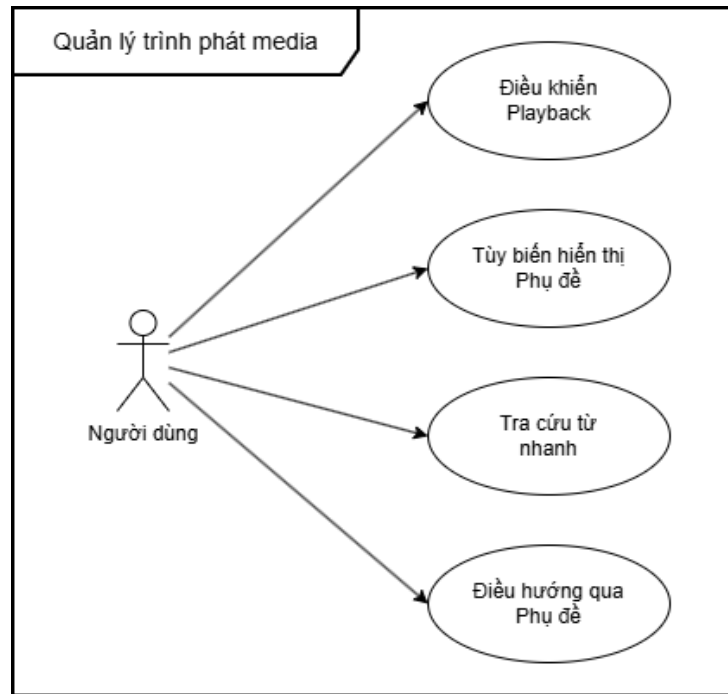
Use case Quản lý thư viện Media gồm 4 chức năng chính gồm: tải lên media từ bộ nhớ của thiết bị di động, nhập đường dẫn media (trong dự án này, sản phẩm của em chỉ tập trung vào xử lý đường dẫn media từ youtube), theo dõi trạng thái xử lý của các media đã tải lên, xóa media đã tải lên.



Hình 2.3. Phân rã Use case Quản lý thư viện Media

Phân rã Use case Quản lý trình phát Media:

Use case Quản lý trình phát Media gồm 4 chức năng chính gồm: điều khiển playback – nhóm chức năng thao tác với media (tạm dừng, tiếp tục, tua, thay đổi tốc độ phát), tùy biến hiển thị 3 lớp phụ đề - ngôn ngữ gốc, phiên âm, bản dịch, tra cứu từ vựng nhanh, điều hướng (tua) media qua phụ đề.



Hình 2.4. Phân rã Use case Quản lý trình phát Media

2.1.4. Mô hình hóa dữ liệu quan hệ hệ thống (ERD)

Cơ sở dữ liệu quan hệ của hệ thống được xây dựng trên thực thể PostgreSQL và ánh xạ thông qua Prisma ORM. Mô hình tập trung điều phối bốn trục nghiệp vụ chính bao gồm: định danh tài khoản, kiểm soát hạn ngạch gói cước SaaS, quản lý vòng đời tệp đa phương tiện và lưu vết sở tay tương tác từ vựng. Kiến trúc thực thể quan hệ tổng thể được mô tả chi tiết thông qua Hình 2.3 dưới đây:

22

Dựa trên sơ đồ thực thể quan hệ ERD, cấu trúc dữ liệu của hệ thống tập trung điều phối bốn trục nghiệp vụ chính thông qua các mối liên kết sau:

Mối quan hệ giữa Người dùng và Gói cước (User - Subscription): Một người dùng có thể có nhiều bản ghi gói cước trong suốt quá trình sử dụng hệ thống, do đó quan hệ giữa User và Subscription được mô hình hóa dưới dạng 1-n thông qua khóa ngoại userId trong bảng subscriptions. Để truy xuất nhanh gói cước hiện tại, bảng users lưu thêm trường currentSubscriptionId tham chiếu đến một bản ghi Subscription đang được áp dụng. Thiết kế này cho phép hệ thống vừa lưu lịch sử thay đổi gói cước, vừa kiểm soát nhanh hạn ngạch và quyền truy cập tính năng của người dùng.

Mối quan hệ giữa Gói dịch vụ, Biến thể gói và Gói cước người dùng (SubscriptionPlan - PlanVariant - Subscription): Một gói dịch vụ có thể có nhiều biến thể định giá khác nhau, ví dụ theo chu kỳ tháng, sáu tháng, năm hoặc trọn đời. Mỗi bản ghi Subscription của người dùng sẽ tham chiếu đến một PlanVariant cụ thể, đồng thời lưu lại các thông tin snapshot như giá, giới hạn thời lượng mỗi tệp, hạn ngạch tháng và số AI credits tại thời điểm đăng ký. Cách thiết kế này giúp hệ thống quản lý linh hoạt danh mục gói cước, đồng thời đảm bảo lịch sử gói của người dùng không bị sai lệch khi cấu hình gói dịch vụ thay đổi về sau.

Mối quan hệ giữa Người dùng và Tài nguyên đa phương tiện (User - MediaItem): Một người dùng có thể sở hữu nhiều tài nguyên đa phương tiện trong thư viện cá nhân, do đó quan hệ giữa User và MediaItem được mô hình hóa dưới dạng 1-n thông qua khóa ngoại userId. Mỗi MediaItem lưu thông tin nguồn dữ liệu, trạng thái xử lý, tiến độ, khóa lưu trữ artifact và các metadata liên quan đến quá trình tạo phụ đề.

Mối quan hệ giữa Người dùng và Sổ tay từ vựng (User - UserVocabulary): Một người dùng có thể lưu nhiều từ vựng hoặc cụm từ trong quá trình tương tác với trình phát phụ đề. Bảng UserVocabulary liên kết người dùng với từ vựng, media, vị trí segment và ngữ cảnh câu nguồn, giúp hệ thống lưu lại kết quả tra cứu theo đúng nội dung mà người dùng đang học.

Cấu trúc bảng người dùng (users):

Bảng này chịu trách nhiệm lưu trữ thông tin định danh tài khoản, trạng thái xác thực và liên kết gói cước thương mại của người dùng.

Bảng 2.1. Cấu trúc bảng người dùng (users)

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Mô tả ý nghĩa
id	String (UUID)	PK	Mã định danh duy nhất của người dùng.

email	String (Unique)		Địa chỉ email tài khoản (Dùng để đăng nhập).
full_name	String		Họ và tên đầy đủ của người dùng.
password_hash	String		Chuỗi mật khẩu đã qua mã hóa băm bảo mật.
email_verified	Boolean		Trạng thái xác thực tài khoản qua mã OTP Email.
current_subscription_id	String (UUID)	FK	Liên kết tới bảng ghi gói cước hoạt động hiện tại.
role	Enum		Vai trò trong hệ thống (USER hoặc ADMIN).

Cấu trúc bảng tài nguyên đa phương tiện (media_items):

Bảng này quản lý toàn bộ vòng đời, nguồn gốc và trạng thái xử lý tiến độ của các tệp đa phương tiện do người dùng nạp vào hệ thống.

Bảng 2.2. Cấu trúc bảng tài nguyên đa phương tiện (media_items)

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Mô tả ý nghĩa
id	String (UUID)	PK	Mã định danh duy nhất của mục media.
user_id	String (UUID)	FK	Liên kết tới người dùng sở hữu (User.id).
title	String		Tiêu đề của video/audio (hoặc tên lấy từ YouTube).

origin_type	Enum		Nguồn gốc tệp (LOCAL hoặc YOUTUBE).
source_url	String		Đường dẫn URL gốc nếu nạp từ YouTube (Có thể null).
audio_s3_key	String		Khóa đường dẫn lưu trữ tệp âm thanh thô trên MinIO.
duration_seconds	Int		Tổng thời lượng của tệp đa phương tiện tính bằng giây.
status	Enum		Trạng thái AI (QUEUED, PROCESSING, COMPLETED, FAILED).
target_language	String		Ngôn ngữ đích cấu hình cho luồng dịch thuật song ngữ.
deleted_at			Đánh dấu mốc thời gian phục vụ cơ chế xóa mềm.

Cấu trúc bảng gói cước người dùng (subscriptions):

Bảng này quản lý thời hạn đóng phí và thực thi việc giám sát tích lũy thời gian xử lý AI để kiểm soát mô hình thương mại SaaS.

Bảng 2.3. Cấu trúc bảng gói cước hạn ngạch (subscriptions)

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Mô tả ý nghĩa
id	String (UUID)	PK	Mã định danh duy nhất của lượt đăng ký gói.
user_id	String (UUID)	FK	Liên kết tới người dùng sở hữu (User.id).

status	Enum		Trạng thái gói (ACTIVE, EXPIRED, CANCELLED)
quota_usage_current_month_seconds	Int		Số giây âm thanh mà người dùng đã tiêu thụ trong tháng.
starts_at	Datetime		Ngày gói cước bắt đầu có hiệu lực.
ends_at	Datetime		Ngày gói cước hết hạn lực lực.

Cấu trúc bảng sổ tay từ vựng (user_vocabularies):

Bảng này ghi nhận dữ liệu tương tác học tập cá nhân hóa, phục vụ tính năng lưu từ vựng kèm ngữ cảnh câu thoại gốc của trình phát.

Bảng 2.4. Cấu trúc bảng sổ tay từ vựng (user_vocabularies)

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Mô tả ý nghĩa
id	String (UUID)	PK	Mã định danh duy nhất của từ vựng được lưu.
user_id	String (UUID)	FK	Liên kết tới người dùng sở hữu (User.id).
word	String		Từ đơn được người dùng nhấn tra cứu và lưu trữ.
context_sentence	String		Câu ngữ cảnh chứa từ đó được trích xuất tự động từ video.
lookup_count	Int		Bộ đếm số lần người dùng lặp lại hành vi tra cứu từ này.
created_at	Datetime		Thời điểm từ vựng được thêm vào sổ tay.

Cấu trúc bảng danh mục gói cước (subscription_plans):

Bảng này định nghĩa các cấu hình phân tầng dịch vụ (ví dụ: gói FREE, gói PREMIUM PRO) và tập hợp các tính năng đi kèm.

Bảng 2.5. Cấu trúc bảng danh mục gói cước (subscription_plans)

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Mô tả ý nghĩa
id	String (UUID)	PK	Mã định danh duy nhất của gói cước.
name	String (UUID)	FK	Tên hiển thị của gói (FREE, PRO).
code	String (Unique)		Mã định danh code hệ thống phục vụ logic code.
Description	String		Mô tả vắn tắt quyền lợi của gói cước.
features	String[]		Mảng danh sách các tính năng được phép truy cập.

Cấu trúc bảng biến thể định giá (subscription_plan_variants):

Bảng này phân tách các lựa chọn thanh toán thương mại cho từng gói cước dựa trên chu kỳ thời gian và định mức hạn ngạch thời gian xử lý AI.

Bảng 2.6. Cấu trúc bảng biến thể định giá (subscription_plan_variants)

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Mô tả ý nghĩa
id	String (UUID)	PK	Mã định danh duy nhất của biến thể.
plan_id	String (UUID)	FK	Liên kết tới danh mục gói chủ quản (SubscriptionPlan.id).

name	String		Tên biến thể (ví dụ: "Gói tháng", "Gói năm").
price	Int		Giá tiền thương mại của biến thể gói.
duration_months	Int		Chu kỳ hiệu lực của biến thể tính bằng tháng.
ai_quota_seconds	Int		Định mức thời lượng xử lý AI cấp cho người dùng.
is_active	Boolean		Trạng thái cho phép hoặc ngừng kinh doanh biến thể này.

Cấu trúc bảng đặt trước hạn ngạch AI (ai_credit_reservations):

Bảng này đóng vai trò tối quan trọng trong luồng xử lý bất đồng bộ. Khi một tác vụ AI bắt đầu đưa vào hàng đợi, hệ thống sẽ tạm khóa (reserve) một lượng số giây tương ứng để tránh người dùng gian lận lạm dụng tài nguyên GPU, và thực hiện hoàn tác (refund) nếu tác vụ gặp sự cố lỗi ngầm.

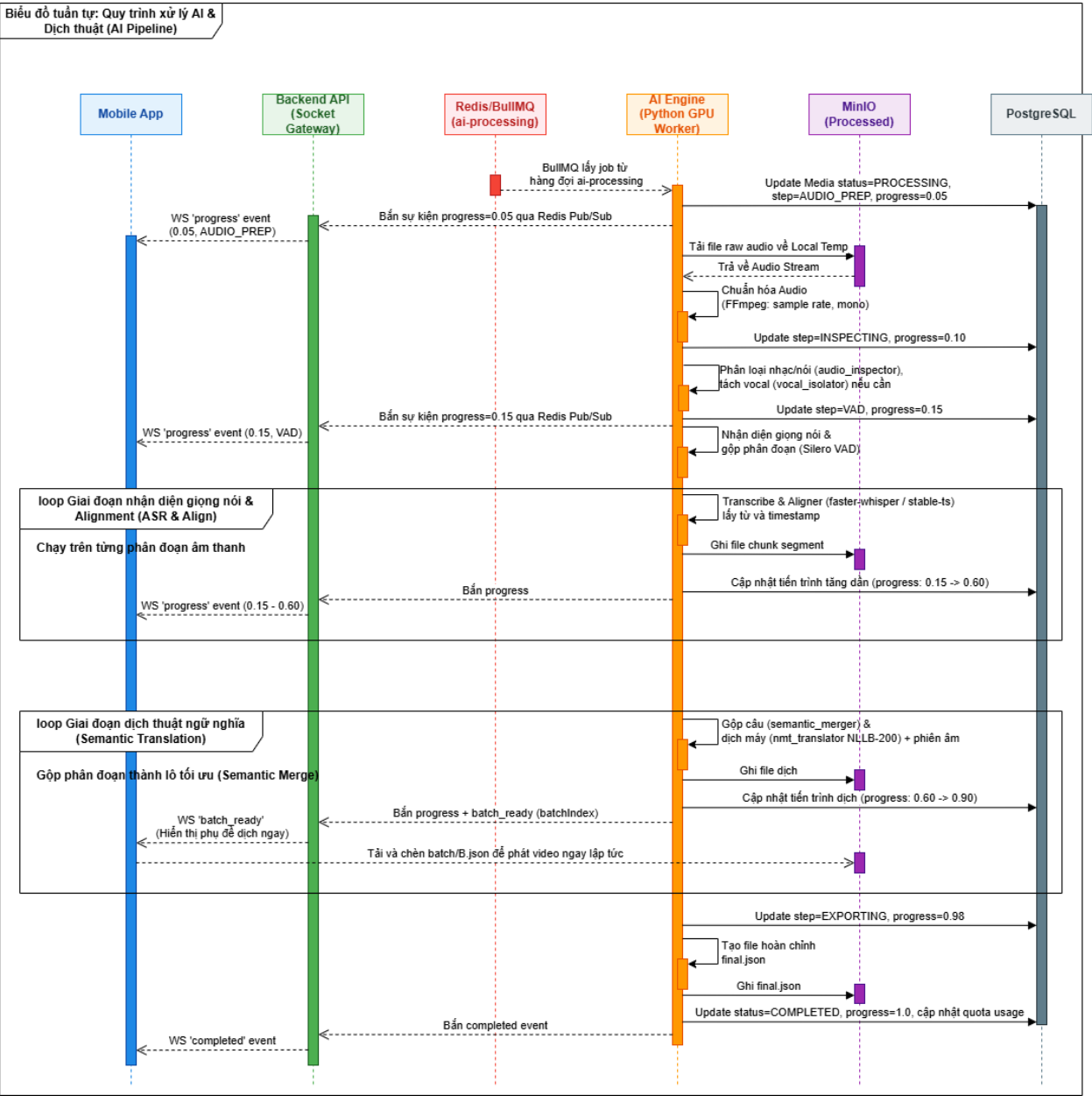
Bảng 2.7. Cấu trúc bảng đặt trước hạn ngạch AI (ai_credit_reservations)

Tên trường (Field)	Kiểu dữ liệu (Type)	Khóa (Key)	Mô tả ý nghĩa
id	String (UUID)	PK	Mã định danh duy nhất của lượt đặt trước.
media_id	String (UUID)	FK	Liên kết tới mục video đang xử lý (MediaItem.id).
subscription_id	String (UUID)	FK	Liên kết tới gói cước chịu trách nhiệm trừ tiền.
reserved_seconds	Int		Số giây thời lượng âm thanh bị tạm khóa tạm thời.
state	Enum		Trạng thái khoản trừ (PENDING,

			CONFIRMED, REFUNDED).
--	--	--	--------------------------

2.1.5. Quy trình điều phối luồng dữ liệu động của AI Pipeline

Đối với bài toán phiên dịch và tạo phụ đề song ngữ, các mô hình học sâu và các module công nghệ không vận hành độc lập, mà được điều phối khép kín thông qua một tiến trình xử lý bất đồng bộ tuần tự (Pipeline Orchestration). Trình tự truyền thông điệp, vòng đời đối tượng và kịch bản tương tác động giữa các phân hệ AI Engine với tầng hạ tầng lưu trữ được đặc tả chi tiết thông qua Biểu đồ tuần tự (Sequence Diagram) dưới đây:



Hình 2.6: Biểu đồ tuần tự luồng xử lý Media của AI Pipeline

Chu trình điều phối dòng chảy dữ liệu trích xuất phụ đề song ngữ đa thông tin bao gồm các giai đoạn tương tác nghiêm ngặt sau:

Giai đoạn Tiếp nhận và Chuẩn hóa Tín hiệu (Ingestion & Preparation): Tiến trình orchestrator trung tâm tiếp nhận tín hiệu kích hoạt Job từ hàng đợi, thực hiện tải tệp đa phương tiện thô từ MinIO Storage thông qua khóa bảo mật S3. Ngay sau đó, lớp xử lý tín hiệu gọi nhân FFmpeg để chuẩn hóa sóng âm về định dạng PCM WAV 16kHz đơn kênh (Mono), đồng thời kích hoạt bộ phân loại phổ âm học AST để xác định đặc trưng môi trường thoại.

Giai đoạn Phân đoạn và Nhận dạng Tiếng nói (Boundary & ASR Extraction): Mảng tín hiệu sạch được nạp vào module VAD để dò tìm ranh giới giọng nói dựa trên mạng mạng học sâu silero-vad. Thuật toán gom cụm tham lam (Greedy Merge) thực hiện tối ưu hóa cửa sổ ngữ cảnh trước khi bàn giao dữ liệu cho bộ điều hướng mô hình. Tại đây, dựa vào kết quả thăm dò ngôn ngữ thô, luồng dữ liệu được định tuyến chính xác đến runtime thực thi của faster-whisper hoặc FunASR để bóc tách chuỗi ký tự nguồn thô.

Giai đoạn Sửa lỗi, Đồng bộ và Biến đổi Ngữ nghĩa (Alignment & Translation): Chuỗi ký tự nguồn đi qua Cổng kiểm soát độ tin cậy văn bản (Trust Gate) nhằm cô lập lỗi ảo giác. Tiếp theo, mô hình Forced Aligner thực thi thuật toán quy hoạch động Monotonic LCS nhằm áp đặt hệ thống nhãn thời gian timestamp chính xác đến từng mili-giây. Ngay khi trục thời gian được bảo toàn, lớp dịch máy nơ-ron thực hiện mã hóa và suy luận song song theo lô qua mô hình NLLB-200 để sinh văn bản dịch tiếng Việt.

Giai đoạn Xuất bản Lũy tiến (Streaming & Publication): Điểm đặc sắc trong logic hành vi động của pipeline là cơ chế xuất bản bất đồng bộ. Hệ thống không cô lập dữ liệu cho đến khi kết thúc toàn bộ tệp tin; ngược lại, các phân đoạn phụ đề hoàn chỉnh (gồm chữ gốc, phiên âm, bản dịch) sẽ được ghi lũy tiến xuống MinIO dưới dạng các mảnh `translated_batches`. Đồng thời, sự kiện Redis Pub/Sub được kích hoạt liên tục để máy chủ API truyền phát streaming về phía ứng dụng di động, hoàn thiện chu trình trải nghiệm thời gian thực của người dùng.

Hệ thống luồng dữ liệu động toàn cục này đóng vai trò là "bản đồ dẫn đường" kiến trúc. Các giải pháp công nghệ, giải thuật tối ưu cấu trúc toán học và mã nguồn cài đặt chi tiết cho từng mắt xích nhận dạng, căn chỉnh và dịch thuật trong sơ đồ tuần tự này sẽ được bóc tách và phân tích chuyên sâu tại các mục 2.2 và 2.3 tiếp theo.

2.2. Xử lý tín hiệu âm thanh kỹ thuật số và nhận dạng giọng nói (ASR)

Quy trình trích xuất và nhận dạng chính xác văn bản từ tín hiệu sóng âm vật lý là mắt xích đầu tiên quyết định đến chất lượng của toàn bộ hệ thống phụ đề song ngữ. Phần này tập trung giải quyết trọn vẹn bài toán tiền xử lý âm học thích ứng, bóc tách phân đoạn hội giọng nói của con người dựa trên mạng Deep Learning chuyên dụng, đồng thời thiết lập tầng kiểm soát độ tin cậy để triệt tiêu các lỗi ảo giác đặc thù của mô hình nhận dạng. Các thuật toán và giải pháp cài đặt tại đây cấu thành trục xử lý cốt lõi nhằm chuẩn bị dữ liệu sạch cho tầng biến đổi ngữ nghĩa phía sau.

2.2.1. Quy trình tiền xử lý âm thanh và Phân đoạn giọng nói

Trong các hệ thống nhận dạng giọng nói tự động (ASR - Automatic Speech Recognition) nói chung và hệ thống phụ đề song ngữ tự động nói riêng, chất lượng tín hiệu âm thanh đầu vào và chiến lược phân cắt dữ liệu (segmentation) đóng vai trò quyết định đến độ chính xác của bản dịch và hiệu năng hệ thống. Quy trình tiền xử lý tín hiệu và phân đoạn giọng nói trong đồ án được thiết kế dưới dạng một đường ống (pipeline) xử lý tuần tự kết hợp bất đồng bộ, bao gồm ba thực thể cốt lõi: *AudioProcessor*, *AudioInspector*, và *VADManager*.

Quy trình này không chỉ chuẩn hóa định dạng sóng âm thanh mà còn phân tích đặc trưng âm học để áp dụng các thuật toán bóc tách giọng nói thích ứng, đồng thời phân đoạn dữ liệu một cách tối ưu nhằm giải quyết triệt để bài toán tràn bộ nhớ VRAM trên các thiết bị tăng tốc phần cứng (GPU) khi xử lý các tệp phương tiện có thời lượng dài.

a. Chuẩn hóa tín hiệu âm thanh kỹ thuật số (**AudioProcessor**):

Tệp phương tiện đầu vào (video hoặc âm thanh) từ phía máy khách thường rất đa dạng về định dạng (MP4, MKV, MP3, AAC, FLAC), tần số lấy mẫu (fsfs từ 8kHz đến 48kHz), cấu hình kênh (Stereo, 5.1 Surround) và chuẩn mã hóa (codec). Để tương thích hoàn toàn với mô hình ASR Whisper, tệp âm thanh bắt buộc phải được đưa về dạng tín hiệu WAV thô (raw PCM 16-bit signed little-endian) với tần số lấy mẫu chuẩn hóa $f_s=16000$ Hz và cấu hình kênh đơn (Mono).

Lớp *AudioProcessor* thực hiện nhiệm vụ này thông qua việc tương tác trực tiếp với bộ công cụ xử lý đa phương tiện **FFmpeg** thông qua thư viện bao bọc *ffmpeg-python*. Quy trình cụ thể như sau:

1 – *Trích xuất thông tin thời lượng (_probe_duration)*: Hệ thống thực hiện một tiến trình con gọi *ffprobe* dưới dạng JSON để phân tích siêu dữ liệu của tệp nguồn. Giá trị thời lượng tệp (*T*) thu được bằng cách ưu tiên đọc trường *duration* trong siêu dữ

liệu chung của định dạng chứa (format). Nếu không tồn tại, hệ thống sẽ duyệt qua các luồng (streams) âm thanh hoặc video để tìm giá trị thời lượng hợp lệ đầu tiên.

2 – Cấu hình luồng FFmpeg (_convert_to_whisper_format):

- Tần số lấy mẫu (Sampling Rate - ar): Thiết lập cố định ở mức 16000 Hz. Quá trình ép tần số (downsampling) từ các tần số cao hơn (như 44.1kHz hoặc 48kHz) được thực hiện thông qua bộ lọc tái mẫu mặc định của FFmpeg (soxr hoặc swresample), giúp loại bỏ các tần số cao không cần thiết cho việc nhận diện giọng nói và giảm đáng kể dung lượng dữ liệu cần xử lý.
- Số kênh âm thanh (Audio Channels - ac): Đưa về dạng đơn kênh (ac=1). Khi chuyển đổi từ Stereo hoặc Surround sang Mono, FFmpeg áp dụng thuật toán trộn kênh (downmixing) tuyến tính theo công thức mặc định:

$$x_{mono}[n] = \frac{x_{left}[n] + x_{right}[n]}{2}$$

Công thức này đảm bảo năng lượng âm thanh từ cả hai kênh trái/phải được bảo toàn và không gây ra hiện tượng triệt tiêu pha tín hiệu nghiêm trọng trong dải tần số thoại (từ 300Hz đến 3.4kHz).

- Chuẩn mã hóa (Codec - acodec): Định dạng đầu ra được cấu hình sử dụng PCM signed 16-bit little-endian (pcm_s16le). Tín hiệu âm thanh lúc này là một chuỗi các số nguyên 16-bit biểu diễn biên độ sóng âm vô hướng, lưu trực tiếp dưới dạng tệp .wav không nén.

Mã nguồn chuẩn hóa âm thanh thực tế:

```
class AudioProcessor:
    SAMPLE_RATE: int = 16000
    CHANNELS: int = 1
    CODEC: str = "pcm_s16le"
    FORMAT: str = "wav"

    def _convert_to_whisper_format(self, input_path: Path, output_path: Path) ->
None:
        try:
            stream = ffmpeg.input(str(input_path))
            stream = ffmpeg.output(
                stream, str(output_path),
                format=self.FORMAT, acodec=self.CODEC,
                ar=self.SAMPLE_RATE, ac=self.CHANNELS,
            )
            ffmpeg.run(stream, overwrite_output=True, capture_stdout=True,
capture_stderr=True, quiet=True)
        except ffmpeg.Error as e:
            if output_path.exists(): output_path.unlink()
            raise AudioProcessingError(f"Failed to convert file: {input_path}",
original_error=e)
```

Hình 2.7. Minh họa mã nguồn AudioProcessor

b. Phân loại âm học (AudioInspector):

Đối với các âm thanh chứa nhạc nền cường độ lớn (music-heavy), tiếng nhạc cụ dễ làm nhiễu kết quả nhận diện giọng nói con người ở bước tiếp theo. Vì vậy hệ thống tích hợp thêm khả năng nhận diện phân loại và cấu hình tách nguồn âm thích ứng.

Phân loại bằng Audio Spectrogram Transformer (AST): AudioInspector sử dụng mô hình phân loại AST (MIT/ast-finetuned-audioset-10-10-0.4593):

Lấy mẫu đa phân đoạn: Trích xuất 3 đoạn mẫu 30 giây tại 10%, 50%, và 90% tổng thời lượng.

Bầu chọn có trọng số: Trọng số [1.0, 2.0, 2.0] (tập trung vào giữa và cuối tệp).

Thoát sớm (Early Exit): Dừng phân tích nếu bất kỳ mẫu nào ngoài intro có `speech_score > 0.3` để kết luận tệp thuộc dạng thoại thông thường (standard).

Bóc tách nguồn âm (VocalIsolator) & Cấu hình VAD thích ứng: Khi tệp được phân loại là music, chế độ Music Mode được kích hoạt:

Tách giọng (Kim_Vocal_2.onnx): Khởi tạo bộ tách nguồn âm qua thư viện audio-separator với thiết lập `output_single_stem="Vocals"`. Tham số này chỉ ghi tệp giọng hát cô lập ra đĩa, giảm 50% băng thông I/O đĩa cứng.

Cấu hình VAD thích ứng:

- Ngưỡng thoại (threshold): 0.4 (bắt được giọng hát nhẹ).
- Khoảng lặng tối thiểu (min_silence_ms): 400 ms (tránh phân đoạn quá vụn).
- Đệm biên thoại (speech_pad_ms): Tăng từ 100 lên 200 ms.

Mã nguồn tách giọng và cấu hình VAD thích ứng thực tế:

```
class VocalIsolator:
    def __init__(self, output_dir: Path | None = None):
        self.output_dir = output_dir or (settings.TEMP_DIR / "vocals")
        self.model_filename = "Kim_Vocal_2.onnx"

    def extract_vocals(self, input_path: Path) -> Path:
        try:
            separator = Separator(
                log_level=logging.INFO, model_file_dir=str(settings.TEMP_DIR /
"models"),
                output_dir=str(self.output_dir), output_format="WAV",
                output_single_stem="Vocals", # Tối ưu hóa ghi đĩa
            )
            separator.load_model(model_filename=self.model_filename)
            output_files = separator.separate(str(input_path))
            return self.output_dir / output_files[0]
        except Exception as e:
            logger.error(f"Fallback to original audio due to: {e}")
            return input_path
```

Hình 2.8. Minh họa hàm tách giọng nói khỏi bài hát

c. Phân đoạn thoại thích ứng dựa trên Silero VAD (VADManager):

Sau khi tín hiệu âm thanh đầu vào được chuẩn hóa ở tầng vật lý, thách thức tiếp theo của phân hệ nhận dạng là phải xác định chính xác các khoảng thời gian chứa giọng nói con người (speech boundaries). Nếu đưa trực tiếp toàn bộ luồng tín hiệu âm thanh liên tục đông dài vào mô hình nhận dạng tiếng nói tự động (ASR), hệ thống sẽ đối mặt với hai rủi ro kỹ thuật lớn:

Chi phí tính toán trên phần cứng tăng mạnh: Do cơ chế Tự chú ý (Self-Attention) của kiến trúc mạng Transformer có chi phí tính toán tỷ lệ thuận với bình phương độ dài chuỗi đầu vào ($O(L^2)$), các phân đoạn âm thanh quá dài, đặc biệt vượt khỏi cửa sổ xử lý khoảng 30 giây của Whisper, sẽ làm kích thước ma trận chú ý tăng nhanh. Điều này làm nhu cầu bộ nhớ GPU tăng theo và có thể dẫn đến lỗi tràn bộ nhớ (Out-Of-Memory).

Mất ngữ cảnh của tầng biến đổi ngữ nghĩa: Ngược lại, nếu phân cắt âm thanh một cách máy móc thành các mảnh quá vụn, tầng dịch máy nơ-ron (NMT) phía sau sẽ bị mất đi ngữ cảnh liên mạch của cấu trúc câu, từ đó làm suy giảm nghiêm trọng độ chính xác của bản dịch nghĩa tiếng Việt.

Để giải quyết mâu thuẫn này, hệ thống xây dựng module VADManager vận hành mô hình học sâu Silero VAD dưới định dạng ONNX (chạy trên runtime ONNX Runtime) để dò tìm các phân đoạn thoại thô một cách thích ứng. Tuy nhiên, để chuyển đổi các nhãn mốc thoại vật lý này thành các cửa sổ ngữ cảnh ngôn ngữ tối ưu, hệ thống cài đặt thuật toán quy hoạch cấu trúc mang tên **Greedy Merge (Gom cụm tham lam)**.

Thuật toán Greedy Merge thực hiện quét tuyến tính qua danh sách các mốc thoại thô R do Silero VAD cung cấp, tính toán liên tục khoảng lặng giữa hai phân đoạn kề nhau ($gap = next_start - current_end$) và thời lượng tích lũy của phân đoạn hiện tại ($duration = current_end - current_start$). Tiêu chí quyết định phân rã biên giới hoặc hợp nhất câu thoại được áp đặt qua hệ thống điều kiện biên nghiêm ngặt sau:

Điều kiện ngắt cứng (Hard Split Window): Nếu tổng thời lượng dự kiến sau khi hợp nhất vượt quá giới hạn an toàn ($potential_duration \geq 20.0s$), thuật toán lập tức cưỡng bách ngắt đoạn và khởi tạo cửa sổ mới để bảo vệ tài nguyên VRAM.

Điều kiện ngắt biên giới tự nhiên (Natural Pause Window): Nếu câu thoại hiện tại đã tích lũy đủ độ dài dài dốc ngữ cảnh ($duration > 5.0s$) và khoảng lặng thoại kế tiếp đủ lớn ($gap > 0.5s$), hệ thống sẽ thực hiện ngắt câu tại khoảng lặng tự nhiên này.

Logic hợp nhất (Greedy Merge logic): Trong tất cả các trường hợp không thỏa mãn hai điều kiện biên trên, thuật toán tiến hành gom cụm bằng cách kéo dài mốc kết thúc của phân đoạn hiện tại ($current_end = next_end$), giữ cho các cụm từ liên quan không bị chặt vụn.

Sau khi khép luồng quét, các phân đoạn thoại hoàn chỉnh được đóng gói thành thực thể VADSegment và gán nhãn phân tầng chất lượng: các phân đoạn có thời lượng tối ưu ($duration \leq 15.0s$) được phân loại vào nhóm HAPPY_CASE để ưu tiên xử lý nhanh; các phân đoạn dài hơn được xếp vào nhóm SPECIAL_CASE để điều phối tài nguyên tính toán thích hợp. Logic hiện thực hóa thuật toán được mô tả chi tiết thông qua đoạn mã nguồn trong Hình 2.8 dưới đây:

Mã nguồn gom cụm phân đoạn thực tế:

```
class VADManager:
    def __init__(self):
        self._model = load_silero_vad(onnx=True)

    def _greedy_merge(self, raw_timestamps: List[dict]) -> List[VADSegment]:
        merged_segments = []
        if not raw_timestamps: return []

        current_start = raw_timestamps[0]['start']
        current_end = raw_timestamps[0]['end']

        for i in range(1, len(raw_timestamps)):
            ts = raw_timestamps[i]
            gap = ts['start'] - current_end
            current_duration = current_end - current_start

            if (ts['end'] - current_start >= 20.0) or (current_duration > 5.0 and gap > 0.5):
                self._add_segment(merged_segments, current_start, current_end)
                current_start = ts['start']
                current_end = ts['end']
            else:
                current_end = ts['end']

        self._add_segment(merged_segments, current_start, current_end)
        return merged_segments
```

Hình 2.9. Minh họa mã nguồn thuật toán Greedy Merge

Kết quả xử lý của chiến lược phân đoạn thích ứng này mang lại một danh sách các phân đoạn âm thanh sạch, cân bằng hoàn hảo giữa tính an toàn của bộ nhớ đồ họa GPU và tính toàn vẹn ngữ nghĩa của mô hình dịch máy.

2.2.2. Định tuyến ngôn ngữ và Cổng kiểm soát độ tin cậy văn bản nguồn

Một trong những thách thức lớn nhất của bài toán nhận dạng giọng nói tự động (ASR) đa ngôn ngữ là hiện tượng nhận dạng sai ngữ hệ chủ đạo khi tệp đa phương tiện chứa các phân đoạn code-switching (chuyển đổi mã ngôn ngữ) hoặc các đoạn mở đầu ngắn bằng tiếng Anh (short English intros) trong một video bài học bằng tiếng Trung. Nếu hệ thống khóa nhầm luồng xử lý vào một mô hình đơn ngữ tiếng Anh, kết quả xuất bản sẽ là các chuỗi văn bản ảo giác (hallucinations) vô nghĩa. Để giải quyết rào cản này, đồ án xây dựng một cơ chế định tuyến động route-aware phối hợp cùng bộ kiểm soát độ tin cậy nghiêm ngặt (Transcript Trust Gate) ở tầng

hậu xử lý. Nếu người dùng cấu hình rõ ngôn ngữ nguồn là tiếng Trung, hệ thống sẽ kích hoạt bộ lọc ưu tiên (ChineseRoutePrior) để cưỡng bức định tuyến trực tiếp vào mô hình đặc thù, bypass hoàn toàn bước dò tìm thô nhằm bảo vệ tính toàn vẹn ngôn ngữ học – cơ chế phục hồi lỗi.

a. Cơ chế lấy mẫu phân đoạn VAD & Whisper Language Probing:

Khi không có thiết lập chỉ định ngôn ngữ nguồn từ người dùng, hệ thống sẽ kích hoạt tính năng dò ngôn ngữ tự động thông qua Whisper.

Giải quyết lỗi Overfitting bằng phương pháp bỏ phiếu đa phân đoạn: Nếu chỉ dò ngôn ngữ trên 30 giây đầu tiên của tệp tin (phương pháp truyền thống), hệ thống rất dễ gặp lỗi quá khớp (overfitting) hoặc định tuyến sai khi tệp tin chứa nhạc dạo đầu (intro) hoặc đoạn giới thiệu ngắn bằng tiếng Anh, trong khi nội dung chính phía sau lại sử dụng ngôn ngữ khác (ví dụ: tiếng Trung).

Để giải quyết bài toán này, WhisperASRProvider thực hiện quy trình dò và bỏ phiếu đa phân đoạn (vote per sampled segment) rải đều trên toàn bộ tệp tin:

Lấy mẫu cách đều: Chọn ra tối đa N phân đoạn phân bố cách đều từ đầu đến cuối danh sách phân đoạn VAD bằng công thức tính chỉ mục:

$$i_k = \text{round}\left(\frac{k \times (M - 1)}{N - 1}\right) \text{ với } k \in [0, N - 1]$$

(Trong đó MM là tổng số phân đoạn VAD, i_k là chỉ mục phân đoạn được chọn).

Bỏ phiếu tích lũy thời lượng: Chạy mô hình Whisper để dò ngôn ngữ nguồn trên từng đoạn mẫu được trích xuất. Kết quả nhận dạng ngôn ngữ của từng phân đoạn sẽ được cộng tích lũy theo thời lượng thực tế của phân đoạn đó:

$$\text{Score}(\text{lang}) = \sum_{j \in S_{\text{lang}}} \Delta t_j$$

(Trong đó S_{lang} là tập hợp các phân đoạn mẫu nhận diện được ngôn ngữ langlang, Δt_j là thời lượng của phân đoạn thứ j).

Quyết định ngôn ngữ nguồn: Ngôn ngữ có tổng thời lượng tích lũy lớn nhất sẽ thắng cuộc. Do tiếng Trung ở phần thân và cuối video có thời lượng thoải vượt trội hơn hẳn đoạn tiếng Anh ngắn ở intro, điểm số tích lũy của tiếng Trung sẽ áp đảo điểm tiếng Anh, đảm bảo định tuyến mô hình chuẩn xác.

Mã nguồn dò ngôn ngữ tự động thực tế:

```
class WhisperASRProvider(ASRProvider):
    def _select_probe_segments(self, segments: list[VADSegment], limit_segments: int)
    -> list[VADSegment]:
        if not segments: return []
        limit = max(1, min(limit_segments, len(segments)))
```

```

if limit >= len(segments): return list(segments[:limit])
if limit == 1: return [segments[0]]

selected_indices: list[int] = []
last_index = len(segments) - 1
for sample_index in range(limit):
    probe_index = round((sample_index * last_index) / (limit - 1))
    if probe_index not in selected_indices:
        selected_indices.append(probe_index)
selected_indices.sort()
return [segments[index] for index in selected_indices]

def _vote_probe_language(self, audio_full: np.ndarray, probe_segments:
list[VADSegment], max_seconds: float | None) -> str | None:
    if not probe_segments: return None
    per_segment_budget = max_seconds / max(1, len(probe_segments)) if max_seconds
else None

    scores: dict[str, float] = {}
    for index, segment in enumerate(probe_segments):
        detected, used_duration = self._probe_segment_language(audio_full,
segment, max_seconds=per_segment_budget)
        if not detected: continue
        scores[detected] = scores.get(detected, 0.0) + used_duration

    if not scores: return None
    winner = max(scores, key=lambda lang: (scores[lang], 1 if
self._is_cjk_probe_language(lang) else 0))
    return winner

```

Hình 2.10. Minh họa mã nguồn dò ngôn ngữ tự động

b. Logic định tuyến thông minh và Thang phục hồi lỗi (Recovery Ladder):

Sau khi xác định được ngôn ngữ nguồn, lớp SmartAligner định tuyến công việc xử lý đến các nhà cung cấp ASR (ASR Providers) chuyên biệt.

Bản đồ định tuyến tối ưu hóa:

Mặc định tiếng Anh: Chuyển hướng qua *distil_whisper_en* (chạy mô hình Distil-Whisper tiếng Anh gọn nhẹ, tốc độ xử lý nhanh gấp nhiều lần Whisper thông thường mà vẫn duy trì chất lượng nhận dạng phù hợp).

Mặc định tiếng Trung: Chuyển hướng qua route đã được chứng nhận chất lượng *sensevoice_small* (sử dụng mô hình *iic/SenseVoiceSmall* qua thư viện FunASR, tối ưu hóa nhận dạng âm tiết tiếng Trung bản địa và hỗ trợ đa ngôn ngữ CJK).

Thang phục hồi lỗi (Recovery/Fallback Ladder):

Hệ thống thiết lập thang phục hồi lỗi đa tầng để tự động xử lý khi một mô hình ASR gặp sự cố phần cứng hoặc lỗi suy luận:

Định tuyến tiếng Trung: *sensevoice_small* – Khôi phục lỗi qua mô hình lớn nhất *whisper_full* (Whisper Large).

Định tuyến tiếng Anh: distil_whisper_en – Khôi phục lỗi qua mô hình trung gian *whisper_turbo*.

Khi có lỗi xảy ra trên luồng xử lý song song, hệ thống tự động gọi hàm `unload_route` để giải phóng GPU và tải động route tiếp theo trong thang khôi phục (*attempted_routes*) nhằm hoàn tất tác vụ thay vì dừng đột ngột.

Mã nguồn khai báo Router phục hồi lỗi thực tế (smart_aligner.py):

```
class SmartAligner:
    def _build_route_configs(self) -> dict[str, ASRRouteConfig]:
        certified_routes = settings.asr_during_asr_certified_routes
        return {
            "distil_whisper_en": ASRRouteConfig(
                route_id="distil_whisper_en", provider_id="whisper",
                model_id=settings.WHISPER_MODEL_DISTIL_EN, display_name="Distil-
Whisper English",
                worker_modes=frozenset({"auto", "turbo_only"}),
                fallback_route_ids=("whisper_turbo",),
                forced_language="en", greedy_only=False, supports_probe=False,
                during_asr_certified="distil_whisper_en" in certified_routes,
            ),
            "whisper_turbo": ASRRouteConfig(
                route_id="whisper_turbo", provider_id="whisper",
                model_id=settings.WHISPER_MODEL_TURBO, display_name="Whisper Turbo",
                worker_modes=frozenset({"auto", "turbo_only"}),
                fallback_route_ids=(),
                greedy_only=True, supports_probe=True,
                during_asr_certified="whisper_turbo" in certified_routes,
            ),
            "whisper_full": ASRRouteConfig(
                route_id="whisper_full", provider_id="whisper",
                model_id=settings.WHISPER_MODEL_FULL, display_name="Whisper Full",
                worker_modes=frozenset({"auto", "full_only"}),
                fallback_route_ids=("whisper_turbo",),
                greedy_only=False, supports_probe=False,
                during_asr_certified="whisper_full" in certified_routes,
            ),
            "sensevoice_small": ASRRouteConfig(
                route_id="sensevoice_small", provider_id="sensevoice",
                model_id=settings.FUNASR_SENSEVOICE_MODEL, display_name="SenseVoice
Small",
                worker_modes=frozenset({"auto", "full_only"}),
                fallback_route_ids=("whisper_full",),
                forced_language="zh", supports_probe=False,
                during_asr_certified="sensevoice_small" in certified_routes,
            ),
        }
```

Hình 2.11. Minh họa mã nguồn cơ chế phục hồi lỗi

c. Cổng kiểm soát độ tin cậy (ChineseTranscriptTrustGate):

Tầng lọc này giám sát đầu ra của tầng ASR theo cơ chế cửa sổ trượt động (sliding window). Nếu phát hiện tỷ lệ lặp lại của một token vượt quá ngưỡng mật độ an toàn, hoặc điểm số log-probability của phân đoạn tụt sâu dưới mức cho phép, Trust Gate sẽ lập tức đánh dấu phân đoạn đó là "Mất tin cậy", kích hoạt phân vùng biên giới

sửa chữa cấp câu (Sentence-level window repair boundaries) để cô lập vùng lỗi, đồng thời gọi module phục hồi lỗi (Hình 2.10).

Cấu trúc phân đoạn động (Deterministic Windows): Hàm *profile_chinese_transcript_windows* phân cắt danh sách câu thành các cửa sổ trượt động dựa trên 4 điều kiện biên:

Khoảng lặng thoại thực tế giữa hai câu (gap >= settings.AI_CHINESE_WINDOW_GAP_SECONDS).

Thời lượng tích lũy của cửa sổ (current_duration >= settings.AI_CHINESE_WINDOW_MAX_SECONDS).

Số lượng câu tối đa cho phép trong cửa sổ (projected_count > settings.AI_CHINESE_WINDOW_MAX_SENTENCES).

Mật độ dịch chuyển pha trộn ngôn ngữ (Code-Switch Density Shift): Tính toán sự thay đổi đột ngột về mật độ Latin/Hán giữa câu đang xét với trung bình các câu đã tích lũy trong cửa sổ hiện tại:

$$\begin{aligned} density_shift = & | sentence.code_switch_density \\ & - current_window.avg_code_switch_density | \\ & \geq Threshold \end{aligned}$$

Đánh giá tín hiệu xác thực (TranscriptTrustSignals): Lớp đánh giá kiểm tra chéo các tín hiệu thống kê âm học và ngữ pháp theo các chỉ số sau:

Tỷ lệ ký tự Trung Hoa (han_ratio, early_han_ratio): Phát hiện tệp tiếng Trung bị nhận dạng nhầm hoàn toàn sang ngôn ngữ khác.

Tỷ lệ từ bính âm giả lập (pinyin_like_ratio): Khi không nghe rõ giọng Trung Quốc, Whisper thường bị ảo giác ghi âm lại bính âm (pinyin không dấu). Hệ thống so khớp các từ Latinh với từ điển bính âm thông dụng. Nếu tỷ lệ này vượt quá ngưỡng cho phép, tệp tin sẽ bị cảnh báo.

Chỉ số lặp từ (repetition_score): Ngăn chặn hiện tượng lặp từ vô hạn (hallucination loop) của Whisper bằng cách kiểm tra tần suất xuất hiện lại của các cặp từ kề nhau.

Mã nguồn thực tế:

```
def profile_chinese_transcript_windows(sentences: list[Sentence]) ->
list[ChineseTranscriptWindow]:
    if not sentences: return []
    sentence_profiles = [_profile_sentence(index, sentence) for index, sentence in
enumerate(sentences)]
    windows: list[ChineseTranscriptWindow] = []
    current: list[ChineseSentenceProfile] = []
    current_gap = 0.0
```

```

def flush() -> None:
    nonlocal current, current_gap
    if not current: return
    windows.append(_build_window(len(windows), current, current_gap))
    current = []
    current_gap = 0.0

for sentence_profile in sentence_profiles:
    if not current:
        current.append(sentence_profile)
        continue
    previous = current[-1]
    gap = max(0.0, sentence_profile.start - previous.end)
    current_duration = current[-1].end - current[0].start
    projected_duration = sentence_profile.end - current[0].start
    projected_count = len(current) + 1
    current_density = _average_code_switch_density(current)
    density_shift = abs(sentence_profile.code_switch_density - current_density)

    should_split = (
        gap >= settings.AI_CHINESE_WINDOW_GAP_SECONDS or
        current_duration >= settings.AI_CHINESE_WINDOW_MAX_SECONDS or
        projected_count > settings.AI_CHINESE_WINDOW_MAX_SENTENCES or
        (len(current) >= settings.AI_CHINESE_WINDOW_MIN_SENTENCES and
        density_shift >= settings.AI_CHINESE_WINDOW_CODE_SWITCH_SHIFT) or
        projected_duration > settings.AI_CHINESE_WINDOW_MAX_SECONDS
    )

    if should_split:
        flush()
        current_gap = gap
        current.append(sentence_profile)

flush()
return windows

```

Hình 2.12. Minh họa mã nguồn kiểm soát độ tin cậy

2.2.3. Trích xuất chuỗi ký tự nguồn qua các mô hình ASR

Sau giai đoạn tiền xử lý âm thanh và định tuyến ngôn ngữ, hệ thống chuyển sang bước nhận dạng giọng nói (ASR) để trích xuất chuỗi ký tự nguồn (text tokens). Tiến trình này được điều phối bởi lớp *SmartAligner*, điều hướng công việc đến các ASR Provider chuyên biệt chạy trên công cụ suy luận tối ưu hóa.

a. Thực thi suy luận trên mô hình Whisper (WhisperASRProvider):

Đối với kênh thoại Tiếng Anh (mô hình chưng cất distil-whisper-large-v3.5), hệ thống sử dụng thư viện faster-whisper chạy trên nền CTranslate2 để tăng tốc tính toán đồ thị trên GPU hỗ trợ CUDA.

Biến đổi đặc trưng: Sóng âm PCM 16kHz của phân đoạn VAD được trích xuất thành tensor Log-Mel Spectrogram 128 kênh (cửa sổ 25ms, bước dịch 10ms).

Giải mã tự hồi quy (Autoregressive): Tensor được mã hóa qua Transformer Encoder, sau đó Transformer Decoder kết hợp cơ chế tự chú ý sinh lần lượt từng token văn bản nguồn.

Mã nguồn thực tế:

```
class WhisperASRProvider(ASRProvider):
    def ensure_loaded(self) -> None:
        if self._model is None:
            # Khởi tạo mô hình Whisper trên GPU CUDA qua CTranslate2
            self._model = WhisperModel(
                self.route.model_id,
                device=settings.DEVICE,
                compute_type=settings.whisper_compute_type
            )
            self._batched = BatchedInferencePipeline(model=self._model)

        def _transcribe_segment(self, audio: np.ndarray, prompt: str, language: str | None) -> dict:
            self.ensure_loaded()
            # Thực thi suy luận phân mảnh song song theo lô (batch_size) kèm dấu thời gian cấp từ
            gen, info = self._batched.transcribe(
                audio,
                batch_size=settings.batch_size,
                beam_size=settings.whisper_beam_size,
                word_timestamps=True,
                initial_prompt=prompt,
                language=language
            )
            segments = list(gen)
            return {"segments": segments, "info": info}
```

Hình 2.13. Minh họa mã nguồn trích xuất văn bản với mô hình Whisper

b. Thực thi suy luận phi tự hồi quy trên mô hình SenseVoice (SenseVoiceASRProvider)

Với các kênh thoại Tiếng Trung hoặc ngữ cảnh đa ngôn ngữ CJK phức tạp, hệ thống chuyển hướng sang mô hình *iic/SenseVoiceSmall* từ Alibaba thông qua FunASR runtime.

Biến đổi đặc trưng: Chuyển đổi tín hiệu thành tensor đặc trưng tần số Fbank 80 chiều.

Suy luận song song phi tự hồi quy (Non-autoregressive): Không giải mã tuần tự từng token như các mô hình autoregressive truyền thống, *SenseVoiceSmall* sử dụng bộ suy luận song song dựa trên CTC (Connectionist Temporal Classification). Kiến trúc này giải phóng toàn bộ chuỗi ký tự nguồn cùng các nhãn ký hiệu sự kiện âm thanh (như nhạc nền, tiếng cười) trong một lượt truyền thẳng duy nhất (single forward pass), giúp cải thiện tốc độ nhận dạng và giảm rủi ro ảo giác (hallucination).

Mã nguồn thực tế:

```

class SenseVoiceASRProvider(ASRProvider):
    def ensure_loaded(self) -> None:
        if self._model is not None:
            return
        from funasr import AutoModel

        self._configure_cache_env()
        self._loaded_model_id = self._resolve_model_id()
        self._model = AutoModel(
            model=self._loaded_model_id,
            hub=settings.FUNASR_MODEL_HUB,
            disable_update=settings.FUNASR_DISABLE_UPDATE_CHECK,
            trust_remote_code=True,
            remote_code="./model.py",
            vad_model=settings.FUNASR_VAD_MODEL,
            vad_kwargs={
                "max_single_segment_time": settings.FUNASR_MAX_SINGLE_SEGMENT_TIME_MS,
            },
            device=self._device_name(),
        )

    def _generate(self, temp_path: Path, *, source_language: str) -> Any:
        language_token = "yue" if source_language == "yue" else "zn"
        return self._model.generate(
            input=str(temp_path), language=language_token, use_itn=True
        )

```

Hình 2.14. Minh họa mã nguồn trích xuất văn bản với mô hình SenseVoiceSmall

2.3. Căn chỉnh thời gian cấp từ (Alignment) và phiên dịch ngữ nghĩa (NMT)

Sau khi thu được chuỗi ký tự nguồn sạch từ phân hệ tiền xử lý âm thanh và trích xuất giọng nói, bài toán đặt ra là phải tiến hành chuyển dịch hệ thống ngữ nghĩa sang ngôn ngữ đích một cách tự nhiên và áp đặt nhãn thời gian timestamp chính xác đến từng mili-giây. Phân hệ này chịu trách nhiệm vận hành bộ hiệu chỉnh quy hoạch động kết hợp mô hình dịch máy nơ-ron chuỗi-sang-chuỗi (string-to-string). Các thuật toán tại đây bảo toàn tính đơn điệu tăng dần của trục thời gian hiển thị phụ đề và giải quyết triệt để sự đảo ngược cấu trúc ngữ pháp giữa các hệ ngôn ngữ phân tách lớn.

2.3.1. Thuật toán Căn chỉnh thời gian và Quy hoạch động Monotonic LCS

Để phục vụ tính năng phát phụ đề hiệu ứng karaoke cấp từ (Word-level timing) và tra cứu từ vựng tương tác mượt mà trên Mobile App, độ chính xác của nhãn thời gian timestamp bắt buộc phải đạt mức mili-giây. Mặc dù Whisper có khả năng sinh timestamp dựa trên cơ chế Attention Weights, độ chính xác này thường bị trôi lệch ở các đoạn nói nhanh hoặc nuốt âm. Hệ thống giải quyết bằng cách tích hợp

thư viện chuyên dụng *stable-ts* để căn chỉnh thô. Đối với các phân đoạn văn bản phức tạp đã đi qua tầng cứu hộ của LLM (vốn bị mất hoàn toàn trục thời gian gốc), AI Engine kích hoạt một giải pháp overlay căn chỉnh thời gian cưỡng bức (Forced Alignment Overlay) sử dụng mô hình ngôn ngữ nhỏ Qwen3-ForcedAligner tối ưu chạy trên CPU để ánh xạ ngược lại sóng âm vật lý.

a. Bộ chuyển đổi đầu ra Qwen3: Mô hình Qwen3-ForcedAligner trả về kết quả lồng nhau phức tạp. Hệ thống sử dụng giải thuật làm phẳng hàng đợi (*_unwrap_alignment_items*) để bóc tách các trường thời gian vật lý và văn bản, sửa lỗi mất nhãn thời gian.

```
class Qwen3ForcedAlignerProvider:
    @staticmethod
    def _coerce_items(result: Any) -> list[ForcedAlignmentUnit]:
        if result is None: return []
        items = Qwen3ForcedAlignerProvider._unwrap_alignment_items(result)
        aligned: list[ForcedAlignmentUnit] = []
        for item in items or []:
            item_dict = Qwen3ForcedAlignerProvider._item_as_mapping(item)
            text = Qwen3ForcedAlignerProvider._first_present(item, item_dict,
("text", "word", "token"), default="")
            start =
Qwen3ForcedAlignerProvider._coerce_float(Qwen3ForcedAlignerProvider._first_present(it
em, item_dict, ("start_time", "start"), default=0.0))
            end =
Qwen3ForcedAlignerProvider._coerce_float(Qwen3ForcedAlignerProvider._first_present(it
em, item_dict, ("end_time", "end"), default=0.0))
            aligned.append(ForcedAlignmentUnit(text=text, start=start, end=end))
        return aligned

    @staticmethod
    def _unwrap_alignment_items(result: Any) -> list[Any]:
        pending = list(result) if isinstance(result, (list, tuple)) else [result]
        flattened: list[Any] = []
        while pending:
            item = pending.pop(0)
            nested = getattr(item, "items", None) or (item.get("items") if
isinstance(item, dict) else None)
            if isinstance(nested, (list, tuple)):
                pending = list(nested) + pending
            else:
                flattened.append(item)
        return flattened
```

Hình 2.15. Minh họa mã nguồn căn chỉnh thời gian với Qwen3-ForcedAligner

b. Thuật toán so khớp chuỗi con đơn điệu (Monotonic LCS Overlay): Nhằm bảo vệ tính toàn vẹn của dòng thời gian không bị đảo ngược (timeline inversion) khi gặp âm vị yếu bị aligner bỏ qua, hệ thống nội suy nhãn thời gian baseline gốc và chạy bộ lọc LCS ở mức độ ký tự.

Nội suy thời gian ký tự baseline:

$$char_start = word \times start + \frac{word_duration \times char_index}{char_count}$$

Quy hoạch động tìm LCS: So khớp chuỗi ký tự baseline với chuỗi ký tự thực tế từ Qwen3.

Kiểm tra tính đơn điệu: Ghi đè nhãn thời gian Qwen3 lên các ký tự khớp nếu thỏa mãn tính tăng dần ($t_{start}[i] \leq t_{start}[i+1]$ và $t_{end}[i] \leq t_{end}[i+1]$). Các ký tự không khớp (âm vị yếu) giữ nguyên thời gian nội suy gốc.

```
def _lcs_character_matches(baseline_text: str, aligned_text: str) -> list[tuple[int, int]]:
    baseline_len, aligned_len = len(baseline_text), len(aligned_text)
    dp = [[0] * (aligned_len + 1) for _ in range(baseline_len + 1)]
    for b_idx in range(baseline_len - 1, -1, -1):
        for a_idx in range(aligned_len - 1, -1, -1):
            if baseline_text[b_idx] == aligned_text[a_idx]:
                dp[b_idx][a_idx] = 1 + dp[b_idx + 1][a_idx + 1]
            else:
                dp[b_idx][a_idx] = max(dp[b_idx + 1][a_idx], dp[b_idx][a_idx + 1])

    matches: list[tuple[int, int]] = []
    b_idx, a_idx = 0, 0
    while b_idx < baseline_len and a_idx < aligned_len:
        if baseline_text[b_idx] == aligned_text[a_idx]:
            matches.append((b_idx, a_idx)); b_idx += 1; a_idx += 1
        elif dp[b_idx + 1][a_idx] >= dp[b_idx][a_idx + 1]: b_idx += 1
        else: a_idx += 1
    return matches

def _matched_char_timings_are_monotonic(aligned_char_timings: list[tuple[float, float]], matched_pairs: list[tuple[int, int]]) -> bool:
    last_start, last_end = None, None
    for _, a_idx in matched_pairs:
        start, end = aligned_char_timings[a_idx]
        if end < start or (last_start is not None and start < last_start - 1e-6) or (last_end is not None and end < last_end - 1e-6):
            return False
        last_start, last_end = start, end
    return True
```

Hình 2.16. Minh họa mã nguồn so khớp chuỗi con

2.3.2. Quy trình dịch thuật song ngữ đa ngữ cảnh bằng mô hình NLLB-200

Trong hệ thống dịch thuật và tạo phụ đề song ngữ tự động, việc dịch văn bản yêu cầu tính chính xác cao về mặt ngữ nghĩa, tốc độ xử lý nhanh để đáp ứng thời gian thực (real-time) và tính ổn định cao về mặt cấu trúc (đặc biệt là ánh xạ 1:1 giữa câu nguồn và câu dịch). Để giải quyết bài toán này, hệ thống tích hợp công cụ dịch thuật máy dịch mạng nơ-ron (NMT) chuyên dụng dựa trên mô hình **NLLB-200-3.3B** của Meta, được tối ưu hóa qua runtime **CTranslate2**.

Ưu điểm kiến trúc và cơ chế tối ưu hóa

Thay vì lựa chọn sử dụng các mô hình ngôn ngữ lớn (LLM) đa năng để dịch thuật (thường gặp các vấn đề về ảo giác cấu trúc JSON, thời gian suy luận dài từ 2-4 giây và tốn tài nguyên GPU), lớp NMTTranslator mang lại các cải tiến kỹ thuật vượt trội:

Ảnh xạ đơn điệu 1:1 (Deterministic 1:1 Mapping): Đảm bảo mỗi câu đầu vào luôn trả về đúng một câu dịch tương ứng. Điều này giảm nguy cơ phát sinh các lỗi lệch chỉ mục phân đoạn hoặc lỗi phân tích định dạng (JSON/Regex parsing errors) khi ghép nối phụ đề.

Độ trễ thấp và Hiệu năng vượt trội: Thời gian suy luận chỉ dao động từ **50-100 ms/câu** trên phần cứng tăng tốc GPU, nhanh gấp 20 đến 40 lần so với việc sử dụng mô hình LLM Instruct (như Qwen2.5-7B).

Tự động thích ứng phần cứng (Compute Type Fallback Chain): Hệ thống thiết lập một chuỗi thử sai cấu hình tự động cho tham số tính toán (compute_type từ float16 → int8 → default) để tối ưu hiệu năng bằng thông và đảm bảo mô hình có thể khởi chạy ổn định trên nhiều kiến trúc GPU khác nhau mà không bị tràn bộ nhớ VRAM.

Quy trình dịch thuật và xử lý Tokenizer

Quá trình dịch thuật diễn ra qua ba bước kỹ thuật chính:

1 – **Mã hóa ký tự (Tokenization):** Sử dụng AutoTokenizer chạy độc lập trên CPU để chuyển câu nguồn thành chuỗi các token đặc trưng, đồng thời gán thẻ ngôn ngữ nguồn tương ứng (ví dụ: eng_Latn, zho_Hans).

2 – **Thiết lập Tiền tố Đích (Target Prefixing):** NLLB-200 yêu cầu gán mã token ngôn ngữ đích làm tiền tố đầu tiên trong chuỗi đầu ra (ví dụ: vie_Latn cho tiếng Việt).

3 – **Suy luận dịch mã (Batch Translation & Decoding):** Chạy phương thức `translate_batch` của CTranslate2 với kích thước chùm (beam size) cấu hình sẵn để suy luận song song theo lô. Sau khi hoàn thành, hệ thống giải mã chuỗi token kết quả trở lại dạng văn bản thô sau khi đã loại bỏ mã định danh ngôn ngữ đích.

Mã nguồn thực tế:

```
# apps/ai-engine/src/core/nmt_translator.py
class NMTTranslator:
    _instance: "NMTTranslator | None" = None

    @classmethod
    def get_instance(cls) -> "NMTTranslator":
        if cls._instance is None:
            cls._instance = cls()
        else:
            cls._instance.ensure_loaded()
        return cls._instance

    def __init__(self) -> None:
```

```

preferred: str = settings.NMT_COMPUTE_TYPE
# Thiết lập thang dự phòng compute_type nhằm ngăn chặn lỗi cấu hình phần cứng
fallback_chain: list[str] = [preferred, "float16", "int8", "default"]
seen: set[str] = set()
self.translator: ctranslate2.Translator | None = None
actual_compute_type: str = preferred

for ct in fallback_chain:
    if ct in seen: continue
    seen.add(ct)
    try:
        # Khởi tạo bộ dịch CTranslate2 trên GPU chỉ định
        self.translator = ctranslate2.Translator(
            str(settings.NMT_MODEL_DIR),
            device=settings.DEVICE,
            device_index=settings.DEVICE_INDEX,
            compute_type=ct,
        )
        actual_compute_type = ct
        break
    except ValueError as exc:
        logger.warning(f"NMTTranslator: compute_type '{ct}' rejected, trying
next...")

if self.translator is None:
    raise RuntimeError("NMTTranslator: no supported compute_type found.")

# Tải Tokenizer trên CPU độc lập
self.tokenizer: AutoTokenizer =
AutoTokenizer.from_pretrained(settings.NMT_TOKENIZER_NAME)
self.compute_type = actual_compute_type

# Hàm phiên dịch
def translate_batch(
    self, texts: list[str], source_lang: str, target_lang: str
) -> list[str]:
    if not texts: return []
    self.ensure_loaded()

    # Ánh xạ mã ngôn ngữ ISO sang mã định danh của Flores-200
    src_code: str = NLLB_LANG_MAP.get(source_lang, source_lang)
    tgt_code: str = NLLB_LANG_MAP.get(target_lang, target_lang)

    # 1. Tokenize văn bản nguồn trên CPU
    self.tokenizer.src_lang = src_code
    tokenized: list[list[str]] = [
        self.tokenizer.convert_ids_to_tokens(self.tokenizer.encode(text))
        for text in texts
    ]

    # 2. Xây dựng tiền tố mã đích bắt buộc cho NLLB
    target_prefix: list[list[str]] = [[tgt_code]] * len(texts)

    # 3. Thực thi dịch hàng loạt bằng CTranslate2
    results = self.translator.translate_batch(
        tokenized,
        target_prefix=target_prefix,
        beam_size=settings.NMT_BEAM_SIZE,
    )

    assert len(results) == len(texts), "NMTTranslator: 1:1 mapping broken"

```

```

# 4. Giải mã chuỗi token đích về văn bản thô (Loại bỏ token ngôn ngữ đầu tiên)
decoded: list[str] = []
for result in results:
    tokens: list[str] = result.hypotheses[0][1:]
    text: str = self.tokenizer.decode(
        self.tokenizer.convert_tokens_to_ids(tokens)
    )
    decoded.append(text)

return decoded

```

Hình 2.17. Minh họa mã nguồn cấu hình phiên dịch sử dụng NLLB

Thiết kế tối ưu hóa này giúp hệ thống xử lý dịch thuật song ngữ chính xác với hiệu suất tối đa, bảo toàn dòng thời gian phụ đề và hạn chế các lỗi ảo giác cấu trúc thường gặp khi dùng LLM.

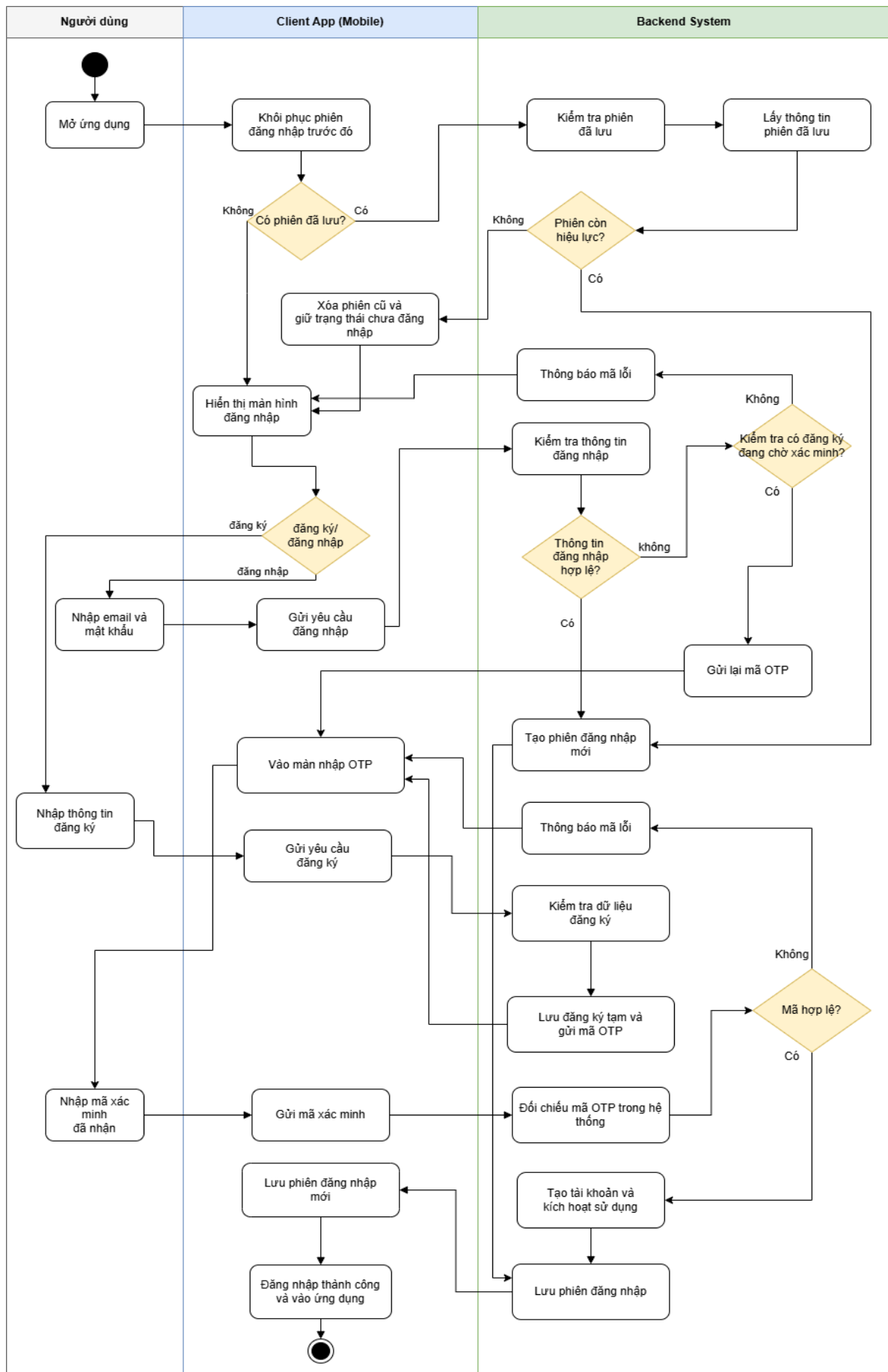
2.4. Tích hợp hạ tầng, nghiệp vụ máy chủ và hiển thị trải nghiệm người dùng

Để đưa các giải pháp thuật toán học máy phức tạp ứng dụng vào thực tế thương mại, hệ thống đòi hỏi một cấu trúc hạ tầng phần mềm bền vững để điều phối dòng chảy dữ liệu. Khối tích hợp đóng vai trò gánh tải toàn bộ các tác vụ xử lý nghiệp vụ trung tâm, quản lý định danh tài khoản, phân phối hàng đợi Job bất đồng bộ và hiện thực hóa kết xuất dữ liệu phụ đề đa lớp lên giao diện người dùng. Đây là phân hệ nền tảng minh chứng cho tính hoàn thiện của một sản phẩm phần mềm dạng dịch vụ SaaS thương mại.

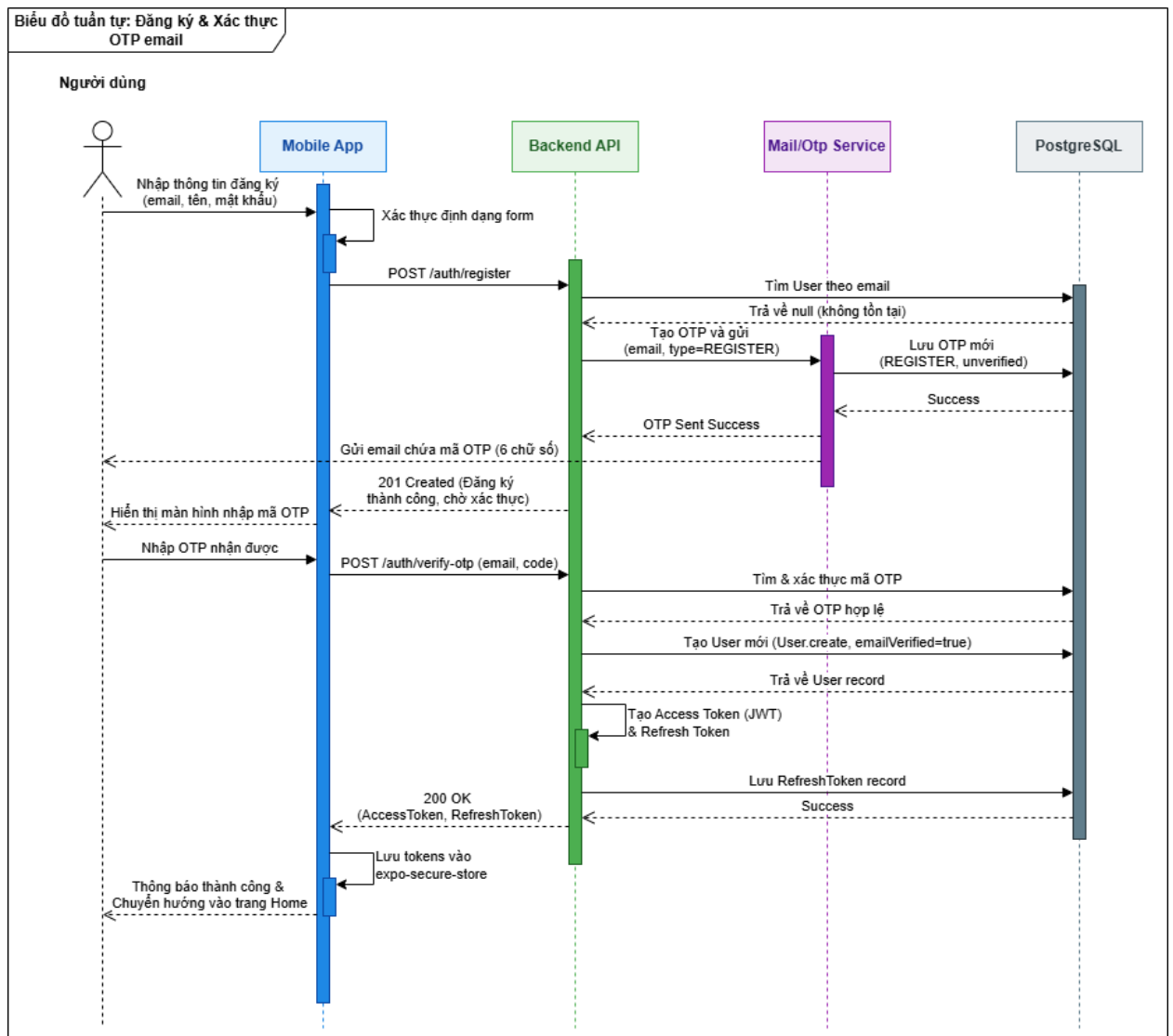
2.4.1. Nghiệp vụ máy chủ và điều phối hàng đợi (Application Logic Layer & Message Queue Layer)

a. Nghiệp vụ Đăng ký và Xác thực tài khoản:

Nghiệp vụ quản lý tài khoản được thiết kế theo cơ chế "Xác thực trước, Khởi tạo sau" (Verify-First Registration) nhằm loại bỏ triệt để các tài khoản ảo gây rác hệ thống cơ sở dữ liệu. Quy trình nghiệp vụ động này được hiện thực hóa qua sự kết hợp của Biểu đồ hoạt động (Hình 2.18) mô tả luồng rẽ nhánh logic và Biểu đồ tuần tự (Hình 2.19) mô tả dòng truyền thông điệp giữa Client, Server, Redis cache và Mail Service:



Hình 2.18. Biểu đồ hoạt động luồng Đăng ký và Xác thực tài khoản



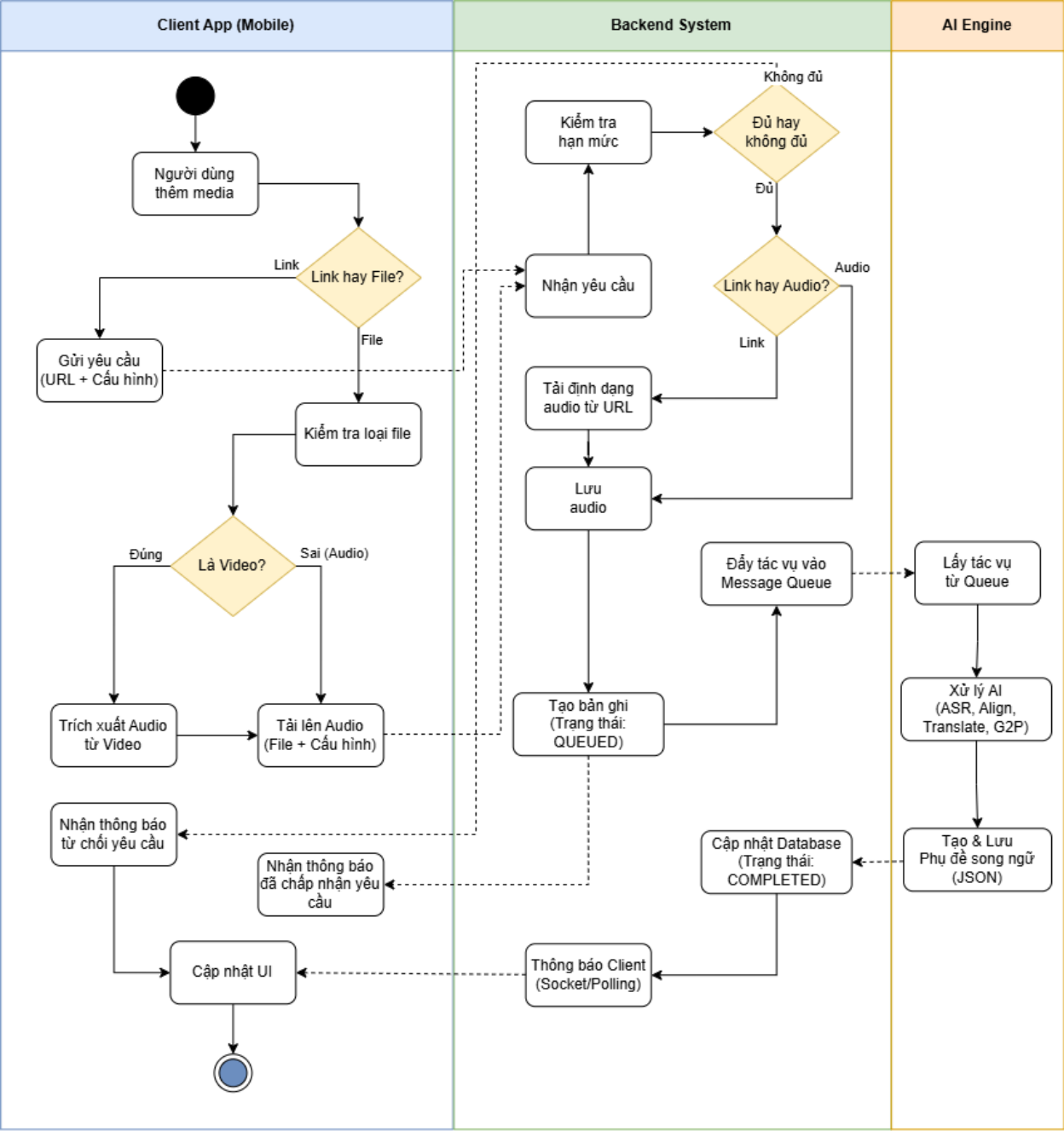
Hình 2.19. Biểu đồ tuần tự luồng Đăng ký và Xác thực OTP

Khi người dùng gửi yêu cầu đăng ký, máy chủ NestJS tiến hành kiểm tra tính duy nhất của Email thông qua Prisma Client. Sau khi vượt qua bước kiểm tra định danh, hệ thống chưa tạo bản ghi chính thức trong bảng users mà tự động sinh một mã OTP ngẫu nhiên có thời gian sống (TTL) nghiêm ngặt trong 5 phút lưu tại Redis cache, đồng thời kích hoạt luồng gửi mail bất đồng bộ. Chỉ khi người dùng nhập chính xác mã OTP, luồng nghiệp vụ mới thực hiện băm mật khẩu bằng thuật toán BCrypt với chuỗi muối an toàn, ghi nhận thực thể vào PostgreSQL và cấp phát cặp mã thông báo JWT (Access/Refresh Token) để thiết lập phiên làm việc bảo mật cho thiết bị di động.

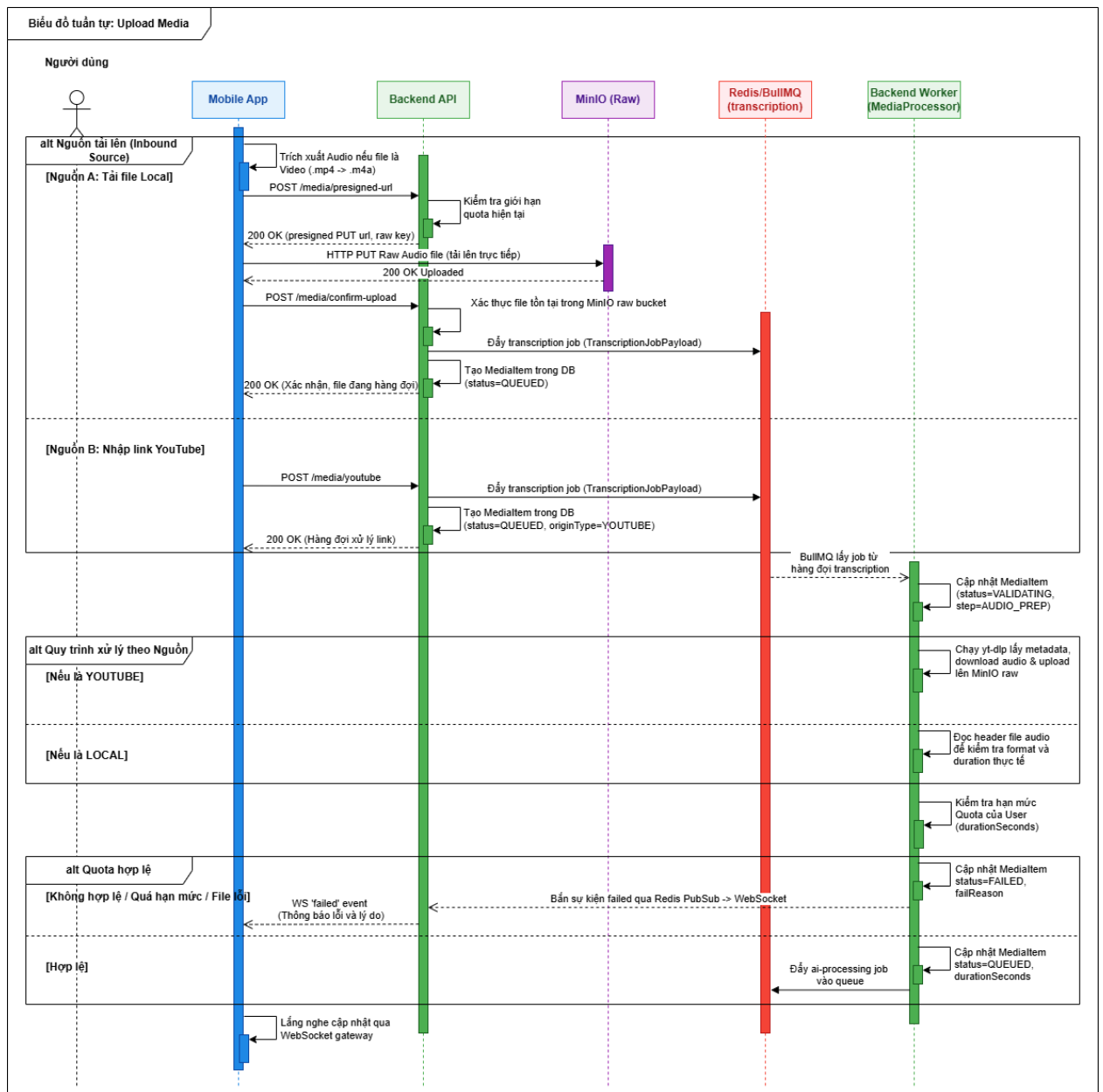
b. Nghiệp vụ Tiếp nhận Media và Điều phối Hàng đợi bất đồng bộ:

Đối với một ứng dụng SaaS xử lý video/audio, luồng tiếp nhận tư liệu đa phương tiện là nơi chịu áp lực tải lớn nhất và dễ xảy ra hiện tượng nghẽn cổ chai (bottleneck). Hệ thống giải quyết bài toán này bằng cách bóc tách quy trình thành hai giai đoạn

độc lập: Tiếp nhận API đồng bộ và Xử lý ngầm bất đồng bộ (Asynchronous Worker Pattern). Kiến trúc động này được thiết kế chi tiết thông qua biểu đồ hoạt động và biểu đồ tuần tự dưới đây:



Hình 2.20. Sơ đồ hoạt động luồng Tải lên và xử lý Media



Hình 2.21. Biểu đồ tuần tự luồng Upload Media

Quy trình vận hành logic của khối bao gồm các bước nghiêm ngặt sau:

Kiểm tra hạn ngạch và Ký URL: Máy khách gửi yêu cầu tải lên tệp tin cục bộ (LOCAL) hoặc đường dẫn trực tuyến (YOUTUBE). Máy chủ NestJS ngay lập tức thực hiện truy vấn snapshot trong bảng subscriptions để kiểm tra quỹ thời gian xử lý còn lại của tài khoản. Nếu hợp lệ, một bản ghi ai_credit_reservations được khởi tạo ở trạng thái PENDING để tạm khóa số giây hạn ngạch. Đối với tệp LOCAL, server trả về mã Presigned URL của MinIO để Client đẩy file trực tiếp lên kho chứa.

Tạo Job vào Hàng đợi Kiểm định (Transcription Queue): Ngay khi có xác nhận upload hoặc nhận được link YouTube, Backend API đẩy một Job mang payload TranscriptionJobPayload vào hàng đợi transcription do Redis quản lý. Lúc này, phản

hồi HTTP lập tức trả về cho Client với trạng thái tác vụ là QUEUED, giải phóng luồng xử lý của server API.

Tiêu thụ tác vụ tại Standalone Worker: Module Backend Worker (vận hành độc lập với API Server) tiêu thụ Job từ hàng đợi transcription. Nếu là tài nguyên YouTube, Worker gọi tiến trình ngầm yt-dlp để thu hoạch metadata và tải luồng stream âm thanh về. Worker thực hiện các bước kiểm định kỹ thuật (độ dài, định dạng codec) nâng cao, đẩy tệp vào phân vùng an toàn trên MinIO, sau đó chuyển trạng thái bảo lưu hạn ngạch sang CONFIRMED.

Bàn giao sang Hàng đợi Xử lý AI (AI Processing Queue): Sau khi tệp tin thô được chuẩn hóa và xác thực toàn vẹn, Backend Worker đóng gói payload mới mang tên AiProcessingJobPayload (chứa khóa S3 chính thức và thời lượng chính xác) và đẩy vào hàng đợi chuyên dụng ai-processing để bàn giao nhiệm vụ tính toán nặng cho microservice AI Engine xử lý trên phần cứng GPU.

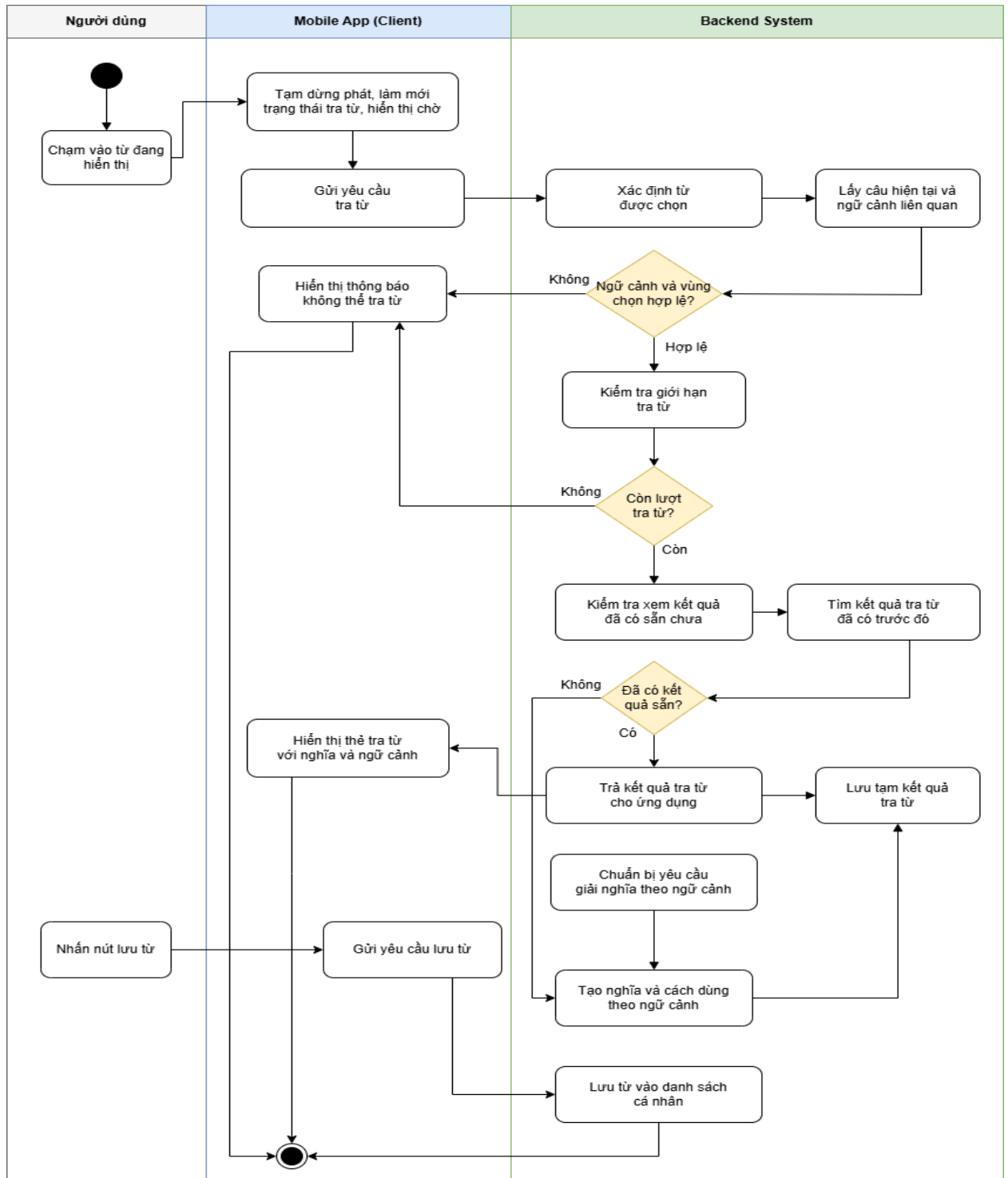
c. Nghiệp vụ Quản lý Thư viện, Đồng bộ Trạng thái và Tra cứu Từ vựng:

Sau khi tài nguyên đa phương tiện vượt qua tầng điều phối hàng đợi và được xử lý bởi AI Engine, máy chủ trung tâm NestJS chịu trách nhiệm vận hành ba trục nghiệp vụ cốt lõi điều khiển toàn bộ kịch bản sử dụng của ứng dụng:

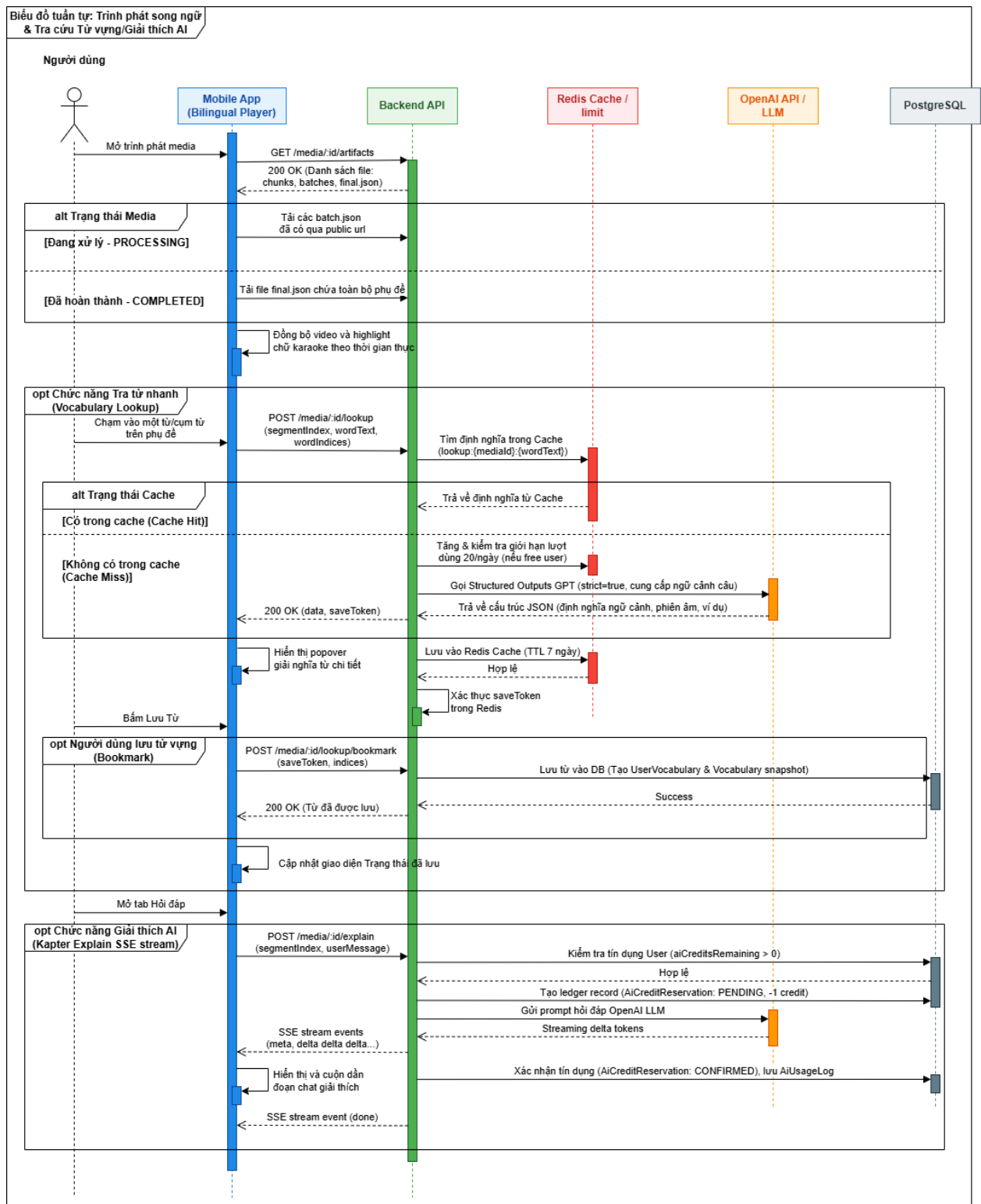
Nghiệp vụ Quản lý Vòng đời Thư viện Media: Tầng Backend API thực thi các truy vấn tối ưu thông qua Prisma ORM để kết xuất danh mục thư viện cá nhân hóa theo từng userId. Để đảm bảo tính an toàn dữ liệu và khả năng kiểm toán hạn ngạch (Audit-ready constraints) của mô hình SaaS, hệ thống không áp dụng cơ chế xóa vật lý (Hard delete). Khi người dùng thực hiện lệnh xóa một video khỏi thư viện, hệ thống thực thi cơ chế xóa mềm (Soft delete) bằng cách cập nhật nhãn thời gian vào trường deleted_at.

Nghiệp vụ Đồng bộ Tiến độ luồng sự kiện thời gian thực (Socket Sync): Để giảm phụ thuộc vào cơ chế truy vấn lặp chủ động từ máy khách (Aggressive Polling) gây cạn kiệt tài nguyên băng thông, hệ thống thiết lập kênh truyền thông hai chiều dựa trên giao thức WebSocket thông qua gateway Socket.io. Khi AI Engine thực hiện ghi lũy tiến các phân đoạn dữ liệu hoặc cập nhật tiến độ xử lý, nó sẽ phát đi một sự kiện thông qua mạng Redis Pub/Sub. Máy chủ Backend API đăng ký lắng nghe kênh này, ngay lập tức bắt giữ thông điệp và chuyển tiếp (broadcast) một cách chính xác tới thiết bị di động của người dùng tương ứng qua kết nối Socket ổn định. Cơ chế này giúp giao diện người dùng cập nhật trạng thái tác vụ (QUEUED → PROCESSING → COMPLETED) và nạp lũy tiến các mảnh phụ đề *translated_batches* theo thời gian thực.

Nghệp vụ Tra cứu và Lưu trữ Từ vựng theo Ngữ cảnh (Instant Word Lookup): Khi người dùng chạm vào một từ đơn lẻ bất kỳ trên dòng phụ đề của trình phát, hệ thống sẽ thực thi một quy trình nghiệp vụ tra cứu động để phục vụ mục đích thụ đắc ngôn ngữ trực quan. Luồng hành vi động và trình tự truyền thông điệp của phân hệ này được thiết lập nghiêm ngặt thông qua cấu trúc của biểu đồ hoạt động và biểu đồ tuần tự dưới đây:



Hình 2.22: Biểu đồ hoạt động Tra cứu từ vựng



Hình 2.23. Biểu đồ tuần tự luồng các tính năng trong Quản lý trình phát Media

Khi một yêu cầu tra từ gửi về endpoint `POST /media/:id/lookup`, Backend API tiếp nhận từ cần tra (word) cùng ngữ cảnh câu thoại gốc (context_sentence). Để bảo vệ tài nguyên hệ thống và tối ưu hóa tốc độ phản hồi, server NestJS không gọi trực tiếp dịch vụ dịch thuật bên ngoài mà tiến hành đối soát qua bộ lọc tần suất và kiểm tra bộ nhớ đệm L1 (Redis Cache). Nếu xảy ra tình trạng thiếu hụt dữ liệu đệm (Cache

miss), hệ thống mới kích hoạt công kết nối từ điển thông minh, bóc tách ngữ nghĩa chính xác dựa trên ngữ cảnh câu và trả về cấu trúc định nghĩa đã chuẩn hóa cho Client. Đồng thời, khi người dùng nhấn nút lưu từ, endpoint *POST /media/:id/lookup/bookmark* sẽ thực hiện lưu vết vĩnh viễn cấu trúc snapshot này vào bảng *user_vocabularies* thuộc PostgreSQL, bảo toàn kho dữ liệu sạch cho số tay học tập của người dùng.

d. Tổ chức lưu trữ đối tượng và cơ chế bảo mật tệp:

Do hệ thống định hướng triển khai self-hosted trên hạ tầng đám mây độc lập, kho lưu trữ đối tượng MinIO được cấu hình tương thích hoàn toàn với chuẩn API Amazon S3 để quản lý toàn bộ các file nhị phân dung lượng lớn. Tiến trình lưu trữ được phân rã thành hai phân vùng bucket chuyên biệt nhằm cô lập dữ liệu thô và dữ liệu kết quả:

Phân vùng dữ liệu thô (Raw Bucket): Lưu trữ các tệp âm thanh gốc sau khi được Client upload hoặc sau khi Backend Worker tải về từ liên kết YouTube.

Phân vùng kết quả xử lý (Processed Bucket): Tổ chức lưu trữ bền vững theo cấu trúc thư mục dạng cây nghiêm ngặt: *processed/{mediaId}/*. AI Engine thực hiện ghi lũy tiến các mảnh dữ liệu streaming vào thư mục *chunks/* (chứa câu thoại gốc) và *translated_batches/* (chứa các lô văn bản dịch nghĩa), trước khi xuất bản tệp cấu trúc hoàn chỉnh cuối cùng là *final.json*.

Để tối ưu hóa tài nguyên mạng và bảo vệ quyền riêng tư của người dùng, khối lưu trữ thực thi giải pháp bảo mật thông qua giao thức mã URL ký trước (*Presigned S3 URL*):

Luồng tải lên (Upload flow): Ứng dụng di động đàm phán với Backend API để lấy mã Presigned URL, sau đó thực hiện đẩy trực tiếp tệp âm thanh từ bộ nhớ máy lên MinIO raw bucket mà không cần truyền dữ liệu qua máy chủ trung tâm, giải phóng hoàn toàn nguy cơ nghẽn băng thông server nghiệp vụ.

Luồng hiển thị (Playback flow): Hệ thống tuyệt đối không để lộ đường dẫn tĩnh của tài nguyên lưu trên đĩa cứng. Tất cả các file đa phương tiện và file kết quả JSON phụ đề khi truyền về Client bắt buộc phải đi qua cơ chế URL ký trước với thời gian hết hạn nghiêm ngặt (chỉ có hiệu lực trong vòng vài phút), ngăn chặn triệt để các hành vi truy cập trái phép hoặc quét trộm dữ liệu.

2.4.2. Thiết kế Giao diện Ứng dụng và tương tác người dùng

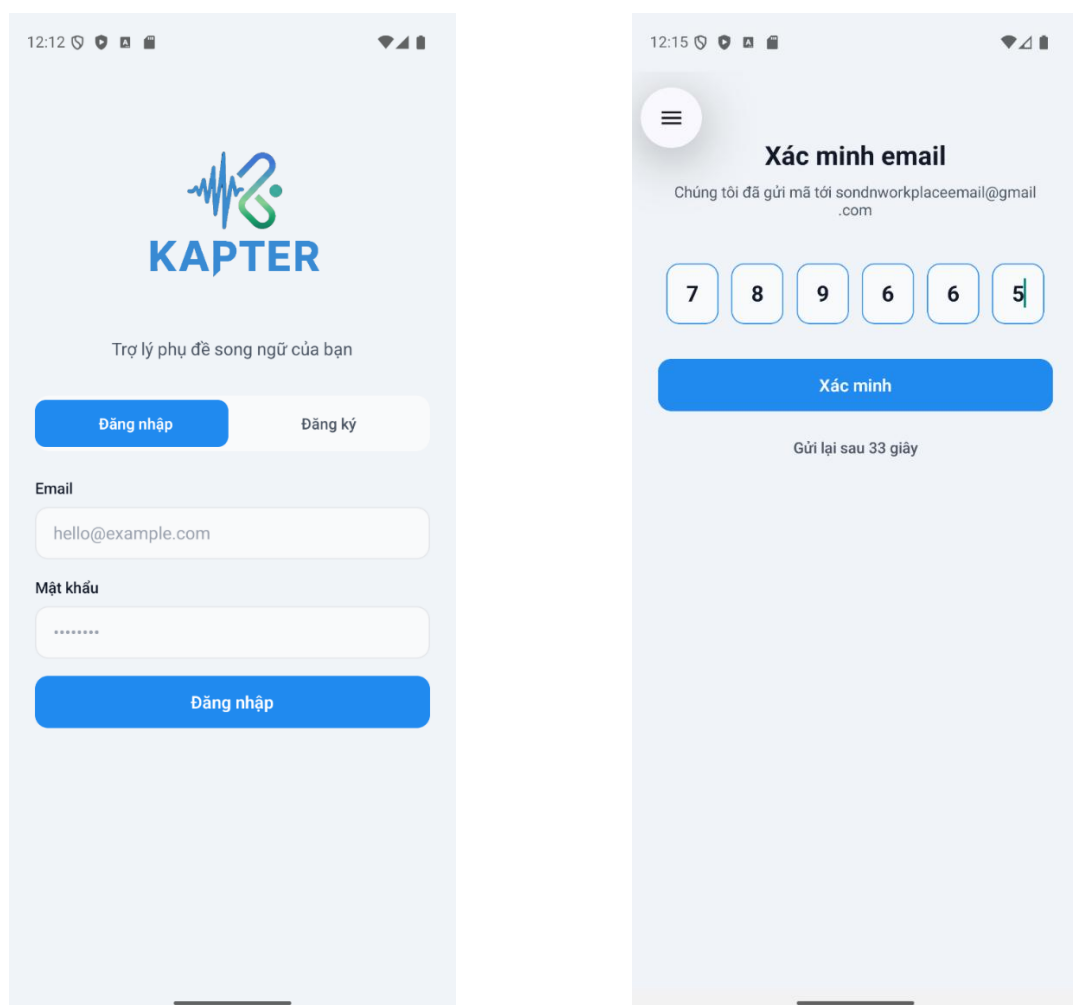
Khối hiển thị và tương tác người dùng (Presentation Layer - Mobile App Client) là tầng kiến trúc trên cùng của hệ thống Kapter, đóng vai trò là điểm chạm vật lý duy nhất chuyển hóa các logic nghiệp vụ máy chủ và tài nguyên AI thành trải nghiệm sư

phạm trực quan. Đối với một ứng dụng di động xử lý phụ đề đa lớp chạy chữ karaoke, thách thức kỹ thuật lớn nhất là phải đảm bảo việc kết xuất đồ họa mượt mà (đạt mức 60 FPS) và đồng bộ chính xác dải tín hiệu nghe nhìn theo thời gian thực mà không gây quá nhiệt hoặc cạn kiệt tài nguyên của thiết bị. Khỏi hiển thị được hiện thực hóa trên nền tảng framework React Native (Expo) kết hợp cùng hệ thống quản lý trạng thái Zustand, thư viện kết nối Socket.io Client.

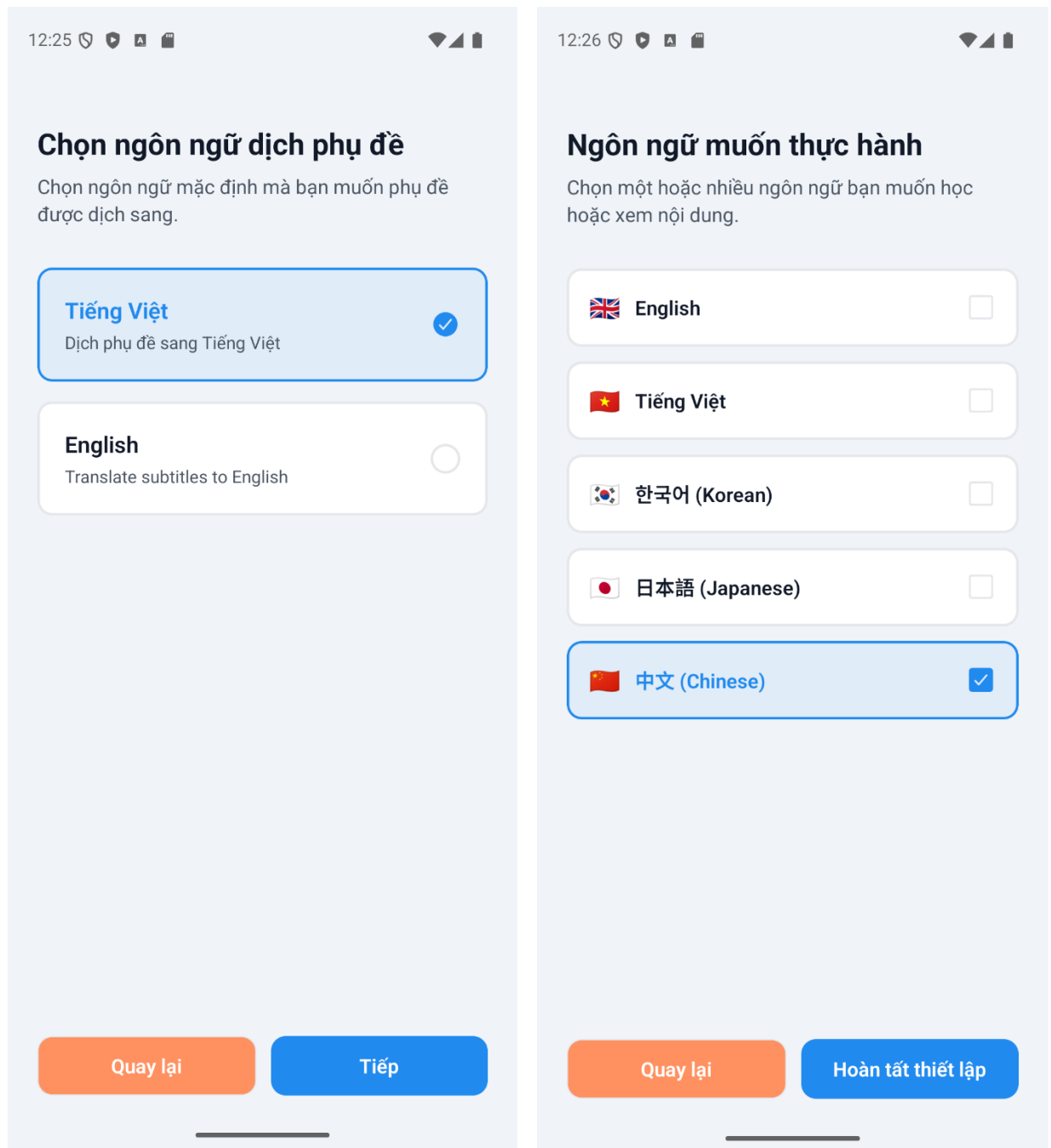
Hệ thống giao diện người dùng của ứng dụng Kapter được thiết kế theo ngôn ngữ phẳng, hiện đại và hướng mục tiêu, tối ưu hóa không gian hiển thị để người học tập trung hoàn toàn vào nội dung thính thị. Trải nghiệm người dùng được hiện thực hóa qua hệ thống ảnh chụp màn hình thực tế từ ứng dụng di động dưới đây:

Phân hệ Xác thực và Khởi tạo Trải nghiệm:

Giao diện thiết kế tối giản, tập trung vào biểu mẫu nhập mã xác minh OTP tinh gọn, kết hợp cùng các màn hình khảo sát đa lựa chọn để lưu trữ hồ sơ cấu hình ngôn ngữ đích mặc định của tài khoản ngay trong phiên làm việc đầu tiên.



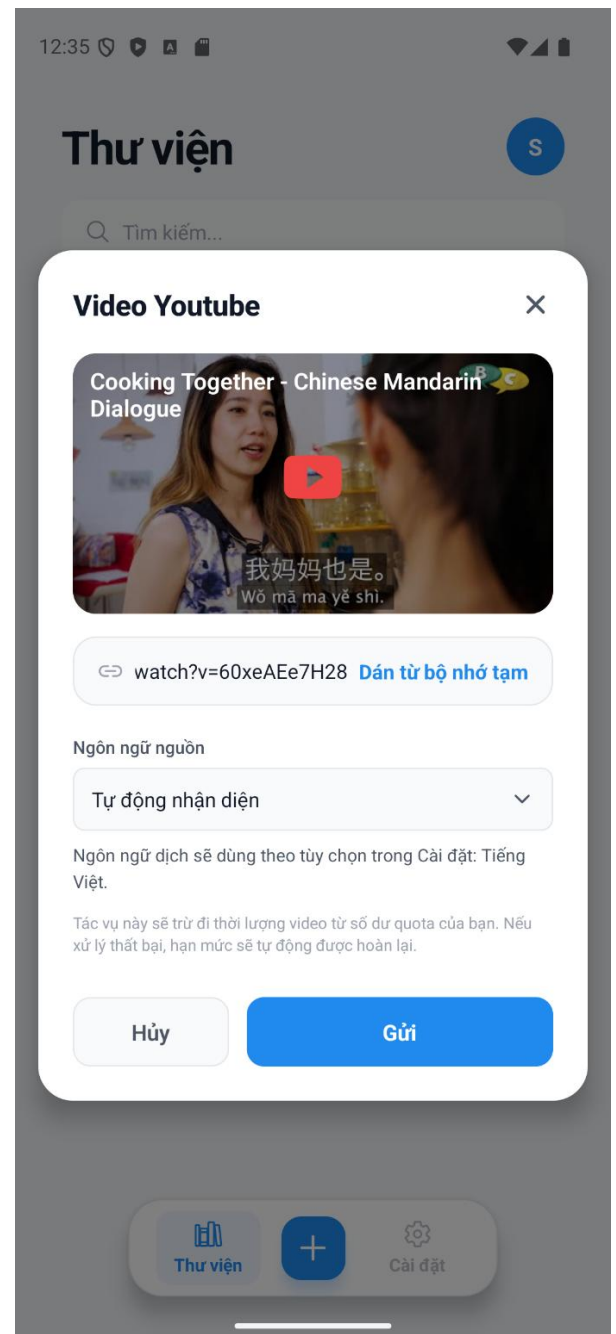
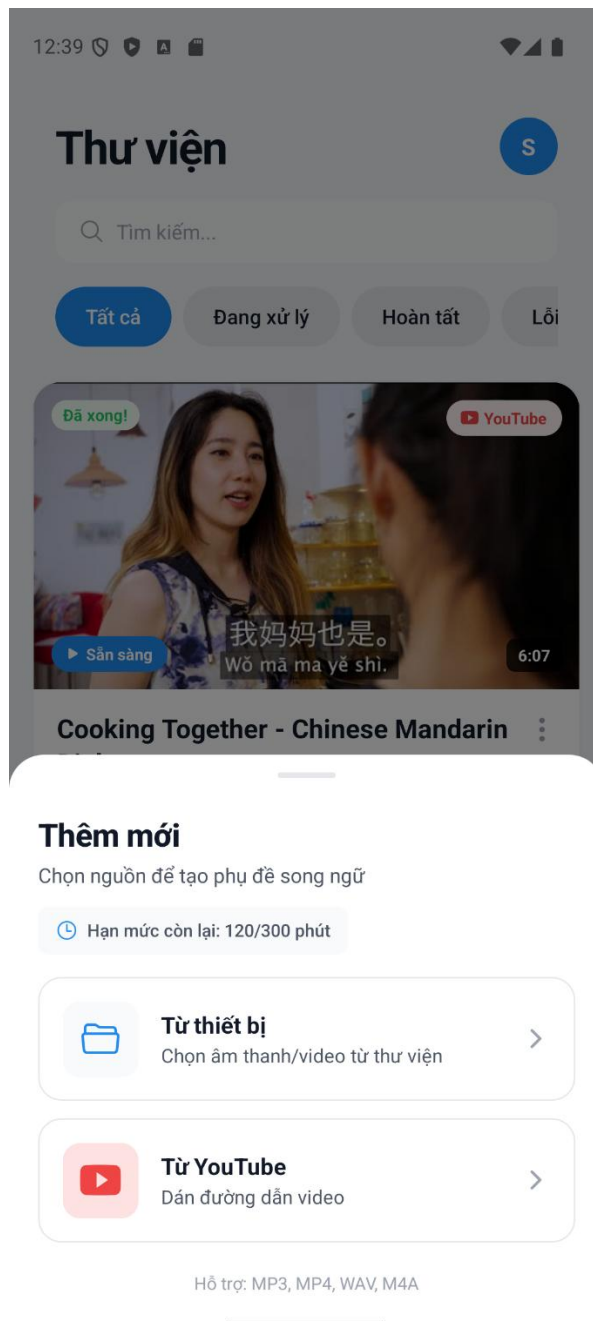
Hình 2.24. Giao diện Đăng nhập và Xác thực OTP



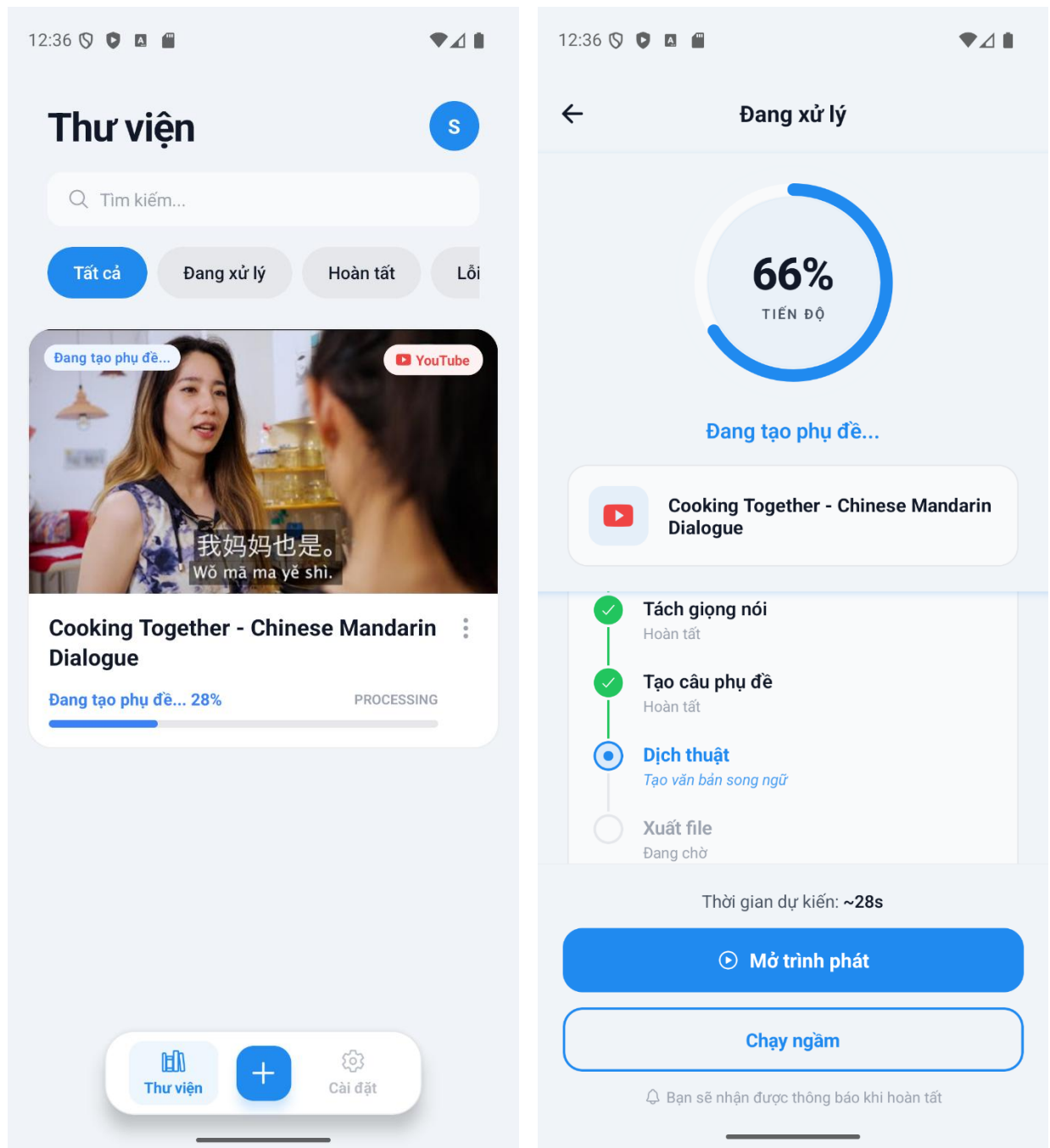
Hình 2.25. Giao diện Khởi tạo trải nghiệm

Phân hệ Thư viện và Tiến độ Xử lý thời gian thực:

Thư viện tổ chức dưới cấu trúc cinematic tỷ lệ 16:9, tích hợp bảng điều khiển nổi (Floating Pill Tab Bar) và vòng tròn tiến độ xử lý đồng bộ qua Socket. Người dùng có thể giám sát chi tiết từng công đoạn ngầm của hệ thống máy chủ (Tách giọng nói → Tạo phụ đề → Dịch thuật) một cách trực quan.



Hình 2.26. Giao diện Tải lên đa phương tiện



Hình 2.27. Giao diện Quản lý thư viện có thể theo dõi tiến độ xử lý thời gian thực

Phân hệ Trình phát Phụ đề Song ngữ và Tra cứu nhanh:

Giao diện player cốt lõi với layout hiển thị ba lớp thông tin đồng bộ (Chữ gốc - Phiên âm IPA/Pinyin - Bản dịch). Thanh công cụ điều khiển được thu gọn thành một viên thuốc nổi (Floating Pill Control Bar) chứa các phím chức năng hình squircle, giải phóng diện tích cho khung hình video và danh sách văn bản lời thoại phía dưới. Tương tác tra từ nhanh được thiết kế với thành phần popup giải thích về ngữ nghĩa và vai trò của từ vựng trong ngữ cảnh của câu ngay trong giao diện của trình phát player.



Hình 2.28. Giao diện Trình phát phụ đề và tương tác tra cứu từ nhanh

2.5. Kết luận chương 2

Chương 2 đã trình bày quá trình phân tích yêu cầu, thiết kế kiến trúc và hiện thực các thành phần cốt lõi của hệ thống tạo phụ đề song ngữ. Trên cơ sở đó, luồng xử lý media được tổ chức thành các tầng riêng biệt gồm ứng dụng di động, Backend API, Backend Worker, AI Engine và Object Storage, đồng thời sử dụng Redis/BullMQ để điều phối các tác vụ bất đồng bộ.

Bên cạnh phần kiến trúc tổng thể, chương này cũng làm rõ vai trò của các route ASR, cơ chế căn chỉnh timestamp, tầng dịch máy NMT và cách phát hành artifact theo từng giai đoạn thông qua `chunks/`, `translated_batches/` và `final.json`. Đây là nền tảng trực tiếp để bước sang Chương 3 và đánh giá khả năng vận hành end-to-end của hệ thống trong các kịch bản thực nghiệm.

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

Mục tiêu của Chương 3 là trình bày quá trình thực nghiệm và đánh giá kết quả của hệ thống sau khi đã hoàn thành các bước phân tích, thiết kế và xây dựng ở Chương 2. Việc đánh giá không chỉ tập trung vào một chỉ số riêng lẻ của mô hình nhận dạng giọng nói, mà xem xét toàn bộ luồng xử lý end-to-end của ứng dụng: từ khi người dùng gửi video/audio, hệ thống tiếp nhận và xử lý tác vụ trên máy chủ, cho đến khi dữ liệu phụ đề song ngữ được sinh ra và có thể sử dụng trên trình phát.

Trong phạm vi đề tài, hệ thống được đánh giá theo bốn nhóm tiêu chí chính. Nhóm thứ nhất là hiệu năng xử lý, bao gồm tổng thời gian xử lý, tỷ lệ giữa thời gian xử lý và thời lượng media, cũng như các mốc thời gian quan sát được trong quá trình hệ thống tạo ra artifact trung gian và kết quả cuối cùng. Nhóm thứ hai là tính đầy đủ và hợp lệ của dữ liệu đầu ra, thông qua số lượng chunk, translated batch, tệp final.json, số segment, bản dịch, phiên âm và tính hợp lệ của timestamp. Nhóm thứ ba là chất lượng nhận dạng văn bản nguồn, được đo bằng các chỉ số WER và CER trên những ca kiểm thử có phụ đề tham chiếu. Nhóm thứ tư là chất lượng bản dịch và khả năng sử dụng phụ đề, được đánh giá thông qua đối chiếu thủ công theo các tiêu chí về độ đúng nghĩa, độ tự nhiên, mức độ đầy đủ thông tin và khả năng đọc khi hiển thị dưới dạng phụ đề.

Yếu tố “thời gian thực” trong chương này được hiểu theo hướng phù hợp với phạm vi của hệ thống: giảm thời gian chờ của người dùng thông qua xử lý bất đồng bộ, cập nhật tiến độ, tạo artifact phụ đề theo từng giai đoạn và cho phép ứng dụng tiếp cận kết quả sớm khi điều kiện runtime cho phép.

Các số liệu trong chương được thu thập bằng bộ mã lệnh benchmark chạy qua luồng xử lý thực tế của hệ thống, bao gồm Backend API, Backend Worker, AI Engine, hàng đợi xử lý, hệ thống lưu trữ artifact.

3.1. Môi trường và kịch bản thực nghiệm

3.1.1. Hạ tầng và môi trường triển khai hệ thống

Nhằm đảm bảo tính khách quan và khả năng tái lập (reproducibility) của các phép đo hiệu năng, toàn bộ các thành phần trong kiến trúc phân tán của hệ thống "Kapter" được triển khai self-hosted trên một hạ tầng phần cứng cố định với các đặc tả kỹ thuật chi tiết sau:

Cụm máy chủ tính toán AI (AI Engine Service): Vận hành trên nhân GPU có dung lượng 16GB VRAM, bộ vi xử lý CPU đa lõi 8 nhân 16 luồng hiệu năng cao, bộ nhớ RAM hệ thống 32GB. Môi trường phần mềm chạy trên nền tảng Python 3.12, tăng tốc xử lý mạng nơ-ron bằng CUDA Toolkit và nhân tính toán CTranslate2.

Cụm máy chủ nghiệp vụ và Worker (Backend API & Background Worker):

Triển khai thông qua các container Docker cô lập, vận hành framework NestJS (Node.js v20+) kết hợp hệ quản trị cơ sở dữ liệu PostgreSQL 16 và kho lưu trữ đối tượng MinIO Object Storage.

Môi trường điều phối thông báo (Message Broker): Sử dụng một thực thể Redis Server phân tán độc lập nhằm duy trì tính ổn định tuyệt đối cho cấu trúc hàng đợi BullMQ.

3.1.2. Bộ dữ liệu mẫu và kịch bản thực nghiệm

Kịch bản kiểm thử được thiết kế toàn diện nhằm thách thức hệ thống dưới các điều kiện biên của môi trường thực tế, bao gồm sự đa dạng về tốc độ nói, tạp âm nền, thuật ngữ chuyên ngành và các phân đoạn code-switching giao thoa ngôn ngữ.

Cấu trúc tập dữ liệu thực nghiệm: Tập dữ liệu mẫu bao gồm 20 tài nguyên đa phương tiện được tuyển chọn trực tiếp từ nền tảng YouTube có thời lượng biến thiên từ ngắn (~5 phút) đến dài (trên 15 phút), phân rã cân bằng thành hai trục ngôn ngữ chủ đạo của hệ thống:

Phân hệ Tiếng Anh (English Track): 10 video bao gồm các bài giảng học thuật (Coursera/Udemy), tin tức tổng hợp và nội dung đàm thoại tự do.

Phân hệ Tiếng Trung (Chinese Track): 10 video bao gồm các đoạn hội thoại colloquial đời sống, phim truyền hình có chứa từ lóng, và các video giảng dạy tiếng Trung dành cho người nước ngoài chứa mật độ từ ghép mixed-script lớn.

Đối với phần đánh giá chất lượng nhận dạng văn bản nguồn, đề tài ưu tiên sử dụng các video có phụ đề ngôn ngữ nguồn do con người biên soạn hoặc có thể sử dụng làm dữ liệu tham chiếu. Các phụ đề tự động của YouTube không được sử dụng làm dữ liệu đối chiếu chính nhằm hạn chế sai số từ nguồn tham chiếu. Trong bộ kiểm thử thực tế, có 18/20 ca có dữ liệu tham chiếu phù hợp để tính WER/CER; các ca còn lại vẫn được giữ trong benchmark để đánh giá hiệu năng xử lý, khả năng sinh artifact và tính ổn định của pipeline.

3.2. Tiêu chí và phương pháp đánh giá

Các tiêu chí đánh giá được chia thành bốn nhóm chính. Nhóm thứ nhất là **hiệu năng xử lý**, bao gồm tổng thời gian xử lý, tỷ lệ giữa thời gian xử lý và thời lượng media, thời điểm quan sát được chunk đầu tiên, thời điểm quan sát được batch dịch đầu tiên và thời điểm hoàn tất xử lý. Nhóm chỉ số này được sử dụng để đánh giá mức độ đáp ứng của hệ thống đối với mục tiêu giảm thời gian chờ khi xử lý video/audio cho yếu tố “thời gian thực” của đồ án.

Nhóm thứ hai là **tính đầy đủ và hợp lệ của dữ liệu đầu ra**. Các chỉ số được xem xét gồm số lượng chunk, số lượng translated batch, sự tồn tại của tệp final.json, số lượng segment, số segment có văn bản gốc, bản dịch, phiên âm, dữ liệu word-level và các lỗi liên quan đến timestamp như segment bị chồng lấn, timestamp không tăng dần hoặc thời lượng âm.

Nhóm thứ ba là **chất lượng nhận dạng văn bản nguồn**. Với các ca có dữ liệu tham chiếu, đề tài sử dụng WER để đo tỷ lệ lỗi từ và CER để đo tỷ lệ lỗi ký tự. WER phù hợp hơn với tiếng Anh do có ranh giới từ rõ ràng, trong khi CER được sử dụng bổ sung khi đánh giá tiếng Trung do đặc thù không tách từ bằng khoảng trắng như tiếng Anh.

Nhóm thứ tư là **chất lượng bản dịch và khả năng sử dụng phụ đề**. Phần này được đánh giá thủ công trên một tập mẫu segment được xuất ra từ kết quả benchmark, dựa trên các tiêu chí: giữ đúng nghĩa, bản dịch tự nhiên, giữ tên riêng/thuật ngữ quan trọng, không bỏ sót thông tin chính và phù hợp khi đọc dưới dạng phụ đề.

Các mốc thời gian trong benchmark được ghi nhận thông qua cơ chế polling trạng thái từ Backend API với chu kỳ 1 giây (1000ms). Vì vậy, các giá trị như thời điểm xuất hiện chunk đầu tiên, batch dịch đầu tiên và *final.json* phản ánh thời điểm benchmark quan sát được artifact qua API trạng thái, không phải timestamp tuyệt đối tại thời điểm object được ghi vào hệ thống lưu trữ hoặc socket event được nhận trên thiết bị người dùng.

3.3. Kết quả hiệu năng xử lý

Bảng dưới đây trình bày **kết quả hiệu năng** tổng hợp của hệ thống trên 20 ca kiểm thử. Các chỉ số được tính từ thời điểm hệ thống tiếp nhận yêu cầu xử lý media cho đến khi hoàn tất và sinh ra kết quả cuối cùng.

Bảng 3.1. Kết quả hiệu năng tổng hợp của hệ thống

Nhóm kiểm thử	Số ca	Thời gian xử lý TB (s)	Tỷ lệ xử lý / thời lượng media	Batch dịch đầu tiên (s)
Tiếng Anh	10	125,288	0,193	67,714
Tiếng Trung	10	161,251	0,253	84,550
Toàn bộ	20	143,270	0,223	76,132

Kết quả cho thấy 20/20 ca kiểm thử đều hoàn tất thành công. Tỷ lệ xử lý trung bình của toàn bộ tập kiểm thử là 0,223, cho thấy hệ thống xử lý nhanh hơn thời lượng

thực tế của media đầu vào trong điều kiện thử nghiệm. Nhóm tiếng Trung có thời gian xử lý trung bình cao hơn nhóm tiếng Anh, chủ yếu do tuyến xử lý tiếng Trung kích hoạt thêm cơ chế kiểm soát độ tin cậy transcript trước khi phát hành kết quả dịch.

Đối với trải nghiệm người dùng, mốc “Batch dịch đầu tiên” cho biết thời điểm hệ thống bắt đầu có dữ liệu phụ đề song ngữ đầu tiên. Trung bình trên toàn bộ tập kiểm thử, mốc này được ghi nhận sau 76,132 giây. Các mốc thời gian trong bảng được ghi nhận thông qua cơ chế polling trạng thái từ Backend API với chu kỳ 1000ms, nên được hiểu là thời điểm benchmark quan sát được artifact, không phải timestamp tuyệt đối của hệ thống lưu trữ hoặc socket event.

Ngoài trạng thái hoàn tất của job xử lý, benchmark cũng kiểm **tra tính hợp lệ của dữ liệu phụ đề đầu ra**. Kết quả cho thấy 20/20 ca đều sinh được tệp final.json hợp lệ, không ghi nhận trường hợp bản dịch rỗng, timestamp không hợp lệ hoặc segment bị chùng lún thời gian. Đồng thời, 18/20 ca có artifact trung gian xuất hiện trước thời điểm hoàn tất, cho thấy hệ thống có hỗ trợ cơ chế phát hành dữ liệu phụ đề theo từng giai đoạn, dù thời điểm xuất hiện artifact còn phụ thuộc vào chính sách runtime của từng tuyến xử lý.

3.4. Đánh giá chất lượng nhận dạng văn bản nguồn

Chất lượng nhận dạng văn bản nguồn được đánh giá trên các ca kiểm thử có dữ liệu tham chiếu phù hợp. Trong bộ benchmark, có 18/20 ca được sử dụng để tính WER/CER, bao gồm 10 ca tiếng Anh và 8 ca tiếng Trung. Hai ca không có phụ đề tham chiếu vẫn được giữ lại trong benchmark hiệu năng, nhưng không được đưa vào phần đánh giá lỗi nhận dạng.

Bảng 3.2. Kết quả nhận dạng văn bản nguồn

Nhóm kiểm thử	Số ca có reference	WER trung bình	CER trung bình
Tiếng Anh	10	0,036	0,021
Tiếng Trung	8	0,247	0,181
Toàn bộ	18	0,130	0,092

Kết quả cho thấy tuyến xử lý tiếng Anh đạt chất lượng nhận dạng tốt trên tập kiểm thử, với WER trung bình 0,036 và CER trung bình 0,021. Đối với tiếng Trung, các chỉ số lỗi cao hơn, với WER trung bình 0,247 và CER trung bình 0,181. Sự chênh lệch này phản ánh mức độ khó cao hơn của tuyến xử lý tiếng Trung, đặc biệt trong

các nội dung hội thoại, ngữ cảnh tự nhiên hoặc có chất lượng phụ đề tham chiếu không đồng đều.

Trong phần đánh giá này, WER được sử dụng để đo lỗi ở mức từ, còn CER được sử dụng để bổ sung góc nhìn ở mức ký tự. Với tiếng Anh, WER thường dễ diễn giải hơn do ranh giới từ rõ ràng. Với tiếng Trung, CER có ý nghĩa tham khảo quan trọng hơn vì văn bản không tách từ bằng khoảng trắng như tiếng Anh. Do đó, hai chỉ số được trình bày song song để phản ánh chất lượng transcript một cách cân bằng hơn.

Nhìn chung, kết quả nhận dạng văn bản nguồn của hệ thống đạt mức ổn định hơn ở nhóm tiếng Anh, trong khi nhóm tiếng Trung còn biến động mạnh giữa các ca kiểm thử. Vì vậy, WER/CER được sử dụng như chỉ số đánh giá chất lượng transcript đầu vào, không được xem là tiêu chí duy nhất để kết luận chất lượng cuối cùng của hệ thống phụ đề song ngữ.

3.5. Đánh giá chất lượng bản dịch và khả năng sử dụng phụ đề

Bên cạnh chất lượng nhận dạng văn bản nguồn, **chất lượng bản dịch** cũng là yếu tố quan trọng quyết định khả năng sử dụng của phụ đề song ngữ. Do hệ thống không chỉ tạo transcript, mà còn cung cấp bản dịch tiếng Việt để người học theo dõi nội dung, phần đánh giá này được thực hiện bằng phương pháp đối chiếu thủ công trên các segment mẫu được xuất ra từ kết quả benchmark.

Từ mỗi ca kiểm thử, hệ thống lấy một số segment đại diện trong final.json, bao gồm văn bản nguồn, bản dịch hệ thống sinh ra, phiên âm và mốc thời gian. Các segment này được đánh giá theo năm tiêu chí: giữ đúng nghĩa so với câu nguồn, độ tự nhiên của tiếng Việt, khả năng giữ tên riêng hoặc thuật ngữ quan trọng, mức độ không bỏ sót thông tin chính và khả năng đọc khi hiển thị dưới dạng phụ đề.

Bảng 3.3. Tiêu chí đánh giá chất lượng bản dịch

Tiêu chí đánh giá	Ý nghĩa
Giữ đúng nghĩa	Bản dịch truyền đạt đúng nội dung chính của câu nguồn
Độ tự nhiên	Câu tiếng Việt dễ hiểu, không quá máy móc
Giữ thuật ngữ/tên riêng	Tên riêng, thuật ngữ hoặc cụm quan trọng không bị dịch sai
Không bỏ sót thông tin chính	Bản dịch không làm mất các ý quan trọng trong câu nguồn

Khả năng đọc dưới dạng phụ đề	Câu dịch đủ gọn và phù hợp để theo dõi khi xem video
-------------------------------	--

Mỗi tiêu chí được chấm theo thang điểm 1-5, trong đó 1 thể hiện chất lượng kém và 5 thể hiện chất lượng tốt. Việc đánh giá thủ công được lựa chọn vì chất lượng bản dịch phụ đề không chỉ phụ thuộc vào mức độ đúng từng từ, mà còn phụ thuộc vào ngữ cảnh, độ tự nhiên và khả năng đọc theo thời gian. Đây là những yếu tố khó phản ánh đầy đủ bằng các chỉ số tự động nếu không có bản dịch tham chiếu đáng tin cậy.

Để đánh giá chất lượng bản dịch, đề tài thực hiện chấm thủ công trên 100 segment được trích xuất từ kết quả benchmark, tương ứng với 5 segment đại diện cho mỗi ca kiểm thử. Các segment được đánh giá theo thang điểm 1-5 trên năm tiêu chí: giữ đúng nghĩa, độ tự nhiên của tiếng Việt, khả năng giữ tên riêng/thuật ngữ, mức độ không bỏ sót thông tin chính và khả năng đọc khi hiển thị dưới dạng phụ đề. Bảng dưới đây trình bày 10 segment minh họa được trích từ tập 100 segment đã chấm thủ công, gồm 5 segment tiếng Anh và 5 segment tiếng Trung. Các ví dụ được chọn nhằm thể hiện cả trường hợp bản dịch tốt và trường hợp còn hạn chế do câu nguồn dài, ngữ cảnh khó hoặc lỗi từ bước nhận dạng/segment.

Bảng 3.4. Minh họa chấm điểm chất lượng dịch thuật

Nhóm	Văn bản nguồn	Bản dịch hệ thống	Điểm TB	Nhận xét
Tiếng Anh	“Sounded simple enough,”	“Nghe có vẻ đơn giản,”	5,0	Dịch ngắn gọn, đúng nghĩa và phù hợp phụ đề.
Tiếng Anh	“yet I'd sit on these silent retreats...”	“nhưng tôi vẫn ngồi trong những khóa tu yên tĩnh đó...”	5,0	Giữ đúng hình ảnh và sắc thái câu nguồn.
Tiếng Anh	“By providing space for constant evolution...”	“Bằng cách tạo không gian cho sự tiến hóa liên tục...”	4,0	Giữ ý chính nhưng cụm “tiến hóa liên tục” còn hơi cứng.
Tiếng Anh	“AI slop is saturating the internet...”	“AI slop đang tràn ngập internet...”	3,4	Một số thuật ngữ mới khó dịch tự nhiên, cần xử lý ngữ cảnh tốt hơn.

Tiếng Anh	“Just last month, the CEO of GitHub...”	“Chỉ tháng trước, CEO của GitHub...”	5,0	Dịch đúng, giữ được tên riêng và thuật ngữ kỹ thuật.
Tiếng Trung	“那你对台湾有什么样的印象？”	“Bạn có ấn tượng gì về Đài Loan?”	5,0	Dịch đúng và tự nhiên.
Tiếng Trung	“说句实话啊，很多人都说台湾很落后...”	“Nói thật là nhiều người nói Đài Loan rất lạc hậu...”	5,0	Dịch đầy đủ, giữ đúng ý so sánh và địa danh.
Tiếng Trung	“台湾的女生我觉得应该挺。”	“Tôi nghĩ các cô gái Đài Loan khá dễ thương.”	3,8	Câu nguồn bị cắt, bản dịch có phần suy diễn thêm sắc thái.
Tiếng Trung	“怪不得我好累，而且我们还没开始煮饭呢...”	“Không ngạc nhiên tôi mệt, mà chúng ta còn chưa bắt đầu nấu ăn...”	4,4	Giữ đủ ý nhưng câu tiếng Việt hơi dài và chưa thật tự nhiên.
Tiếng Trung	“教小解是。”	“Bài học nhỏ là.”	3,0	Câu nguồn có dấu hiệu lỗi ASR/segment nên bản dịch chỉ giữ được nghĩa rời rạc.

Bảng chấm đầy đủ 100 segment được lưu trong phụ lục/kết quả thực nghiệm. Trong thân báo cáo, bảng trên chỉ được sử dụng để minh họa cách đánh giá và một số dạng lỗi điển hình. Dưới đây là bảng kết quả chấm điểm tổng hợp của 100 segment:

Bảng 3.5. Tổng hợp kết quả chấm điểm dịch thuật

Nhóm kiểm thử	Số segment	Đúng nghĩa	Tự nhiên	Giữ thuật ngữ/tên riêng	Không mất ý	Dễ đọc phụ đề	Trung bình
Tiếng Anh	50	4,800	4,640	4,900	4,800	4,540	4,736
Tiếng Trung	50	4,560	4,540	4,820	4,600	4,560	4,616

Toàn bộ	100	4,680	4,590	4,860	4,700	4,550	4,676
---------	-----	-------	-------	-------	-------	-------	-------

Từ bảng trên, chúng ta có thể nhận xét như sau:

Chất lượng bản dịch từ tiếng Anh ổn định hơn, đặc biệt ở mức đúng nghĩa và giữ thuật ngữ. Một số điểm bị trừ chủ yếu do câu dịch hơi literal hoặc segment quá dài khi hiển thị dạng phụ đề.

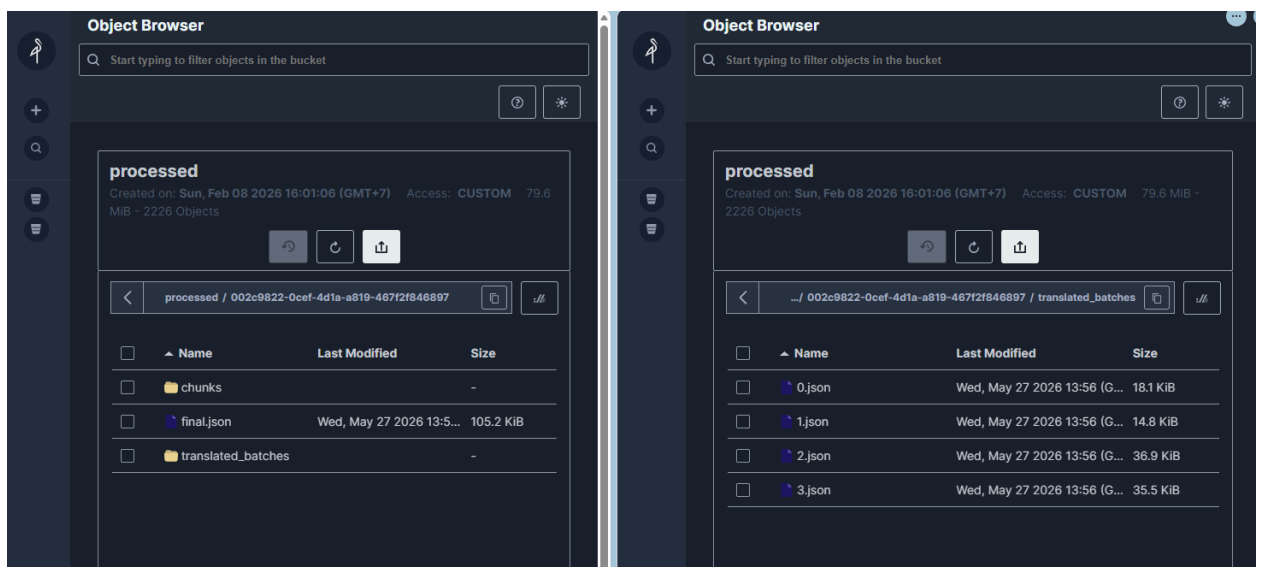
Chất lượng bản dịch tiếng Trung hiện chỉ ở mức có thể sử dụng được, nhưng có vài đoạn bị ảnh hưởng bởi transcript nguồn/segment lỗi, ví dụ câu nguồn bị cắt hoặc khó hiểu. Những lỗi này không hoàn toàn là lỗi dịch, mà là lỗi lan truyền từ quá trình ASR hay segmentation.

Điểm dễ đọc phụ đề thấp hơn các tiêu chí khác vì nhiều segment dài, dù bản dịch đúng nghĩa nhưng chưa tối ưu cho việc đọc nhanh trên màn hình phụ đề.

3.6. Minh chứng kết quả đầu ra của hệ thống

Bên cạnh các bảng số liệu thực nghiệm, đề tài cũng ghi nhận trực tiếp các artifact và giao diện kết quả do hệ thống sinh ra sau quá trình xử lý. Các minh chứng này nhằm cho thấy dữ liệu đầu ra không chỉ tồn tại ở dạng log benchmark, mà có thể được lưu trữ, truy xuất và sử dụng bởi ứng dụng di động.

Sau khi một media được xử lý thành công, hệ thống lưu các kết quả phụ đề theo nhiều tầng artifact. Trong đó, *chunks/* lưu các kết quả trung gian theo từng phần, *translated_batches/* lưu các batch phụ đề đã dịch, còn *final.json* là tệp kết quả cuối cùng được trình phát sử dụng khi media hoàn tất xử lý.



Hình 3.1. Cấu trúc artifact của một media sau khi xử lý trên MinIO

Trong tệp `final.json`, dữ liệu phụ đề được tổ chức theo từng segment. Mỗi segment bao gồm các thông tin cần thiết cho việc hiển thị phụ đề như thời điểm bắt đầu, thời điểm kết thúc, văn bản gốc, bản dịch tiếng Việt, phiên âm và dữ liệu word-level nếu có. Cấu trúc này cho phép ứng dụng di động đồng bộ phụ đề với video/audio và hỗ trợ các thao tác học tập như xem phụ đề song ngữ hoặc tra cứu từ trong ngữ cảnh.

```
apps > ai-engine > outputs > C:\english_moW9jvMr4\final.json > {} segments > {} 1 > {} words > {} 5
1 {
2   "metadata": {
3     "duration": 565.0,
4     "engine_profile": "HIGH",
5     "source_lang": "en",
6     "target_lang": "vi",
7     "model_used": "large-v3-turbo"
8   },
9   "segments": [
10    {
11      "text": "When I was first learning to meditate, the instruction was to simply pay attention to my breath, and when my mind wandered, to bring it back.",
12      "start": 12.8,
13      "end": 19.32,
14      "words": [ ...
156    ],
157      "translation": "Khi tôi mới học thiền, hướng dẫn chỉ đơn giản là chú ý đến hơi thở của tôi, và khi tâm trí tôi lang thang, để đưa nó trở lại.",
158      "phonetic": "wɪn aɪ wəz fɜːst ˈlærnɪŋ tɪ ˈmedəˌteɪt, ðə ˈɪnˈstrʌkʃən wəz tɪ ˈsɪmpli peɪ əˈtenʃən tɪ maɪ breθ, and wɪn maɪ maɪnd ˈwɒndəd, tɪ brɪŋ ɪt bæk.",
159      "detected_lang": "en",
160      "segment_index": 0
161    },
162    {
163      "text": "Sounded simple enough, yet I'd sit on these silent retreats, sweating through T-shirts in the middle of winter.",
164      "start": 20.7,
165      "end": 28.2,
166      "words": [
167        {
168          "word": "Sounded",
169          "start": 20.7,
170          "end": 21.04,
171          "confidence": 0.751,
172          "phoneme": "'saundɪd"
173        },
174        {
175          "word": "simple",
176          "start": 21.04,
177          "end": 21.3,
178          "confidence": 0.997,
179          "phoneme": "'sɪmpəl"
180        }
181      ]
182    }
183  ]
184 }
```

Hình 3.2. Ví dụ cấu trúc một segment trong tệp `final.json`

Trên ứng dụng di động, kết quả xử lý được hiển thị thông qua trình phát media. Người dùng có thể theo dõi nội dung với phụ đề gốc, bản dịch tiếng Việt và thông tin hỗ trợ phát âm. Đây là minh chứng cho việc dữ liệu sinh ra từ pipeline xử lý có thể được sử dụng trực tiếp trong giao diện người dùng, thay vì chỉ dừng lại ở kết quả xử lý phía máy chủ.

3.7. Đánh giá thực nghiệm

Đánh giá chung: Dựa trên các kết quả thực nghiệm, có thể đánh giá rằng hệ thống đã đạt được mục tiêu chính ở khía cạnh xây dựng một ứng dụng tạo phụ đề song ngữ cho video/audio sử dụng các mô hình học máy có sẵn. Hệ thống đã hoàn thành được luồng xử lý end-to-end trên 20/20 ca kiểm thử, từ tiếp nhận video YouTube, xử lý qua pipeline AI, sinh artifact phụ đề, đến tạo tệp kết quả cuối cùng phục vụ trình phát. Kết quả này cho thấy đề tài không chỉ dừng lại ở mức thử nghiệm một mô hình riêng lẻ, mà đã xây dựng được một hệ thống phần mềm có khả năng vận hành đầy đủ theo luồng nghiệp vụ đặt ra.

Về chất lượng đầu ra, hệ thống đã sinh được phụ đề có cấu trúc gồm văn bản nguồn, bản dịch tiếng Việt, phiên âm và timestamp. Các tệp final.json được tạo đầy đủ cho toàn bộ ca kiểm thử, không ghi nhận lỗi nghiêm trọng về schema, timestamp hoặc bản dịch rỗng. Kết quả nhận dạng tiếng Anh đạt mức tốt với WER trung bình 0,036, trong khi chất lượng bản dịch được chấm thủ công trên 100 segment đạt điểm trung bình 4,676/5. Những kết quả này cho thấy hệ thống có khả năng tạo ra phụ đề song ngữ có thể sử dụng được cho mục tiêu học ngoại ngữ trong phạm vi thử nghiệm.

Hạn chế: đối chiếu trực tiếp với tên đề tài, phần “thời gian thực” vẫn là giới hạn lớn nhất của hệ thống hiện tại. Benchmark cho thấy thời gian xử lý trung bình của hệ thống là 143,270 giây, tương ứng tỷ lệ xử lý/thời lượng media là 0,223, nghĩa là nhanh hơn thời lượng media gốc khoảng 4,48 lần, mốc xuất hiện batch dịch đầu tiên trung bình là 76,132 giây, trong đó tiếng Anh là 67,714 giây và tiếng Trung là 84,550 giây. Với trải nghiệm người dùng, khoảng chờ này vẫn còn dài so với kỳ vọng của một hệ thống “thời gian thực”. Yếu tố “thời gian thực” ở đây được hiểu là hệ thống có cơ chế xử lý bất đồng bộ, cập nhật tiến độ và phát hành artifact theo từng giai đoạn, thay vì phiên dịch trực tiếp đồng thời với lời nói đầu vào. Hệ thống hiện tại đã đạt được hướng tiếp cận này ở mức kiến trúc và pipeline, nhưng chưa đạt mức tối ưu về thời gian phản hồi ban đầu. Nói cách khác, hệ thống đã có nền tảng để phát triển theo hướng thời gian thực, nhưng thời gian để người dùng bắt đầu xem được phụ đề song ngữ vẫn cần tiếp tục được cải thiện.

Ngoài ra, chất lượng nhận dạng giữa hai nhóm ngôn ngữ còn chưa đồng đều. Tuyến tiếng Anh có kết quả ổn định, nhưng tuyến tiếng Trung đạt WER trung bình 0,247 và CER trung bình 0,181, cao hơn đáng kể. Điều này cho thấy pipeline tiếng Trung vẫn chịu ảnh hưởng lớn bởi đặc điểm ngôn ngữ, chất lượng âm thanh, ngữ cảnh hội thoại và độ khớp của phụ đề tham chiếu. Vì vậy, hệ thống hiện tại phù hợp hơn với mục tiêu chứng minh tính khả thi của ứng dụng tạo phụ đề song ngữ, chưa thể kết luận rằng chất lượng nhận dạng đã ổn định trên mọi loại nội dung đầu vào.

Tổng kết: hệ thống đã đạt được phần cốt lõi của đề tài: xây dựng ứng dụng có khả năng tự động tạo phụ đề song ngữ cho video/audio bằng cách tích hợp các thành phần xử lý tiếng nói và ngôn ngữ tự nhiên vào một pipeline phần mềm hoàn chỉnh. Tuy nhiên, hệ thống vẫn chưa đạt được mức “thời gian thực” nếu so với kỳ vọng của các ứng dụng thương mại có trải nghiệm phản hồi nhanh. Các hướng cải thiện quan trọng gồm rút ngắn thời gian xuất hiện batch dịch đầu tiên, tối ưu chính sách xử lý cho tiếng Trung, mở rộng đánh giá chất lượng bản dịch và đo chính xác hơn trải nghiệm phía client trên thiết bị di động.

3.8. Vai trò của các giải pháp kỹ thuật trong kết quả thực nghiệm

Các kết quả thực nghiệm ở Chương 3 không chỉ đến từ năng lực của các mô hình học máy có sẵn, mà còn phụ thuộc vào cách hệ thống tổ chức pipeline xử lý, điều phối tác vụ và kiểm soát chất lượng dữ liệu đầu ra. Trong phạm vi đồ án, một số giải pháp kỹ thuật được triển khai nhằm giúp các mô hình có thể vận hành ổn định hơn trong luồng ứng dụng thực tế.

Thứ nhất, hệ thống sử dụng các bước tiền xử lý âm thanh, phân đoạn giọng nói và định tuyến mô hình theo ngôn ngữ để chuẩn bị đầu vào phù hợp hơn cho ASR. Các bước này giúp giảm ảnh hưởng của dữ liệu đầu vào không đồng nhất, đồng thời cho phép hệ thống lựa chọn tuyến xử lý khác nhau cho tiếng Anh và tiếng Trung.

Thứ hai, cơ chế kiểm soát độ tin cậy transcript được sử dụng ở tuyến tiếng Trung nhằm hạn chế việc phát hành kết quả từ các transcript chưa đủ tin cậy. Cơ chế này làm tăng độ thận trọng khi xử lý tiếng Trung, nhưng cũng tạo ra trade-off về thời gian phản hồi ban đầu, như đã phân tích trong phần đánh giá thực nghiệm.

Thứ ba, hệ thống tổ chức dữ liệu đầu ra theo các tầng artifact như `chunks/`, `translated_batches/` và `final.json`. Cách tổ chức này giúp benchmark có thể kiểm tra được tiến trình xử lý, đồng thời tạo cơ sở để ứng dụng di động truy xuất kết quả phụ đề theo từng giai đoạn khi điều kiện runtime cho phép.

Thứ tư, việc tách Backend API, Backend Worker, AI Engine và hệ thống lưu trữ artifact giúp các tác vụ AI có thời gian xử lý dài không chặn trực tiếp luồng request-response của người dùng. Đây là yếu tố quan trọng để hệ thống có thể vận hành end-to-end trên các video/audio dài trong phạm vi thử nghiệm.

Nhìn chung, các giải pháp kỹ thuật trên đóng vai trò hỗ trợ quá trình tích hợp mô hình học máy vào một ứng dụng phần mềm hoàn chỉnh. Tuy nhiên, chúng không loại bỏ hoàn toàn các hạn chế về chất lượng nhận dạng tiếng Trung hoặc thời gian xuất hiện batch dịch đầu tiên. Đây vẫn là các hướng cần tiếp tục cải thiện trong những phiên bản sau.

3.9. Kết luận chương 3

Chương 3 đã trình bày bộ tiêu chí đánh giá, môi trường thực nghiệm và các kết quả đo được trên luồng xử lý thực tế của hệ thống. Kết quả cho thấy hệ thống có thể hoàn thành pipeline end-to-end, sinh được các artifact trung gian và tệp final.json, đồng thời hỗ trợ hiển thị phụ đề song ngữ trên ứng dụng di động.

Tuy vậy, các số liệu thực nghiệm cũng chỉ ra rằng độ trễ để xuất hiện batch phụ đề dịch đầu tiên vẫn còn khá lớn, đặc biệt ở tuyến tiếng Trung. Vì vậy, có thể kết luận rằng hệ thống đã chứng minh được tính khả thi về mặt kỹ thuật và tích hợp sản phẩm, nhưng vẫn cần tiếp tục tối ưu nếu muốn tiến gần hơn tới mục tiêu phản hồi nhanh trong trải nghiệm sử dụng.

KẾT LUẬN

1. Kết luận chung

Đề tài đã xây dựng được một hệ thống ứng dụng hỗ trợ tạo phụ đề song ngữ cho video/audio, phục vụ nhu cầu học ngoại ngữ qua nội dung đa phương tiện. Hệ thống kết hợp ứng dụng di động, máy chủ backend, worker xử lý nền, AI Engine và hệ thống lưu trữ artifact để tạo thành một luồng xử lý end-to-end từ lúc tiếp nhận media đến khi người dùng có thể xem phụ đề trên trình phát.

Trong phạm vi đồ án, các mô hình và kỹ thuật Machine Learning được sử dụng theo hướng tích hợp vào hệ thống phần mềm, bao gồm nhận dạng giọng nói, dịch máy, căn chỉnh thời gian và sinh phiên âm. Trọng tâm của đề tài tập trung vào việc lựa chọn, phối hợp và triển khai các thành phần này trong một pipeline xử lý có thể vận hành được trên sản phẩm ứng dụng.

Kết quả thực nghiệm cho thấy hệ thống có thể tự động tạo phụ đề song ngữ gồm văn bản nguồn, bản dịch tiếng Việt, phiên âm và timestamp cho các video/audio đầu vào. Tuy nhiên, hệ thống hiện tại vẫn còn khoảng cách so với kỳ vọng “thời gian thực” ở góc độ trải nghiệm người dùng, đặc biệt là thời gian để có batch phụ đề dịch đầu tiên. Vì vậy, có thể xem phiên bản hiện tại là một nền tảng khả thi cho bài toán tạo phụ đề song ngữ tự động, nhưng vẫn cần tiếp tục tối ưu để đạt trải nghiệm phản hồi nhanh hơn và ổn định hơn.

2. Hướng phát triển tiếp theo

Trong các phiên bản tiếp theo, hướng phát triển quan trọng nhất là rút ngắn thời gian xuất hiện phụ đề song ngữ đầu tiên, nhằm đưa hệ thống tiến gần hơn tới mục tiêu thời gian thực trong trải nghiệm sử dụng. Việc này có thể thực hiện thông qua tối ưu chiến lược chia đoạn, thời điểm phát hành artifact, cơ chế phối hợp giữa nhận dạng giọng nói và dịch máy, cũng như chính sách xử lý đối với các tuyến ngôn ngữ có độ phức tạp cao.

Bên cạnh đó, hệ thống cần tiếp tục nâng cao chất lượng mô hình lỗi, đặc biệt ở tuyến xử lý tiếng Trung. Một số hướng có thể nghiên cứu gồm tinh chỉnh các mô hình nhận dạng giọng nói trên dữ liệu phù hợp hơn, thử nghiệm thêm các route ASR chuyên biệt, hoặc tích hợp tầng hiệu chỉnh ngữ nghĩa bằng mô hình ngôn ngữ lớn để giảm lỗi trong các trường hợp thoại khẩu ngữ, ngữ cảnh tự nhiên, từ lóng hoặc transcript bị nhiễu.

Về mặt hạ tầng, hệ thống có thể được mở rộng theo hướng phân phối tác vụ thông minh hơn. Các job xử lý media có thể được điều hướng theo ngôn ngữ, loại mô hình hoặc yêu cầu tài nguyên để tận dụng tốt hơn các worker và GPU khác nhau. Điều này giúp cải thiện throughput tổng thể khi hệ thống có nhiều người dùng hoặc phải xử lý nhiều video/audio cùng lúc.

Cuối cùng, hệ thống có thể tiếp tục mở rộng các tính năng phục vụ học ngoại ngữ trên ứng dụng di động. Các hướng phát triển gồm xây dựng sổ tay từ vựng thông minh hơn, bổ sung công cụ phân tích cú pháp/cấu trúc câu, gợi ý ôn tập theo ngữ cảnh và mở rộng nguồn học liệu ngoài YouTube. Khi các tính năng này được tích hợp sâu hơn với dữ liệu phụ đề song ngữ, ứng dụng có thể không chỉ dừng ở việc tạo phụ đề, mà còn hỗ trợ người dùng học và ôn tập nội dung một cách chủ động hơn.

TÀI LIỆU THAM KHẢO

- [1] Cisco, "Cisco Annual Internet Report (2018–2023) White Paper," Cisco Systems, San Jose, CA, USA, Updated Mar. 2020. [Online]. Available: <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html>. [Accessed: January 14, 2026].
- [2] K. Montero Perez, M. Peters, and E. Desmet, "Enhancing vocabulary learning through captioned video: An eye-tracking study," *The Modern Language Journal*, vol. 102, no. 1, pp. 202–221, 2018.
- [3] B. Liao, "Effects of bilingual and monolingual subtitles on incidental acquisition of multiword expressions," in *Second Language Studies*, vol. 40, no. 2, University of Hawai'i at Manoa, pp. 1–45, 2022. [Online]. Available: <https://hawaii.edu/sls/research/publications/sls-papers/>. [Accessed: January 14, 2026].
- [4] Verbit, "How Long Does It Take to Transcribe 1 Hour of Audio?," Verbit.ai, 2022. [Online]. Available: <https://verbit.ai/real-time-cart/how-long-it-really-takes-to-transcribe-accurate-audio/>. [Accessed: January 14, 2026].
- [5] GoTranscript, "Transcription Turnaround Time: How Long Does It Take?," GoTranscript Blog. [Online]. Available: <https://gotranscript.com/blog/how-long-does-it-take-to-transcribe-an-hour-of-audio>. [Accessed: January 14, 2026].
- [6] J. Smith, "The State of Automatic Speech Recognition in 2024," *Speech Technology Magazine*, vol. 29, no. 2, pp. 14–19, Feb. 2024. [Online]. Available: <https://www.speechtechmag.com/>. [Accessed: January 14, 2026].
- [7] A. Radford *et al.*, "Robust Speech Recognition via Large-Scale Weak Supervision," in *Proc. Int. Conf. Mach. Learn. (ICML)*, PMLR, 2023, pp. 28492–28518. [Online]. Available: <https://proceedings.mlr.press/v202/radford23a.html>. [Accessed: January 14, 2026].
- [8] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.
- [9] G. Tay, W. Ma, J. Lee, Y. Tang, D. Lee, W. Yin, D. Shen, S. Meng, Y. Zhu, M. Li, and A. Smola, "Back to Basics: Revisiting ASR in the Age of Voice Agents," arXiv preprint arXiv:2603.25727, 2026.
- [10] T.-H. Cheng, L. H. Clemmensen, and S. Das, "Beyond Word Error Rate: Auditing the Diversity Tax in Speech Recognition through Dataset Cartography," arXiv preprint arXiv:2603.05267, 2026.
- [11] Deepgram, "Semantic Error Rate: The Next ASR Accuracy Metric for Platform Builders," *Technical Report*, Deepgram Research, 2024.

- [12] E. Kumalija and Y. Nakamoto, "Performance evaluation of automatic speech recognition systems on integrated noise-network distorted speech," *Frontiers in Signal Processing*, vol. 2, Article 999457, 2022.
- [13] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [14] LDS Team, "The Transformer Architecture Explained," Let's Data Science, Mar. 2026. [Online]. Available: <https://letsdatascience.com/blog/attention-is-all-you-need-transformer-revolution>. [Accessed: January 14, 2026].
- [15] OpenAI, "Introducing Whisper," *OpenAI Research*, Sep. 2022. [Online]. Available: <https://openai.com/research/whisper>. [Accessed: January 14, 2026].
- [16] M. S. I. Ridoy, S. Akter, and M. A. Rahman, "Adaptability of ASR Models on Low-Resource Language: A Comparative Study of Whisper and Wav2Vec-BERT on Bangla," arXiv preprint arXiv:2507.01931, 2025.
- [17] S.-L. Yeh, Y. Meng, and H. Tang, "Whisper Has an Internal Word Aligner," arXiv preprint arXiv:2509.09987, 2025.
- [18] A. M. A. Alqadasi, A. M. Zeki, M. S. Sunar, M. S. H. Salam, R. Abdulghafor, and N. A. Khaled, "Improving Automatic Forced Alignment for Phoneme Segmentation in Quranic Recitation," *IEEE Access*, vol. 12, pp. 229–244, 2024.
- [19] S. Niveditha, P. K. Das, and J. Shafee, "Implementation and Analysis of a Hidden Markov Model Based Speech Recognition System," *International Journal of Creative Research Thoughts (IJCRT)*, vol. 13, no. 5, 2025.
- [20] H. Sakoe and S. Chiba, "Dynamic Programming Algorithm Optimization for Spoken Word Recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978.
- [21] T. Nguyen, H. Nguyen, and P. Tran, "Mixed-Level Neural Machine Translation," *Computational Intelligence and Neuroscience*, vol. 2020, Article ID 8859452, 2020.
- [22] T. H. Viet, N. V. Vinh, V. T. Huyen, and N. L. Minh, "Dependency-based Pre-ordering for English-Vietnamese Statistical Machine Translation," *VNU Journal of Science: Computer Science and Communication Engineering*, vol. 33, no. 2, pp. 14–27, 2017.
- [23] H.-V. Tran, M.-Q. Nguyen, and V.-V. Nguyen, "ViBidirectionMT-Eval: Machine Translation for Vietnamese-Chinese and Vietnamese-Lao Language Pair," arXiv preprint arXiv:2501.08621, 2025.
- [24] K. Y. Hong, L. Han, R. Batista-Navarro, and G. Nenadic, "CANTONMT: Investigating Back-Translation and Model-Switch Mechanisms for Cantonese-English Neural Machine Translation," arXiv preprint arXiv:2405.08172, 2024.

- [25] S. Cheng, P. Zhu, J. Liu, and Z. Wang, "A Survey of Grapheme-to-Phoneme Conversion Methods," *Applied Sciences*, vol. 14, no. 24, Article 11790, 2024.
- [26] Y. Shun and R. Sa, "The Application and Development of Grapheme-Phoneme Conversion," *Preprints.org*, 2025.
- [27] B. Milde, C. Schmidt, and J. Köhler, "Multitask Sequence-to-Sequence Models for Grapheme-to-Phoneme Conversion," in *Proc. Interspeech* 2017, 2017.
- [28] J. Li, Z. Zhang, M. Chen, J. Ma, S. Wang, and J. Xiao, "Improving Polyphone Disambiguation for Mandarin Chinese by Combining Mix-Pooling Strategy and Window-Based Attention," in *Proc. Interspeech* 2021, pp. 4104–4108, 2021.
- [29] G. Van Rossum and F. L. Drake, "Python 3 Reference Manual," CreateSpace, Scotts Valley, CA, USA, 2009.
- [30] S. Ramírez, "FastAPI framework, high performance, easy to learn, fast to code, ready for production," [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: January 14, 2026].
- [31] FFmpeg Developers, "FFmpeg Multimedia Framework," [Online]. Available: <https://ffmpeg.org>. [Accessed: January 14, 2026].
- [32] Mitrasish, "Self-Host Faster-Whisper on GPU Cloud: Production Deployment Guide for Real-Time ASR," *Spheron Blog*, May 2026. [Online]. Available: <https://www.spheron.network/blog/faster-whisper-gpu-cloud-production-deployment-guide/>. [Accessed: January 14, 2026].
- [33] NestJS, "NestJS - A progressive Node.js framework for building efficient, reliable and scalable server-side applications," [Online]. Available: <https://nestjs.com>. [Accessed: January 14, 2026].
- [34] PostgreSQL Global Development Group, "PostgreSQL 16.0 Documentation," PostgreSQL Documentation, 2023.
- [35] MinIO Inc., "MinIO Object Storage for Cloud-Native Applications," [Online]. Available: <https://github.com/minio/minio>. [Accessed: January 14, 2026].
- [36] J. L. Carlson, "Redis in Action," Manning Publications, Shelter Island, NY, USA, 2013.
- [37] BullMQ Authors, "BullMQ: Message queue and batch processing for NodeJS," [Online]. Available: <https://bullmq.io>. [Accessed: January 14, 2026].
- [38] Meta Open Source, "React Native: A framework for building native apps using React," [Online]. Available: <https://reactnative.dev>. [Accessed: January 14, 2026].
- [39] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, Internet Engineering Task Force (IETF), Dec. 2011.

[40] Socket.io Authors, “Socket.io: Bidirectional and low-latency communication for every platform,” [Online]. Available: <https://socket.io>. [Accessed: January 14, 2026].

[41] IEEE Computer Society, "IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications," *IEEE*, New York, NY, USA, 1998.